Demonstrate Once, Execute on Many: Kinematic Intelligence for Cross-robot Skill Transfer

Sthithpragya Gupta[1*†], Durgesh Haribhau Salunkhe[1†], Aude Billard[1]

[1]Learning Algorithms and Systems Laboratory

École Polytechnique Fédérale de Lausanne, Lausanne & 1015, Switzerland.

[*]Corresponding author. Email: sthithpragya.gupta@epfl.ch

[†]These authors contributed equally to this work.

Teaching robots new skills should be as natural as showing rather than programming. Learning from Demonstration (LfD) moves toward this goal by allowing users to guide a robot or sketch a desired motion, enabling learning without writing a line of code. Yet most LfD methods remain tied to the robot they were trained on. Changes in morphology, different link lengths, joint orientations, or limits often break the learned behaviour, making retraining unavoidable. Here we introduce a framework that endows robots with Kinematic Intelligence: an internal understanding of their own joint limits, singularities, and connectivity. Instead of correcting for these constraints after learning, we embed them directly into the control policy from the outset. The approach takes one or multiple demonstrations, extracts a globally stable dynamical system, and produces behaviours that remain valid across robots with different kinematic structures. Our method is grounded in a comprehensive analytical classification of noncuspidal three-revolute (3R) robots, which form the building blocks of many commercial robots. This classification enables a joint space policy that preserves user intent and adapts to robot-specific constraints. We validate the framework on diverse simulated and real robots, both redundant and non-redundant, with varied link geometries and joint configurations. The demonstrated skill executes safely and consistently across robots without

retuning, thereby achieving cross-robot skill transfer. Kinematic Intelligence enables skills demonstrated once to be safely executed across robots with different kinematic structures.

INTRODUCTION

As robots become more common in our daily lives, from homes and hospitals to warehouses and factories, the ability to teach them new skills quickly and safely is becoming increasingly important. Instead of relying solely on expert programming, a more natural and intuitive alternative is to let users show robots what to do. Learning from Demonstration (LfD) provides this convenience (*1–3*). It allows users, expert or not, to teach a robot by simply demonstrating the desired behaviour. Demonstrations can take many forms: physically guiding the robot's joints (kinesthetic teaching) (*4*), using a remote control (teleoperation) (*5*), or even drawing a desired path or motion in the workspace (as shown in Figure 1A). This removes the need for coding and lowers the barrier for human-robot interaction. Though LfD simplifies how we teach robots, there's a critical limitation: most of today's LfD systems are tied to the specific robot they were trained on. If a user upgrades to a new robot, perhaps one with longer arms or different motion capabilities, the same taught skill may no longer work. Ideally, transferring skills between robots should be as seamless as syncing your preferences and apps when switching to a new phone or laptop. Unfortunately, that's not the case today. The problem becomes even more difficult as new robots offer greater motion articulation, more degrees of freedom, and more complex joint configurations, each of which changes how the robot moves and what motions are feasible.

To address this challenge, we argue that robots need more than the ability to mimic demonstrations. They must possess what we call Kinematic Intelligence: an internal awareness of their kinematic constraints, joint limits, and feasible motion paths. Rather than relying on extensive demonstrations that implicitly avoid failure, our approach explicitly incorporates this structural awareness into the control policy. Kinematic intelligence enables robots to generalise behaviour across different body structures. This is important because the human body rarely matches the mechanical structure of robots, and even among robots, link lengths and joint orientations vary widely, leading to different unstable configurations (singularities). Yet the task itself (such as wiping a surface, picking up an object, or following a curved path) remains the same. To ensure that a behaviour learned on one system can be reused by others, we would typically need to reprogram or retrain the task with full knowledge of each robot's constraints. Transfer learning offers a potential solution by reusing previously learned behaviours. However, it also faces challenges related

to explainability, abstraction of transfer, and transfer metrics (*6*). Kinematic intelligence addresses this gap, allowing behaviours learned on one robot, or even from human-guided input, to execute safely and effectively on another robot without expert intervention.

In the past, mechanical intelligence was introduced as a way to build part of a robot's capability directly into its physical design, material properties, and passive mechanics (*7*). In this sense, it is conceptually related to Kinematic Intelligence, as both leverage the robot's inherent structure to support task execution. The key difference is that mechanical intelligence relies on physical embodiment, whereas Kinematic Intelligence relies on analytical properties, making it well-suited to transferring demonstrated behaviours across different robot designs.

Challenges set forth by kinematic constraints in transfer learning

Previous work in LfD has made notable progress in learning smooth and repeatable behaviours from demonstrations, particularly in simple or well-controlled environments. The methods focused on directly imitating motion trajectories in the workspace, allowing robots to reproduce tasks like reaching, drawing, or object manipulation (*8, 9*). These approaches worked well when the target robot had similar physical characteristics to the one used during training. Some probabilistic models offer flexibility but still require separate handling of kinematic feasibility (*10, 11*). Other techniques, though more integrated, depend on detailed tuning or assumptions about the robot's workspace (*12, 13*). However, when the robot changes, or when its physical structure leads to limitations near joint limits or singular configurations, the learned behaviour may become infeasible or unsafe. Some approaches attempted to solve this by abstracting the task into a higher-level representation that is independent of the robot's embodiment (*14–17*). This abstract version is then adapted for each robot individually. Although this approach shows promise, it often requires multiple demonstrations, extensive parameter tuning, or manual safety corrections near the robot's constraints (*18, 19*). Other research has explored learning mappings from workspace goals to joint configurations, commonly known as inverse models. These mappings work well in predictable areas of the robot's motion space, but they tend to fail near boundaries where movement becomes unpredictable or unstable. As a result, many methods include additional safety filters or online corrections to patch these issues in real time (*20*). Existing approaches typically emphasize task feasibility and constraints, but often treat robot-specific constraints like singularities and joint limits as post hoc considerations rather than integral components of the learning process. Though useful, these strategies treat the robot's motion constraints as a separate concern rather than as part of the learning process itself. However, due to a lack

of global understanding of the robot's constraints, using numerical tools in post hoc, can make the robot follow paths that may contain unstable configuration(s) or hit a motion limit risking task failure or even endangering human safety (*21–25*).

More recently, some data-driven methods aim to build shared task spaces or embeddings that allow robots to learn behaviours that can be reused across different platforms. These methods have demonstrated generalisation capabilities but often rely on large datasets, robot-specific fine-tuning, or access to every target robot during training [8–10]. This makes them less practical in real-world settings where only a few demonstrations are often available, as robot teaching is a time-consuming endeavour for a non-expert user. Overall, a gap remains in unifying robot-specific kinematic characteristics with generalizable learning frameworks that can function reliably with minimal demonstrations.

Many of these recent approaches are built around seven-degree-of-freedom (7-DoF) wrist-partitioned robots, which have become standard in collaborative and humanoid robotics. Their popularity stems from their structural resemblance to the human arm and their enhanced flexibility, offering redundancy that allows for more natural and adaptive motion in complex environments, by offering more degrees of freedom (seven) than strictly necessary (six). However, redundancy also poses a challenge for learning and reproducing behaviour. Unlike non-redundant robots, redundant robots (see Figure 1A) admit infinitely many joint space solutions for a given task, making it difficult to learn and generalise violation-free behaviours consistently across different configurations.

Kinematic analysis to facilitate transfer across robotic manipulators

To analyze the behaviour of 7-DoF noncuspidal robots, it is useful to decompose them into smaller kinematic units, or subchains, that isolate the functional components of motion. Most modern 7-DoF robots employ a spherical wrist, composed of three intersecting rotational joints that control the end-effector orientation. Wrist-partitioned robots, widely adopted in research and certain industrial applications such as the Kuka LWR iiwa and Motoman SIA series, exhibit noncuspidal kinematic architectures (*26*). For such robots, inverse kinematic solutions (IKS) are separated by singularity boundaries in joint space, leading to well-defined, predictable transitions between configurations. The spherical wrist not only decouples orientation singularities from the positioning subchain but is also noncuspidal, exhibiting two IKS separated by $\sin(\theta_w) = 0$, where θ_w denotes the second wrist joint. Hence, the analytical framework proposed for noncuspidal $3R$ (three revolute joints) positioning chains can be directly applied to the wrist subchain, providing certified, singularity-free orientation control. The remaining four joints form the positional

subchain, redundant for 3D positioning (see Figure 1B (i-ii)). To manage the redundancy-induced ambiguity, the $4R$ positional chain can be treated as a family of $3R$ noncuspidal subchains. By setting the angle of a redundant joint in the $4R$ subchain to a specific value, we effectively eliminate one degree of freedom, reducing the $4R$ subchain to a non-redundant $3R$ subchain that is just sufficient for spatial positioning. Thus, the behaviour of the $4R$ chain can be studied by analyzing the properties of its constituent $3R$ subchains, where the redundant joint serves as a parameter indexing different members of the family. In this way, the entire $4R$ positional chain can be interpreted as a family of $3R$ subchains, each corresponding to a different value of the redundant joint angle. This creates a parameterized representation, where the value of the redundant joint angle acts as a parameter that indexes a continuum of $3R$ subchains embedded within the original $4R$ subchain. Each of these $3R$ subchains has the same joint limit constraints but exhibits different kinematic properties, such as singularities, number of IKS, and the feasible joint space regions.

This decomposition provides a systematic means to analyze the core positional capabilities of 7-DoF wrist-partitioned robots through a family of noncuspidal $3R$ subchains. These $3R$ units constitute the kinematic foundation of the robot and form the basis for our global, constraint-aware analysis. Since the wrist joint angles depend functionally on the configuration of the positional subchain and can be computed reactively without ambiguity, the positional feasibility results obtained from these subchains can be directly extended to full end-effector pose control.

Kinematic-aware learning of control policies

Following the classification proposed by Burdick (*27*), $3R$ robots can be grouped as generic, non-generic, or degenerate according to the nature of their singularities. Generic robots exhibit well-behaved singular surfaces whose topology remains stable under small variations of link parameters. Non-generic robots correspond to special geometries where these surfaces intersect or bifurcate, leading to multiple branches or self-intersections. Degenerate cases arise typically in robots with symmetric link lengths or right-angle joint arrangements. In such configurations, two joint axes may align or intersect, reducing the robot's effective mobility and leading to indeterminate motions if not explicitly modeled. Recognizing these degeneracies is therefore crucial: when known, they can be avoided for safety or exploited to improve task dexterity. Our framework systematically incorporates this awareness, embedding Kinematic Intelligence across all categories of noncuspidal robots, generic, non-generic, and degenerate, thus ensuring constraint-aware and predictable behaviour regardless of design variations.

We design safe and generalizable control strategies that are not tailored to a specific robot. Safety refers

here to control strategies that guarantee violation-free task execution by embedding certified redirection mechanisms near kinematic constraints. To this end, instead of trying to remove or ignore a robot's physical constraints as done in many LfD approaches, we embed these constraints analytically into the policy architecture. By doing so, we ensure that the learned behaviour is automatically adapted to each robot's structure in a safe and stable way (see Figure 1C (i-iii), and Figure 1D (i-iii)). Specifically, we use a topological and differential classification of singularities to understand how different parts of the robot's configuration space are separated by constraints (see Figure 2). This classification allows us to identify singularity-free zones, distinguish between reachable and unreachable regions, and design joint space control policies that avoid constraint violations.

With such kinematic-aware embedding of the learned policies, even a single demonstration is enough to create a behaviour that is safe, robust, and executable on many different robots. We demonstrate this framework across a range of robots, including both redundant and non-redundant robots, each with different kinematic structures and singularity patterns. This is achieved without any retraining or robot-specific reprogramming.

RESULTS

We demonstrated transfer of skills in simulation with a series of non-redundant $3R$ robots. For redundant robots, we considered the KUKA IIWA LWR 7 and the Neura Maira M robot, both seven-degree-of-freedom commercially available robots. The Maira robot had a peculiarity in that it had a degeneracy, but the degenerate configurations remained outside the feasible joint space. We observed across all robot configurations that the system successfully acquired a generalised policy of the user behaviour, resulting in smooth, stable, and physically feasible robot execution (see Figure 3, and Figure 4). Even with varying robot structures and joint constraints, the embedded Kinematic Intelligence ensured that the task execution remained safe and accurate, demonstrating the framework's ability to generalise a skill across different robot bodies even from a single user demonstration. We further demonstrated redundancy parametrisation and how it facilitated behaviour transfer to redundant robots (see Figure 5).

Kinematic intelligence

Kinematic intelligence refers to the embedding of analytical kinematic properties into the control framework using deterministic, fixed-compute algorithms. This enabled certified, constraint-compliant path planning and policy synthesis that generalised across robot embodiments without retraining. A central result of this

work was a theoretically grounded classification of all noncuspidal $3R$ robots into six distinct categories, each defined by the global structure of their kinematic constraints. This classification formed the backbone of our Kinematic Intelligence framework, enabling the synthesis of robot-specific control strategies that were inherently safe and generalizable. The classification is based on the algebraic and topological analysis of the robot's Jacobian determinant. The determinant's factors and its root structure gets determined by the robot's design parameters, such as link lengths and joint offsets, which define the location and shape of the locus of singular configurations, singularities, in the joint space. These singularities, together with joint limits, partition the joint space into distinct feasible regions, referred to in the remainder of this paper as aspects (*28*). Each category captured a specific class of kinematic behaviours, characterized by how singularities folded, looped, and intersected to segment the joint space. Crucially, the structure of each class also dictated a corresponding near-boundary control strategy. Rather than treating constraint handling as an afterthought, the categorization informed the embedding of corrective actions directly into the control policy. Once a robot's class was known, the corresponding strategy could be instantiated without further tuning or demonstration, yielding a scalable method for constraint-compliant behaviour transfer across robot types. Figure 2 provided an overview of the topological structure of the singularities and joint limits in each class. This structure was not just descriptive; it was operational. It enabled one to build controllers that adapted dynamically to robot-specific constraints and preserved the fidelity of the demonstrated task.

Robot Categorization

We established a classification scheme for noncuspidal $3R$ robots through algebraic analysis of the Jacobian determinant, $\det \mathbf{J}(\mathbf{q})$, where $\mathbf{q}$ represents the joint configuration. Singularities depended only on the second and third joints (q_2 and q_3), allowing analysis in a reduced space defined by their angles, referred to as the q_2-q_3 slice of the joint space. The determinant $\det \mathbf{J}(\mathbf{q})$ can have up to three irreducible factors whose zero sets defined closed branches on the torus $\mathbb{T}^2$ generated by $q_2 - q_3$. Each branch's winding numbers (n_1, n_2) indicated how many times it wrapped around the torus generators.

Existing homotopy-based classifications (*29, 30*) characterized singularities by their branches' winding numbers. Though exhaustive, this classification does not inherently support constrained motion planning. Consequently, robots within the same homotopy category may exhibit different differential properties affecting constraint-compliant motion planning. Additionally, this classification is limited to generic robots only. To address these limitations, we introduced a symbolic notation $(n_1, n_2)[n_3, n_4]$ that retained the global winding behaviour and captured the branch's differential structure. Here, n_3 and n_4 denoted the

number of horizontal and vertical turning points on a factor branch. By incorporating both topological and differential properties, this categorization ensured that robots within the same class shared identical constraint structures and allowed a unified path planning strategy near constraints.

Applying this framework to generic and non-generic noncuspidal $3R$ robots, we identified six canonical categories describing the global structure of the joint space. The six categories shown in Figure 2B were: (I) non-loop, non-intersecting factors with only horizontal turning points—branches of type $(1, 0)[\geq 2, 0]$; (II) non-loop, non-intersecting branches with no horizontal turning points—branches of type $(0, 1)[0, > 0]$ or $(1, 1)[0, \geq 0]$; (III) non-loop intersecting branches—two or more factors with intersecting non-loop branches; (IV) non-loop, non-intersecting branches with vertical turning points—branch type $(1, 0)[2, > 0]$; (V) loop, non-intersecting branches—branch type $(0, 0)[2, \geq 2]$; and (VI) intersecting branches with a loop—intersecting factors where one branch formed a loop. Each class reflected a distinct constraint-induced partitioning of the joint space and corresponded to different boundary-following behaviours. This classification organized singularities across $3R$ robots and provided a basis for designing constraint-aware control policies that generalised across robot geometries. Examples of noncuspidal robots with varying Denavit–Hartenberg (D-H) parameters are presented in Movie S1.

When a user demonstrated a task, the behaviour was encoded as a task-level objective that each robot executed in a constraint-aware manner based on its categorisation. In contrast to traditional LfD methods that rely on post-hoc corrections or multiple robot-specific demonstrations, the proposed framework generalised from a single demonstration by using an internally encoded model of joint space structure that was robot-specific yet reusable across all robot categories. For example, if executing a demonstrated behaviour on a new robot would cause it to approach a singular configuration or violate a joint limit, the system automatically adjusted the trajectory to follow the boundary of the feasible region until it safely returned to the nominal path.

In essence, the structural classification functioned as a map that each robot used to safely interpret and execute a shared task. By integrating this into the learning process, we moved beyond treating kinematic constraints as external concerns and instead made them an intrinsic part of behaviour generalisation. Details of the robot categorization framework, its properties, joint space topology, constraints, and algorithms are provided in Supplementary Methods section Robot Categorisation.

Near-constraint strategy

A key outcome of this work was the establishment of a one-to-one correspondence between the robot's

kinematic category and the control strategy that governs robot behaviour along aspect boundaries. Each aspect is bounded on at least one side by a factor of $\det \mathbf{J}(\mathbf{q})$, and on the remaining sides (if any) by the joint limits. Traversal along joint limits is straightforward because they form a linear variety in joint space, but motion along the factor boundary is nontrivial and varies markedly across categories. The core insight was that the traversal along the factor boundary within an aspect dictated the constraint-aware motion strategy, and was uniquely determined by the robot's category. Each category is characterised by a key boundary-aligned direction(s), vector(s) intrinsic to the structure of the singularity factor, that governs motion near the constraint (see Figure 2C). This direction is projected onto the tangent of the boundary to yield the feasible motion along it. This approach avoided motion stalling along the singularity branch, even in presence of curvature reversals.

Robots from Categories I, II, IV, and V exhibited a single boundary-aligned direction: horizontal (along q_2) in Category I, vertical (along q_3) in Category II, horizontal motion with intermittent vertical switching in Category IV, and motion along a closed loop in Category V (see Figure 2C). In contrast, Categories III and VI were defined by two boundary-aligned directions arising from two distinct factors in $\det \mathbf{J}(\mathbf{q})$, corresponding to intersecting singularities. Each factor contributed its own traversal direction. This differed from Category IV, where only one factor existed despite mixed motion, as horizontal and vertical behaviours occurred along the same branch. Multiple factors also did not necessarily imply multiple directions: Category I robots could have two factors, but since their branch types were identical ($(1, 0)[\geq 2, 0]$), the horizontal direction sufficed.

We referred to the structured control strategies incorporating these category-specific boundary-aligned directions for governing the robot behaviour near constraints as track cycles. When the nominal dynamics, modelled from user demonstration(s), generated a trajectory $\mathbf{Q}$ that exited the current aspect, convergence to the goal required eventual reentry. The track cycle defined a stable control policy active between the first exit and last reentry, redirecting motion along the aspect boundaries. Overall, the track cycle captured the sequence of boundary transitions that guided the robot from any boundary configuration back to a safe configuration consistent with the original trajectory $\mathbf{Q}$. We adopted a hybrid control framework that overlaid the track cycle policy onto the nominal dynamics. When the robot approached safety margins around singularities or joint limits, the corresponding corrective policies were activated promptly. Further details on factors, their properties, and track cycles are provided in the section Behaviour near Kinematic Constraints in Supplementary Methods.

Transfer Across Robots

To evaluate cross-robot behaviour transfer, we considered a scenario where a user demonstrated tasks such as drawing shapes or letters, hitting, rearranging, or throwing objects, which different robots attempted to reproduce. These robots varied in mechanical design: some were non-redundant (with just enough degrees of freedom), and others were redundant (with more joints than necessary). The framework generated robot-specific joint-space control policies that preserved the demonstrated intent and ensured safety. The results validated the core hypothesis: embedding structural constraints into the learning framework allowed a behaviour demonstrated once to transfer safely across robots with different kinematics without expert tuning or additional data.

Transfer to Non-redundant robots

To evaluate how a single demonstration could be extended across multiple non-redundant robots with different kinematic structures, we tested our framework on four $3R$ robots - one each from Categories I through IV. For each robot, the goal was to generate a control policy that generalised user-demonstrated motion and respected robot-specific constraints like singularities and joint limits. As shown in Figure 3, our method models joint space behaviour aspect-wise, where each aspect is a region free of singularities, and incrementally builds a constraint-aware policy. The process is presented for the four aforementioned robots in Figure 3B-E. The figure outlines each step of the framework: from modelling the workspace behaviour (Figure 3 Step (i)), learning aspect-specific joint space behaviours (Figure 3 Step (ii)), identifying feasible transfer trajectories (Figure 3 Step (iii)), and workspace-to-joint space transfer and updating the policy based on most novel transfer trajectories (Figure 3 Step (iv)). Finally, Figure 3 Step (v) formulated the resulting workspace motion that was faithful to the original demonstration and stayed entirely within safe, constraint-compliant regions. Across all tested robot categories, the learned joint space policy remained free from constraint violations without requiring retraining, enabling robust and scalable behaviour transfer to diverse non-redundant robots from just a single demonstration.

Transfer to Redundant robots

To assess transfer to redundant robots, we considered 7 DoF wrist-partitioned robots. The user-demonstrated end-effector trajectory generated a control policy for position control of these redundant robots. Redundant robots can be parameterized by an arm angle q_R, representing a family of non-redundant $6R$ robots whose architecture depends on this angle (*31, 32*). Prior work showed that $\det \mathbf{J}(\mathbf{q})$ of redundant robots can be

expressed as a function of q_R (*26*), implying that the redundant robot can be treated as a non-redundant robot at any arbitrary value of q_R. In practice, we discretised candidate q_R values, performed classification and connectivity analysis for the induced $3R$ subchain, and selected a fixed q_R that realized the task within a single feasible aspect without joint-limit or singularity violation. The q_R value remained constant during execution, so that, from the viewpoint of the positional chain, the robot behaved like a non-redundant $3R$ robot and no discontinuities due to online q_R switching were introduced.

Figure 5 outlines redundancy parametrisation into five behavioural modes. The redundant $4R$ positional subchain was identified and parameterised using $q_R = 0, \pi/6, \pi/4, \pi/3$, and $\pi/2$ radians (Figure 5A). All five of these $3R$ robots had non-loop intersecting singularity branches with types $(1, 0)[\infty, 0]$ and $(1, 1)[0, \geq 0]$ when $q_R = 0$, and $(1, 0)[\infty, 0]$ and $(0, 1)[0, > 0]$ otherwise. Figure 5B shows the resulting joint-space constraints. At each q_R, multiple IKS existed for the starting configuration. Some of these configurations were feasible for task completion, as shown in Figure 5B, and produced workspace behaviour in Figure 5C. However, some of the other IKS (silver) were not feasible. For $q_R = 0$ and $\pi/3$, feasible behaviours were not detected because the goal configuration lay on the joint-limit boundary. The feasible workspace bounded by singularities varied with q_R (Figure 5C–D). For different values of q_R, the procedure yielded a set of behavioural modes, each encoding the demonstrated behaviour and enabling selective retrieval depending on the task constraints. This modular arrangement preserved the safety and robustness properties of each individual behaviour mode, thereby providing a violation-free control policy for wrist-partitioned redundant robots.

Certified Execution

To model the robot control policy from joint trajectories, each trajectory was represented as a globally stable dynamical system in a one-shot manner (*33*). The resulting policy, composed of such systems, was resilient to temporal and spatial perturbations. Small deviations were corrected passively, and the larger displacements such as those that moved the robot into a different joint space region, triggered a topology-aware feasibility analysis to determine whether the robot's current aspect admitted a constraint-compliant trajectory to the demonstrated goal. Because not all aspects admitted a valid goal configuration, we developed certified factor-specific connectivity. Five factor types arose from their branch types: $(1, 0)[\geq 2, 0]$, $(0, 1)[0, > 0]$, $(1, 0)[2, > 0]$, $(0, 0)[2, \geq 2]$, and $(1, 1)[0, \geq 0]$ (see Supplementary Methods section Robot Categorisation). Each irreducible factor of $\det \mathbf{J}$ partitioned the joint space into multiple regions, and feasibility required the start and goal configurations to lie in the same partition for every factor. This

incorporated the global topological-differential structure of the robot's kinematic space into the control logic and enabled fast, deterministic connectivity analysis.

To leverage the certified kinematic properties in learning the demonstrated behaviour and formulating a control policy, the framework undertook a multi-step process summarised in Figure 6, which depicts the implementation pipeline for executing a learned policy on a single robot. For each $3R$ robot at a fixed value of q_R, we first grouped the offline, one-time pre-processing steps used to determine the robot category as shown in Figure 6A (see Supplementary Material, section Robot Categorisation). Figure 6B corresponds to the planning stage, where we learned the aspect-specific nominal joint behaviour (see the Supplementary Material, section Modelling the User Behaviour). The resulting policy governed the reactive control loop shown in Figure 6C, which combined connectivity checks with the attractor, constraint-violation monitoring, nominal policy execution, and track-cycle computations to ensure faithful replication of the demonstrated behaviour and respect kinematic constraints. The wall-clock timings and latency induced by each step are reported in Figure 6D; these timings showed that pre-processing was negligible relative to demonstration time and that both planning and reactive control incurred only modest computational overhead, making the certified execution layer compatible with real-time control on the tested platforms.

Figure 7 illustrates certified execution and its aspect-based failure modes on a robot from Category I with regions of two and four IKS in the workspace. From any chosen IKS, the controller executed the demonstration only if a feasible aspect connected the start and goal configurations; otherwise, it halted safely. For non-redundant 6R robots, failures occurred when no admissible path existed, or when perturbations drove the state into joint-limit or singularity margins, triggering track cycles or a safe stop. The same logic extended to wrist-partitioned 7R robots (Figure 5), where tasks were declared infeasible if no redundant angle q_R yielded a feasible aspect. A detailed discussion of the failure modes is presented in the Supplementary Materials, section Robustness.

Experimental evaluation of "Demonstrate once, execute on many" principle

We experimentally validated the "demonstrate once, execute on many" principle on three industrial robots. These included a compact 6-DoF Duatic DynaArm with tighter joint limits, a 7-DoF KUKA LWR IIWA 7 with a wrist-partitioned architecture and moderate limits, and a 7-DoF Neura Robotics Maira M with longer links and more relaxed limits, yielding markedly different workspaces, proximity to limits, and feasible aspect layouts for the same end-effector task.

In Experiment 1, a human demonstrated a path resembling the letters of "SCIENCE" once per letter

using a motion-capture glove and OptiTrack. The recorded trajectories were encoded as dynamical-system-based policies and replayed on the two 7-DoF robots, which reproduced the characters without joint-limit or singularity violations across multiple aspects and redundant angles (refer to Movies S2, S3). In Experiment 2, we constructed a mock multi-robot assembly line with three skills—pushing, pick-and-place, and throwing. Each skill was demonstrated once, encoded once as a joint space policy, and executed on three robots without retraining. Pushing and throwing drove the joints toward workspace boundaries and induced large excursions, whereas pick-and-place required aspect queries combined with connectivity checks. The same policy was reused; only the kinematic embedding and certified execution layer changed, keeping trajectories within feasible aspects and redirecting them along analytic track cycles near constraints. Figure 4 shows the three robots performing the three skills; full executions appear in Movie S4.

DISCUSSION

Teaching robots through demonstration is an effective way for humans to communicate complex skills. As robotic platforms diversify, sharing learned behaviours across systems without reprogramming becomes increasingly important. This is the goal of transfer learning from demonstration: to teach once and generalise broadly. We propose integrating behaviour learning with robot-specific constraint enforcement by embedding analytic kinematic knowledge directly into the control policy. This enables proactive avoidance of infeasible configurations without post-hoc checks. Our framework advances Learning from Demonstration by enabling safe and generalizable skill transfer across robot architectures through Kinematic Intelligence, separating user intent from robot morphology. Each demonstrated trajectory is modelled as a globally asymptotically stable dynamical system, ensuring convergence to its target configuration. This stability at the most fundamental level yields robustness to temporal and spatial perturbations, without requiring reactive correction. As the control policy is incrementally built by adding more demonstrations, the stability of each dynamical system ensures that the entire policy remains stable at every stage of learning.

Although behaviour is demonstrated in workspace, it is ultimately executed in joint space. Reliable replication therefore requires preserving user intent in joint space accounting for robot-specific constraints. Our framework achieves this through a topological-differential classification of singularities to categorise robot types, followed by connectivity analysis to assess feasibility and finally a constraint-aware policy synthesis. Control policies are encoded within joint-space aspects, ensuring trajectories remain within feasible boundaries. If a trajectory approaches a constraint, a near-boundary strategy redirects motion along a safe path called the track cycle and rejoins the original trajectory once clear. These category-specific

strategies are embedded into the control logic, enabling certifiable behaviour through bounded-complexity feasibility checks. All connectivity and constraint handling methods are derived analytically from the robot's structure, ensuring that motion is always grounded in the robot's capabilities. This analysis also extends to redundant robots: it executes successfully for all redundant angles in feasible aspects, and correctly halts when a configuration becomes infeasible. The framework also supports incremental generalisation of demonstrated behaviour. At any instance, the already-learnt behaviour is used to generate feasible joint space trajectories, which are then ranked by a novelty metric based on how much new information they contribute, incrementally aligning the generalised behaviour more closely with the user intent. Experiments validate the robustness of this approach on robots with different kinematics. Our framework synthesizes robot-specific joint space controllers in a transparent and explainable manner, establishing user trust by making robot behaviour predictable and legible (*34, 35*).

Although many recent collaborative and industrial robots are cuspidal (such as ABB GoFa, Fanuc CRX, Kinova Link 6) or use non-spherical wrists (for instance UR5), the wrist-partitioned architecture remains widely adopted (including Kuka LWR, Neura Maira, DynaArm) (*26*). Studying wrist-partitioned noncuspidal cases therefore provides a foundation for certified, singularity-aware control policies, as it provides a clear separation of inverse kinematic solutions allowing rigorous analysis. In contrast, cuspidal robots have multiple inverse kinematic solutions in an aspect, introducing ambiguity in control policies due to non-singular change of solutions (*23, 36*). We therefore focused on noncuspidal architectures to establish safety and predictability, with the extension to cuspidal architectures as the next step. Recent results show that all 3R robots admit reduced aspects (*37*), which subdivide joint space even when multiple IKS exist within an aspect. These reduced aspects, together with admissible path types in cuspidal robots (*22*), could support generalised connectivity checks and track-cycle strategies. In principle, collision handling could also be incorporated by treating collision manifolds as additional joint-space boundaries within the track-cycle framework, though developing and validating such collision-aware, cuspidal-ready extensions remains future work.

In summary, our framework provides a constraint-aware foundation for transferable robot learning. Kinematic Intelligence in the learning pipeline supports scalable, and safe deployment across diverse platforms, bringing us closer to seamless integration of robotics in everyday life.

MATERIALS AND METHODS

Overview

This research aimed to develop a framework for transferring demonstrated behaviour across a class of $3R$ robots and redundant $4R$ robots, laying the foundation of behaviour transfer across wrist-partitioned $6R$ and $7R$ robots. The framework first modelled the user behaviour agnostic to the robot, and then autonomously embedded the behaviour model with Kinematic Intelligence, resulting in a control policy that oversaw safe and violation-free execution on a non-redundant robot. A modular arrangement of such non-redundant robot control policies, when composed together, enabled behaviour execution on redundant robots. An overview of the challenges, a brief motivation to kinematic analysis, and behaviour transfer is presented in Movie S5.

Behaviour modelling from demonstration

We began by modelling the user-demonstrated behaviour in both the workspace $\mathcal{X}$ and joint space $\mathcal{Q}$ of the robot (see Figure 8A-C). Each user demonstration (trajectories in $\mathcal{X}$) and its inverse-kinematics mapping in $\mathcal{Q}$ were individually embedded as globally asymptotically stable dynamical systems: $\dot{\mathbf{x}} = \mathbf{g}(\mathbf{x})$ for the workspace trajectory, and $\dot{\mathbf{q}} = \mathbf{g}(\mathbf{q})$ for the joint trajectory. The dynamical system formulation ensured that a trajectory from any starting point in the workspace or joint space converged to the demonstrated goal over time. Each dynamical system was constructed in a latent space $\mathcal{U}$, a transformed coordinate system obtained via a bijective mapping $\psi : \mathcal{U} \to \mathcal{Z}$, where $\mathcal{Z}$ was either $\mathcal{X}$ or $\mathcal{Q}$. In latent space, the dynamics became (quasi)linear, simplifying stability guarantees. The learned dynamics in $\mathcal{U}$ were then pulled back to the original space via the inverse mapping, ψ^{-1}, producing non-linear but stable dynamical systems in both spaces, $\mathcal{X}$ and $\mathcal{Q}$.

To generalise across multiple demonstrations, all joint space trajectories that terminated in the same aspect $\mathcal{A}_i$ were grouped together (see Figure 8C). Each resulting dynamical-system embedding $\mathbf{g}_j^{(\mathcal{Q})}$ within this aspect was associated with a localized region of influence in aspect $\mathcal{A}_i$ (see Figure 8C), defined as the set of joint configurations where $\mathbf{g}_j^{(\mathcal{Q})}$ governed the motion. That is, for any $\mathbf{q}$ in this region, the robot's behaviour was given by $\dot{\mathbf{q}} = \mathbf{g}_j^{(\mathcal{Q})}(\mathbf{q})$. An analogous partitioning was performed in the workspace when multiple demonstrations were available. The workspace partitioning spanned the entire domain, in contrast to the joint space, where the partitioning was aspect-specific.

These regions were defined via cones of influence (see Figure 8D) in the latent space:

$$\mathcal{L}_j = \left\{ \mathbf{u} \in \mathcal{U} \,\middle|\, \angle(\overset{*}{\mathbf{u}} - \mathbf{u}, \mathbf{v}_j) \leq \lambda_j \right\}, \quad \mathcal{R}_j = \psi_j(\mathcal{L}_j) \tag{1}$$

Here, $\mathcal{L}_j$ is the cone of influence of control policy $\mathbf{g}_j^{(Q)}(\mathbf{q})$ in the latent space indexed by j. $\mathbf{u}$ refers to a general point in the latent space $\mathcal{U}$, and $\overset{*}{\mathbf{u}} = \mathbf{1}_n^T$ is the latent embedding of the goal, where $\mathbf{1}_n^T$ represents an $n-$dimensional column vector denoting a latent space point with all coordinates = 1 and T states transpose to indicate that this is a column vector. $\mathbf{v}_j$ is the vector from the start to goal state. $\mathcal{R}_j$ is the region of the joint space corresponding to $\mathcal{L}_j$. The angle λ_j controls the size of the neighbourhood influenced by the dynamical system. At inference time, the control law is given by:

$$\dot{\mathbf{q}} = \mathbf{g}_j^{(Q)}(\mathbf{q}), \quad \text{where } j = \arg\max \left\{ P(\mathcal{R}_j) \,\middle|\, \mathbf{q} \in \mathcal{R}_j \right\} \tag{2}$$

Priority $P(\mathcal{R}_j)$ ensured disambiguation when regions overlapped. To prevent high-priority demonstrations from overriding lower-priority behaviours, the opening angle λ_j was adjusted using projection checks:

$$\lambda_j = \min \left\{ \angle \left(\psi_j^{-1}(\mathbf{q}_l) - \mathbf{1}_n^T,\ \mathbf{v}_j \right) \right\}_{\mathbf{q}_l \in \mathbf{Q}_l} \tag{3}$$

Here, $\mathbf{Q}_l$ is the trajectory associated with $\mathrm{g}_l^{(Q)}(\mathbf{q})$, $1 \leq l < j$. By combining such aspect-specific behaviours, we constructed the full joint space policy. The process of modelling the joint space policy from multiple workspace demonstrations for the case in Figure 8 is presented in Movie S6. For full derivations and algorithmic procedures, see the "Modelling the Robot Control Policy" section in Supplementary Materials and Methods.

Embedding Kinematic Intelligence

To generalise user-demonstrated behaviours and ensure safe execution, we embedded Kinematic Intelligence into the robot's control policy. Up to this point, we had modelled the behaviour in both workspace and joint space. Either of these models can act as the control policy. Directly embedding constraints in the workspace model is challenging due to complex singularity features such as nodes and cusps, and non-linear boundaries formed by joint limits. Moreover, the workspace lacks dimensional homogeneity in position and orientation, complicating violation-free control. Conversely, joint space singularities form continuous, differentiable C^∞ functions, and joint limits define a linear subspace, making joint space more suitable for embedding constraints and preserving user behaviour. We thus adopted the joint space model as the primary control policy, enriched via: singularity-based robot categorisation and incremental generalisation that transfers behavioural variability from workspace to joint space.

We introduced an exhaustive categorisation of generic and non-generic noncuspidal $3R$ robots into six categories. This revealed how motion becomes constrained, enabling targeted strategies for stable, feasible behaviour around constraints. We analysed the joint space singularities, classifying them by loop structure and turning points to design locally stable, constraint-aware strategies. Categories differed by whether branches looped, intersected, or had folds, dictating directional constraints. Further details are in Supplementary Methods: Robot Categorisation. If the joint space trajectory risked violating limits or singularities, we activated a near-boundary control policy to keep motion within the same aspect, bounded by joint limits and singularities, tailored per robot based on its singularity layout.

We defined safety margins near aspect boundaries and continuously monitored motion. If a violation was imminent, we switched to the near-boundary policy, which temporarily deviated to follow along constraint-safe paths, termed the track cycle, before resuming nominal policy. Near-constraint strategies for all six categories are in Supplementary Methods: Behaviour near Kinematic Constraints. The framework integrated these strategies with deterministic connectivity checks of fixed computational complexity, ensuring stable and safe motion near constraints. These checks verified that the configuration was in a feasible aspect and could reach the goal without violating it; otherwise, execution halted, preventing unsafe behaviour. Details for all singularities in noncuspidal $3R$ robots are in Supplementary Methods: Certified Execution. Kinematic Intelligence made the framework robot-agnostic during learning, but robot-specific during execution, allowing robots to safely and faithfully replicate user intent, even near constraints.

Workspace to jointspace transfer for enhanced generalisation

To improve generalisation of the joint space control policy, we introduced an incremental workspace-to-joint space transfer mechanism. This began by uniformly sampling the joint space $\mathcal{Q}$ to generate a grid of candidate joint configurations $\{\mathbf{Q}\}^{\text{sample}}$, which were then mapped to corresponding workspace positions $\{\mathbf{x}\}^{\text{sample}}$ using forward kinematics. For each $\mathbf{x}_i \in \{\mathbf{x}\}^{\text{sample}}$, a transfer trajectory $\mathbf{X}_i^{\text{transfer}}$ was generated using the workspace dynamical system model $\mathrm{B}^{(\mathcal{X})}$ such that:

$$\mathbf{X}_i^{\text{transfer}} = \left\{ \mathbf{x}_t \;\middle|\; \mathbf{x}_{t=0} = \mathbf{x}_i,\ \lim_{t\to\infty} \mathbf{x}_t = \overset{*}{\mathbf{x}} \right\} \tag{4}$$

These transfer trajectories were then mapped back into joint space, yielding a set $\mathbf{Q}^{\text{transfer}}$ of candidate joint space trajectories. Each trajectory was validated and trimmed to remove unsafe or constraint-violating segments using certified constraint checks. To ensure efficient expansion of the joint space model $\mathrm{B}^{(\mathcal{Q})}$,

we introduced a novelty metric that prioritized trajectories contributing the most new information. The novelty of a trajectory was evaluated by computing its angular deviation in latent space from existing demonstrations. Let $\psi_j : \mathcal{U} \to \mathcal{Q}$ denote the embedding function of the j^{th} learned dynamical system, and let $\mathbf{q}_t$ be a point on a transfer trajectory $\mathbf{Q}^{\text{transfer}}$. The angular deviation was given by:

$$\Theta_j(\mathbf{Q}^{\text{transfer}}) = \left\{ \angle \left(\psi_j^{-1}(\mathbf{q}_t) - \mathbf{1}^\top, \mathbf{1}^\top - \mathbf{0}^\top \right) \right\}_{t=1}^{|\mathbf{Q}^{\text{transfer}}|} \tag{5}$$

For an aspect $\mathcal{A}_i$, let $\mathrm{B}^{(\mathcal{A}_i)}$ be the corresponding aspect-specific behaviour. The novelty score of a trajectory is defined as:

$$\text{novelty}(\mathbf{Q}^{\text{transfer}}) = \min_j \left\{ \Theta_j(\mathbf{Q}^{\text{transfer}}) \;\middle|\; \psi_j \in \mathrm{B}^{(\mathcal{A}_i)} \right\} \tag{6}$$

The most novel trajectory was selected as:

$$\mathbf{Q}^{\text{transfer}}_{\text{novel}} = \underset{\mathbf{Q}_i^{\text{transfer}} \in \{\mathbf{Q}\}^{\text{transfer}}}{\arg\max} \; \text{novelty}(\mathbf{Q}_i^{\text{transfer}}) \tag{7}$$

This most novel trajectory was then used to update the aspect-specific model $\mathrm{B}^{(\mathcal{A}_i)}$, and by extension, the overall joint space behaviour $\mathrm{B}^{(\mathcal{Q})}$. This ensured that the control policy became richer over time without compromising constraint adherence, stability, or model compactness. Further technical details, including latent space construction and trajectory validation steps, are provided in the Supplementary Methods section titled Incremental Update by Workspace-to-Joint Space Transfer.

Controlling redundant robots

In robots with redundant degrees of freedom, those exceeding the minimal requirement to define end-effector poses, internal configurations can vary without affecting the external task. This inherent redundancy, although advantageous for optimising secondary criteria (such as obstacle avoidance, joint limits, or ergonomics), introduces challenges for characterising and generalising robot behaviour. Using Kinematic Intelligence, we addressed this by parameterising the redundant joint angle, denoted q_R, and reducing the complete $7R$ robot to a continuum of simpler, equivalent $3R$ chains, similar to one reported in (*26*). Each value of q_R yielded a distinct robot architecture with well-defined kinematic properties, particularly concerning singularities and solution multiplicity.

This dimensionality reduction enabled a structured encoding of robot behaviour as the redundant angle served as an intrinsic coordinate along which motion solutions could be organised. For instance, motion

trajectories executed at different q_R values corresponded to distinct internal configurations achieving the same external task. These configurations could be interpreted as behavioural modes, with q_R acting as a modulating variable. In noncuspidal robots, where inverse kinematic solutions remained separated by singularities for each q_R, this parameterisation ensured that the operation mode was consistent and predictable.

This modular arrangement of behaviours enabled redundancy resolution and generalisation of behaviour across platforms. Crucially, it provided a modular, explainable framework for managing the solution null space of redundant robots, integrating geometric insight with practical utility in control and learning. For applications demanding safety, repeatability, or human interaction, such an interpretable structure was essential for ensuring trustworthy robot behaviour. A detailed discussion of why classical numerical null-space strategies are insufficient in this context is provided in the Supplementary Materials, section Issues with numerical path-planning strategies.

STATISTICAL ANALYSIS

Quantitative evaluation focused on the computational performance of the execution pipeline and the learning statistics of the demonstrated behaviour (Figure 6). Wall-clock execution times were recorded for each stage and aggregated across robot instances. Unless otherwise stated, statistics were computed over 21,787 noncuspidal $3R$ robots. The per-robot performance metrics (Figure 6) were measured and summarized using the mean (μ), standard deviation (σ), 95th percentile, minimum, and maximum.

For preprocessing steps (A (i-ii)), execution time was measured once per robot. Reactive-control components were evaluated on a grid of 100 joint configurations per robot, obtained by uniformly discretising the q_2 and q_3 axes. Connectivity checks (C (i)) measured the time to determine connectivity between each grid point and a fixed query configuration $[0, 0.5, 0.1]$ radians, with the mean across queries defining per-robot performance. Violation checks (C (ii)) measured the time to detect kinematic constraint violations. Track-cycle computation (C (iv)) and velocity output (C (v)) were evaluated using synthetic piecewise trajectories (that violated aspect boundaries) generated from the sampled configurations, with the mean across 100 instances defining per-robot performance.

Learning-related metrics for planning stage (B), oneshot sampling (C (iii)), nominal velocity output (C (vi)), model training duration (D (i)), and trajectory error (D (ii)) were evaluated on a subset of 330 robots. Seven demonstrations of the `Multi_Models_1` character from the LASA Handwritten Dataset (*38*) were used. Model training duration (D (i)) was recorded during control-policy learning. For the other metrics,

inverse kinematic solutions for the demonstration start points were, yielding up to 28 starting configurations across feasible aspects. Each configuration generated a joint-space trajectory using the learned policy. Per-robot performance was computed as the mean execution time and trajectory error across these queries. Trajectory error was measured using dynamic time warping (DTW) between reproduced and demonstration joint-space trajectories.

Acknowledgments

We thank Dimitri Jacquemont for their invaluable assistance in setting up and executing the robot experiments, and Natacha Jeannot for generously providing the demonstration data used in this work. We are also grateful to Isabelle Derivaz-Rabii for the essential administrative support during the project. We also used ChatGPT to assist with revising and refining the language in this paper. **Funding:** This work was supported by EU project DARKO under Grant H2020 ICT-46-2020, and Horizon Europe under Grant 101070596, euROBIN. **Author contributions:** Conceptualization, simulation, investigation, analysis, visualization, and writing: S.G, D.H.S. Resources, supervision, writing, and revision: A.B. **Competing interests:** There are no competing interests to declare. **Data and Materials Availability:** All code is available online at the following DOI:https://zenodo.org/records/18992244. All other materials, electronics, and hardware used in this work are commercially available.

Supplementary materials

Materials and Methods

Figs. S1 to S24

Tables S1 to S3

Movies S1 to S6

References and Notes

1. H. Ravichandar, A. S. Polydoros, S. Chernova, A. Billard, Recent Advances in Robot Learning from Demonstration. *Annual Review of Control, Robotics, and Autonomous Systems* **3**, 297–330 (2020), number: 1, doi:10.1146/annurev-control-100819-063206, https://www.annualreviews.org/doi/10.1146/annurev-control-100819-063206.

2. E. Gribovskaya, A. Billard, Combining Dynamical Systems control and programming by demonstration for teaching discrete bimanual coordination tasks to a humanoid robot, in *2008 3rd ACM/IEEE International Conference on Human-Robot Interaction (HRI)* (2008), pp. 33–40.

3. K. Kronander, A. Billard, Learning Compliant Manipulation through Kinesthetic and Tactile Human-Robot Interaction. *IEEE Transactions on Haptics* **7** (3), 367–380 (2014), doi:10.1109/TOH.2013.54.

4. A. L. P. Ureche, K. Umezawa, Y. Nakamura, A. Billard, Task Parameterization Using Continuous Constraints Extracted From Human Demonstrations. *IEEE Transactions on Robotics* **31** (6), 1458–1471 (2015), doi:10.1109/TRO.2015.2495003, http://ieeexplore.ieee.org/document/7339616/.

5. A. Pervez, A. Ali, J.-H. Ryu, D. Lee, Novel learning from demonstration approach for repetitive teleoperation tasks, in *2017 IEEE World Haptics Conference (WHC)* (2017), pp. 60–65, doi:10.1109/WHC.2017.7989877.

6. N. Jaquier, *et al.*, Transfer learning in robotics: An upcoming breakthrough?: A review of promises and challenges. *The International Journal of Robotics Research* p. 02783649241273565 (2024), doi:10.1177/02783649241273565, https://doi.org/10.1177/02783649241273565.

7. R. R. Ma, A. Spiers, A. M. Dollar, M2 Gripper: Extending the Dexterity of a Simple, Underactuated Gripper, in *Advances in Reconfigurable Mechanisms and Robots II*, X. Ding, X. Kong, J. S. Dai, Eds. (Springer International Publishing, Cham) (2016), pp. 795–805.

8. B. D. Argall, S. Chernova, M. Veloso, B. Browning, A survey of robot learning from demonstration. *Robotics and Autonomous Systems* **57** (5), 469–483 (2009), number: 5, doi:10.1016/j.robot.2008.10.024, https://linkinghub.elsevier.com/retrieve/pii/S0921889008001772.

9. S. Calinon, F. Guenter, A. Billard, On Learning, Representing, and Generalizing a Task in a Humanoid Robot. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)* **37** (2), 286–298 (2007), doi:10.1109/TSMCB.2006.886952.

10. A. Shaw, J. Lee, J. Park, Constrained dynamic movement primitives for collision avoidance in novel environments. *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (2023).

11. L. Yang, M. Fischer, D. Kragic, Enhancing Learning from Demonstration with DLS-IK and ProMPs, in *International Conference on Automation and Computing (ICAC)* (2024).

12. E. Pignat, S. Calinon, Learning from demonstration using products of experts: Applications to manipulation and task prioritization. *The International Journal of Robotics Research* **41** (4), 373–394 (2022).

13. L. Bakker, V. Koltun, M. Toussaint, TamedPUMA: Safe and Stable Imitation Learning with Geometric Fabrics, in *Learning for Dynamics and Control (L4DC)* (2025).

14. B. Trabucco, M. Phielipp, G. Berseth, AnyMorph: Learning Transferable Polices By Inferring Agent Morphology, in *Proceedings of the 39th International Conference on Machine Learning*, K. Chaudhuri, *et al.*, Eds. (PMLR), vol. 162 of *Proceedings of Machine Learning Research* (2022), pp. 21677–21691.

15. C. Sferrazza, D.-M. Huang, F. Liu, J. Lee, P. Abbeel, Body Transformer: Leveraging Robot Embodiment for Policy Learning, in *8th Annual Conference on Robot Learning* (2024), https://openreview.net/forum?id=0ce2215aJE.

16. H. Klein, N. Jaquier, A. Meixner, T. Asfour, A Riemannian Take on Human Motion Analysis and Retargeting, in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (2022), pp. 5210–5217, doi:10.1109/IROS47612.2022.9982127.

17. N. Makondo, B. Rosman, O. Hasegawa, Knowledge transfer for learning robot models via Local Procrustes Analysis, in *2015 IEEE-RAS 15th International Conference on Humanoid Robots (Humanoids)* (2015), pp. 1075–1082, doi:10.1109/HUMANOIDS.2015.7363502.

18. P. Pastor, H. Hoffmann, T. Asfour, S. Schaal, Learning and generalization of motor skills by learning from demonstration, in *2009 IEEE International Conference on Robotics and Automation* (IEEE, Kobe) (2009), pp. 763–768, doi:10.1109/ROBOT.2009.5152385, http://ieeexplore.ieee.org/document/5152385/.

19. S. Schaal, J. Peters, J. Nakanishi, A. Ijspeert, Learning Movement Primitives, in *Robotics Research. The Eleventh International Symposium*, P. Dario, R. Chatila, Eds. (Springer Berlin Heidelberg, Berlin, Heidelberg) (2005), pp. 561–572.

20. M. Ewerton, G. Maeda, G. Kollegger, J. Wiemeyer, J. Peters, Incremental imitation learning of context-dependent motor skills, in *2016 IEEE-RAS 16th International Conference on Humanoid Robots (Humanoids)* (2016), pp. 351–358, doi:10.1109/HUMANOIDS.2016.7803300.

21. D. H. Salunkhe, D. Chablat, P. Wenger, Trajectory planning issues in cuspidal commercial robots, in *2023 IEEE International Conference on Robotics and Automation (ICRA)* (2023), pp. 7426–7432, doi:10.1109/ICRA48891.2023.10161444.

22. D. H. Salunkhe, T. Marauli, A. Müller, D. Chablat, P. Wenger, Kinematic issues in 6R cuspidal robots, guidelines for path planning and deciding cuspidality. *The International Journal of Robotics Research* p. 02783649241293481 (2024), doi:10.1177/02783649241293481.

23. P. Wenger, J. El Omri, Changing posture for cuspidal robot manipulators, in *Proceedings of IEEE International Conference on Robotics and Automation* (IEEE, Minneapolis, MN, USA), vol. 4 (1996), pp. 3173–3178.

24. P. Wenger, Design of cuspidal and non-cuspidal robot manipulators, in *Proceedings of International Conference on Robotics and Automation*, vol. 3 (1997), doi:10.1109/ROBOT.1997.619284.

25. P. Wenger, Uniqueness domains and regions of feasible paths for cuspidal manipulators. *IEEE Transactions on Robotics* **20** (4), 745–750 (2004), doi:10.1109/TRO.2004.829467.

26. D. H. Salunkhe, S. Gupta, A. Billard, Cuspidal Redundant Robots: Classification of Infinitely Many IKS of Special Classes of 7R Robots. *IEEE Robotics and Automation Letters* **10** (12), 12509–12516 (2025), doi:10.1109/LRA.2025.3623011.

27. J. W. Burdick, A classification of 3R regional manipulator singularities and geometries. *Mechanism and Machine Theory* **30** (1), 71–89 (1995), doi:https://doi.org/10.1016/0094-114X(94)00043-K, `https://www.sciencedirect.com/science/article/pii/0094114X9400043K`.

28. P. Borrel, A. Liegeois, A study of multiple manipulator inverse kinematic solutions with applications to trajectory planning and workspace determination, in *Proceedings. 1986 IEEE International Conference*

on Robotics and Automation (Institute of Electrical and Electronics Engineers, San Francisco, CA, USA), vol. 3 (1986).

29. P. Wenger, Classification of 3R Positioning Manipulators. *Journal of Mechanical Design* **120** (2), 327–332 (1998), doi:10.1115/1.2826976.

30. D. Paganelli, *Topological Analysis of Singularity Loci for Serial and Parallel Manipulators*, Ph.D. thesis, Universita di Bologna, Bologna, Italy (2008).

31. M. Asgari, I. Bonev, C. Gosselin, Singularities of ABB's YuMi 7-DOF robot arm. *Mechanism and Machine Theory* **205** (2025), doi:https://doi.org/10.1016/j.mechmachtheory.2024.105884.

32. A. J. Elias, J. T. Wen, Redundancy parameterization and inverse kinematics of 7-DOF revolute manipulators. *Mechanism and Machine Theory* **204**, 105824 (2024), doi:https://doi.org/10.1016/j.mechmachtheory.2024.105824.

33. S. Gupta, A. Nayak, A. Billard, Compact Oneshot Modelling of High Dimensional Demonstrations using Laplacian Eigenmaps. *IEEE Transactions on Robotics* pp. 1–18 (2026), doi:10.1109/TRO.2026.3666134.

34. A. Dragan, K. Lee, S. Srinivasa, Legibility and predictability of robot motion, in *Proceedings of the 8th ACM/IEEE International Conference on Human-Robot Interaction*, HRI '13 (IEEE Press) (2013), p. 301–308, doi:10.1109/HRI.2013.6483603.

35. J. Geldenbott, K. Leung, Legible and Proactive Robot Planning for Prosocial Human-Robot Interactions, in *2024 IEEE International Conference on Robotics and Automation (ICRA)* (2024), pp. 13397–13403, doi:10.1109/ICRA57147.2024.10611294.

36. P. Wenger, D. Chablat, A Review of Cuspidal Serial and Parallel Manipulators. *Journal of Mechanisms and Robotics* **15** (4), 040801 (2022), doi:10.1115/1.4055677.

37. D. H. Salunkhe, C. Spartalis, J. Capco, D. Chablat, P. Wenger, Necessary and sufficient condition for a generic 3R serial manipulator to be cuspidal. *Mechanism and Machine Theory* **171**, 104729 (2022), doi:10.1016/j.mechmachtheory.2022.104729.

38. S. M. Khansari-Zadeh, A. Billard, Learning Stable Nonlinear Dynamical Systems With Gaussian Mixture Models. *IEEE Transactions on Robotics* **27** (5), 943–957 (2011), number: 5, doi:10.1109/TRO.2011.2159412, http://ieeexplore.ieee.org/document/5953529/.

39. J. Denavit, R. S. Hartenberg, A Kinematic Notation for Lower-Pair Mechanisms Based on Matrices. *Journal of Applied Mechanics* **22** (2), 215–221 (1955), doi:10.1115/1.4011045.

40. D. Kohli, J. Spanos, Workspace Analysis of Mechanical Manipulators Using Polynomial Discriminants. *Journal of Mechanisms, Transmissions, and Automation in Design* **107** (2), 209–215 (1985), doi:10.1115/1.3258710.

41. J. Burdick, A classification of 3R regional manipulator singularities and geometries, in *Proceedings. 1991 IEEE International Conference on Robotics and Automation* (1991), pp. 2670–2675 vol.3, doi:10.1109/ROBOT.1991.132033.

42. D. Pai, M. Leu, Genericity and singularities of robot manipulators. *IEEE Transactions on Robotics and Automation* **8** (5), 545–559 (1992).

43. J. W. Burdick, A classification of 3R regional manipulator singularities and geometries. *Mechanism and Machine Theory* **30** (1), 71–89 (1995), doi:https://doi.org/10.1016/0094-114X(94)00043-K, https://www.sciencedirect.com/science/article/pii/0094114X9400043K.

44. J. El Omri, P. Wenger, How to recognize simply a non-singular posture changing 3-DOF manipulator, in *Proc. 7th Int. Conf. on Advanced Robotics* (1995), pp. 215–222.

45. D. L. Pieper, *The Kinematics of Manipulators Under Computer Control*, Ph.D. thesis, Stanford University, USA (1968).

46. D. H. Salunkhe, *Robots cuspidaux : étude théorique, classification et applications aux robots commerciaux*, Ph.D. thesis, Ecole Centrale de Nantes, France (2023).

47. D. Chablat, R. Prébet, M. Safey El Din, D. H. Salunkhe, P. Wenger, Deciding Cuspidality of Manipulators through Computer Algebra and Algorithms in Real Algebraic Geometry, in *Proceedings of the 2022 International Symposium on Symbolic and Algebraic Computation*, ISSAC '22 (Association for Computing Machinery, Villeneuve-d'Ascq, France) (2022), p. 439–448, doi:10.1145/3476446.3535477.

48. K. Makarychev, Y. Makarychev, Certified Algorithms: Worst-Case Analysis and Beyond, in *11th Innovations in Theoretical Computer Science Conference (ITCS 2020)*, T. Vidick, Ed. (Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany), vol. 151 of *Leibniz International Proceedings in Informatics (LIPIcs)* (2020), pp. 49:1–49:14, doi:10.4230/LIPIcs.ITCS.2020.49.

49. S. Salvador, P. Chan, FastDTW: Toward Accurate Dynamic Time Warping in Linear Time and Space p. 11.

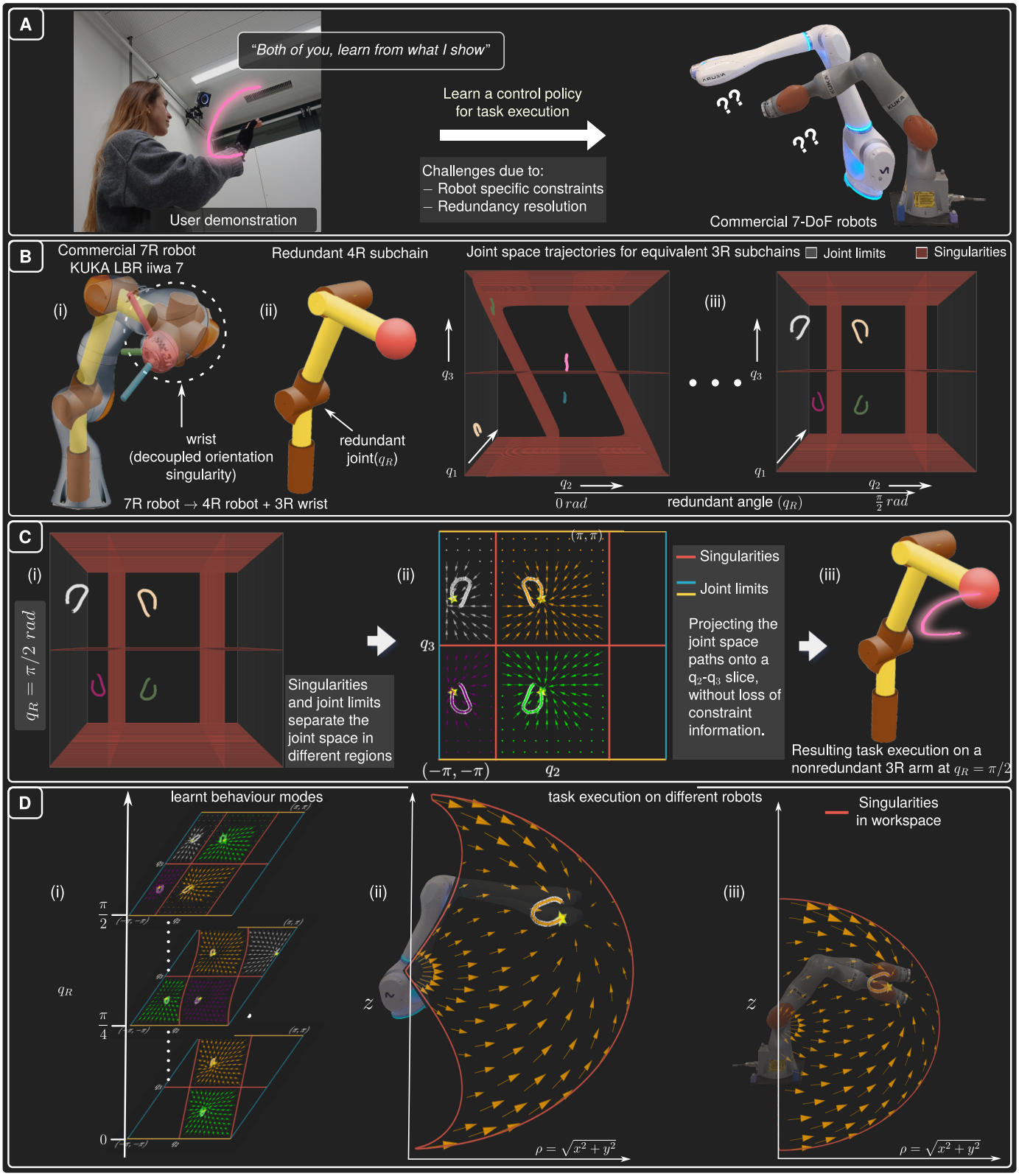
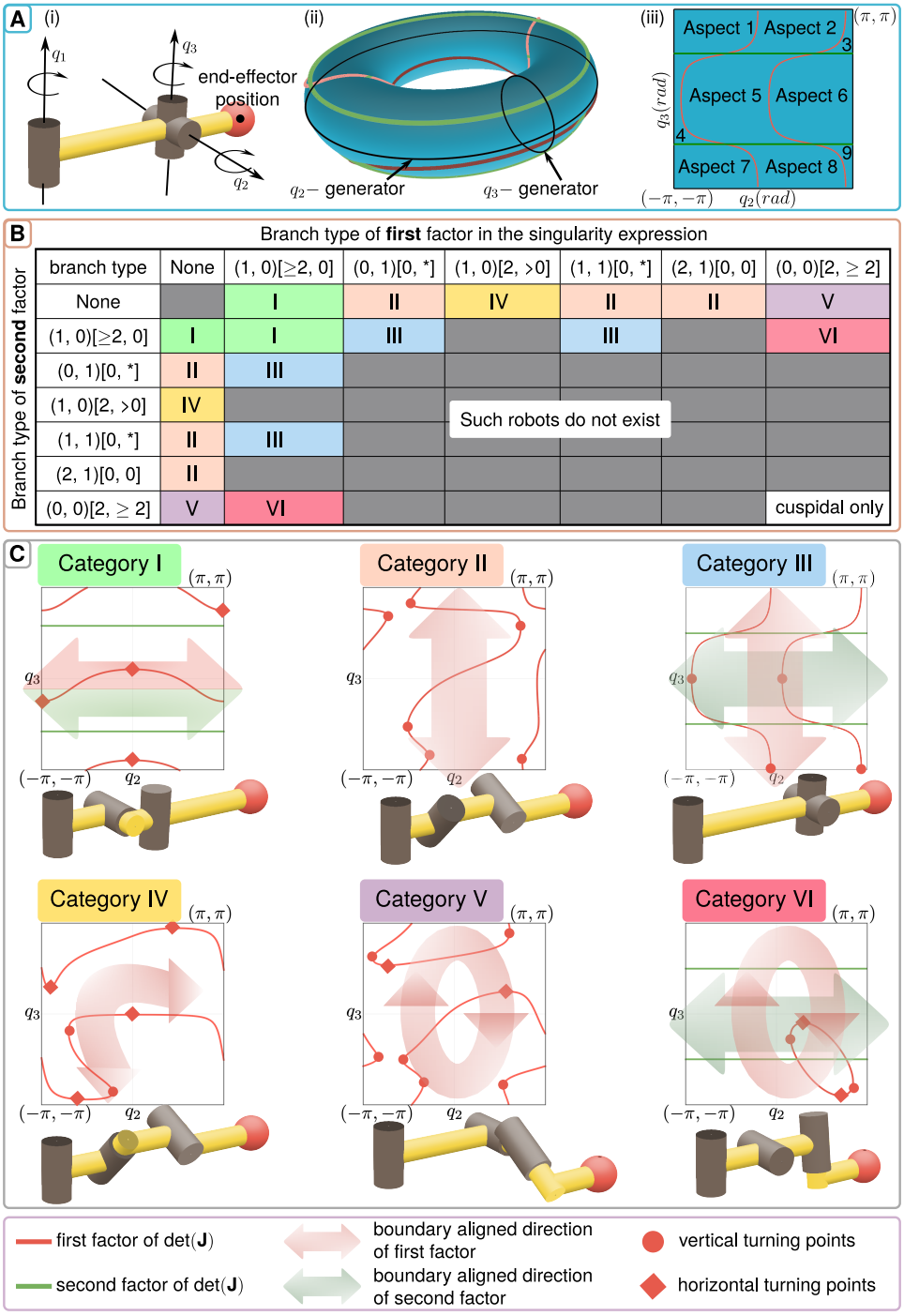


Figure 1: **Embedding Kinematic Intelligence in human-demonstrated behaviour for transferable skill policies.** (**A**) A human demonstration defines the task, the policy for which must be learned and transferred across two commercially available 7-DoF robots. (**B**) Transfer reduction via kinematic structure: (i) wrist-partitioned 7-DoF robot; (ii) simplification to a redundant 4-DoF subchain; (iii) redundancy parameterized by joint q_R, establishing equivalence across non-redundant $3R$ robots. (**C**) Kinematic constraints of an equivalent $3R$ robot (at $q_R = \pi/2$): (i–ii) singularities and joint limits partition joint space; (ii-iii) a global control policy generates stable subspace dynamics for constraint-respecting execution. (**D**) Cross-robot transfer: (i) policies parameterized by q_R; (ii–iii) successful robot execution.



branch type	None	(1, 0)[≥2, 0]	(0, 1)[0, *]	(1, 0)[2, >0]	(1, 1)[0, *]	(2, 1)[0, 0]	(0, 0)[2, ≥ 2]
None		I	II	IV	II	II	V
(1, 0)[≥2, 0]	I	I	III		III		VI
(0, 1)[0, *]	II	III					
(1, 0)[2, >0]	IV						
(1, 1)[0, *]	II	III					
(2, 1)[0, 0]	II						
(0, 0)[2, ≥ 2]	V	VI					cuspidal only

Figure 2: **Topological-differential properties of singularities for categorising noncuspidal $3R$ robots and defining boundary-aligned directions.** (**A**) Representation of a $3R$ robot: (i) robot schematic; (ii) torus generated by (q_2, q_3); (iii) aspects partitioning the $q_2 - q_3$ slice. (**B**) Topological-differential analysis of the factors of the Jacobian determinant to assign any noncuspidal $3R$ to one of six categories (I–VI). (**C**) Representative robots from each category with their singularity curves and boundary-aligned directions (arrows) in the q_2–q_3 slice of joint space. Arrows indicate the control actions executed by the track-cycle control policy when the robot is near constraints. Category I: horizontal traversal; Category II: vertical traversal; Category III: switching between horizontal and vertical traversal; Category IV: predominantly horizontal traversal with turning-point considerations; Category V: clockwise or counterclockwise traversal along closed singular loops; Category VI: loop traversal combined with horizontal drift along extended branches. The track-cycle policy, near constraints, guides the robot along aspect boundary towards the goal maintaining motion within the same aspect.

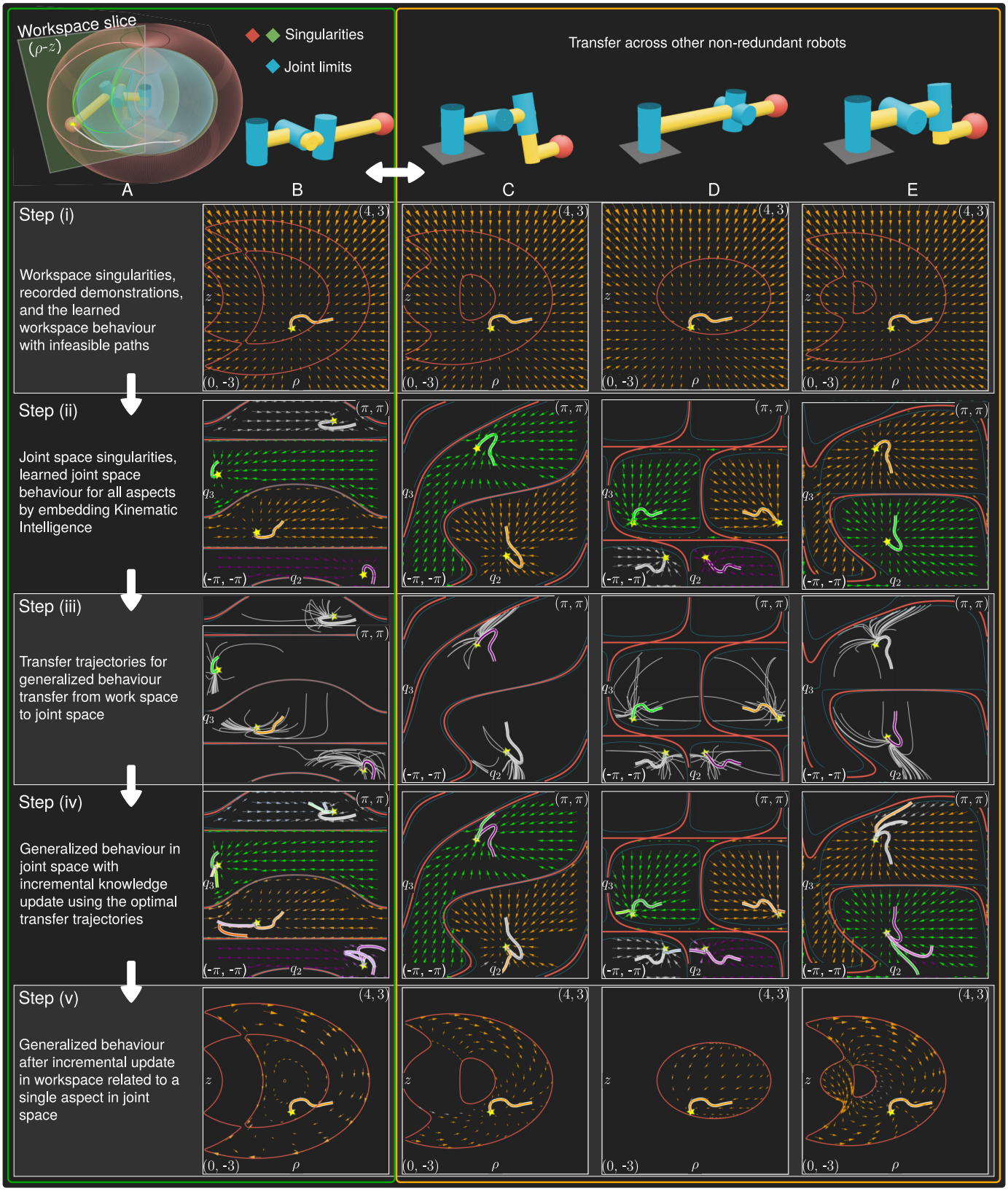


Figure 3: **Transfer of demonstrated behaviour to multiple non-redundant robots.** (**A**) Workspace of a 3R robot (schematic in **B**) with the learning framework steps (i–v). The demonstrated trajectory is visualized in a ρ–z workspace slice. Columns (**B**–**E**) correspond to representative robots from Categories I–IV, and rows show the framework stages: (i) nominal workspace policy and singularities in the ρ–z slice; (ii) learned control policies for feasible joint space aspects; (iii) transfer trajectories for incremental policy updates; (iv) most novel transfer trajectory and updated behaviour in each aspect; (v) resulting constraint-compliant workspace behaviour.

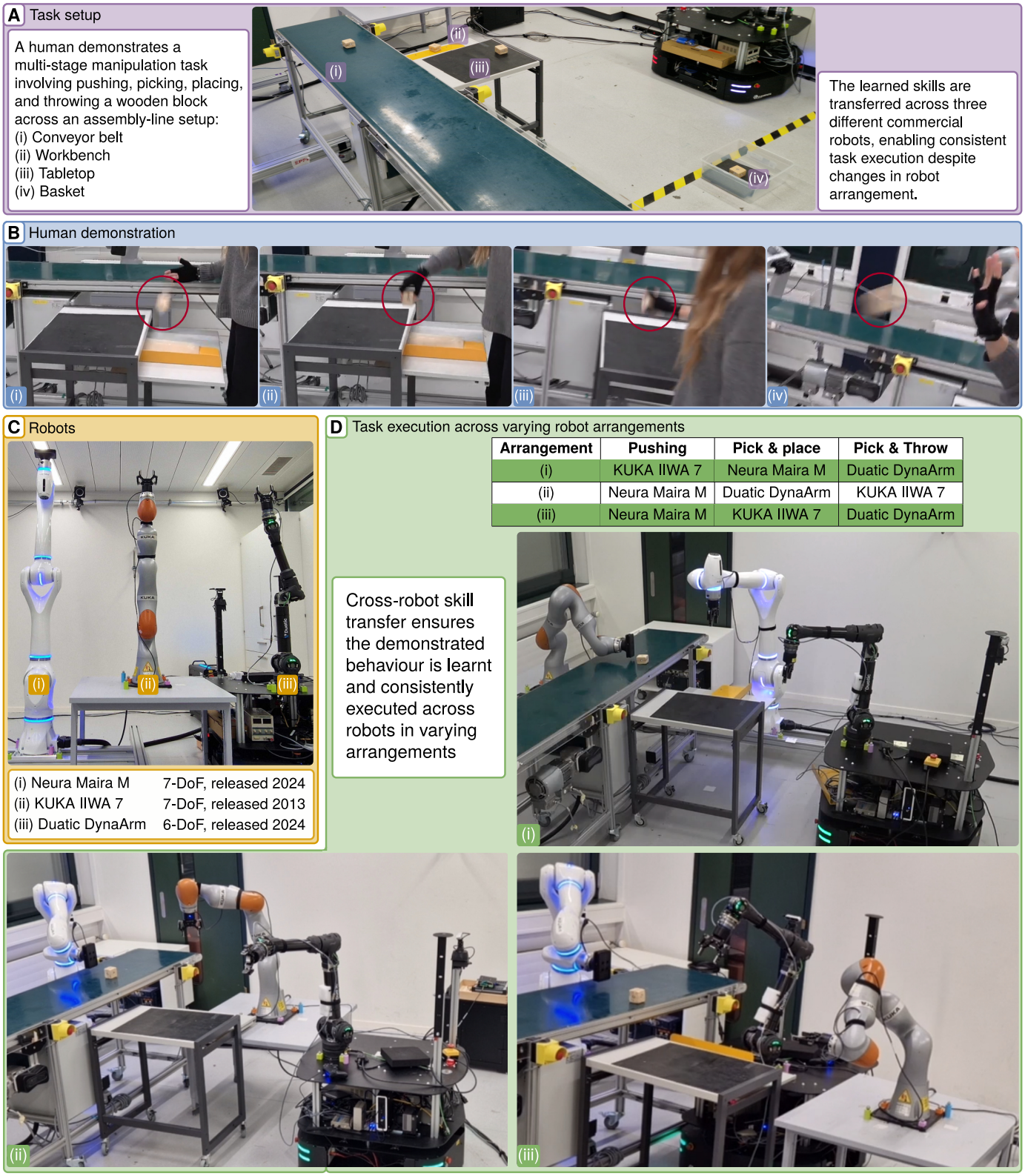


Figure 4: **Experimental validation in a mock multi-robot automated assembly line.** (**A**) Assembly-line setup consisting of: (i) a conveyor belt; (ii) workbench; (iii) tabletop; (iv) basket. (**B**) Human demonstration of the task: (i) pushing the wooden block off the conveyor belt on to the workbench; (ii) picking the block off workbench and placing it on the tabletop; (iii) lifting it into a throwing pose; (iv) throwing wooden block into the basket. (**C**) Three commerical robots to reproduce the demonstrated behaviour: (i) Neura Maira M; (ii) KUKA IIWA 7; (iii) Duatic DynaArm. (**D**) Varying robot arrangements (i-iii) in the assembly line for task execution.

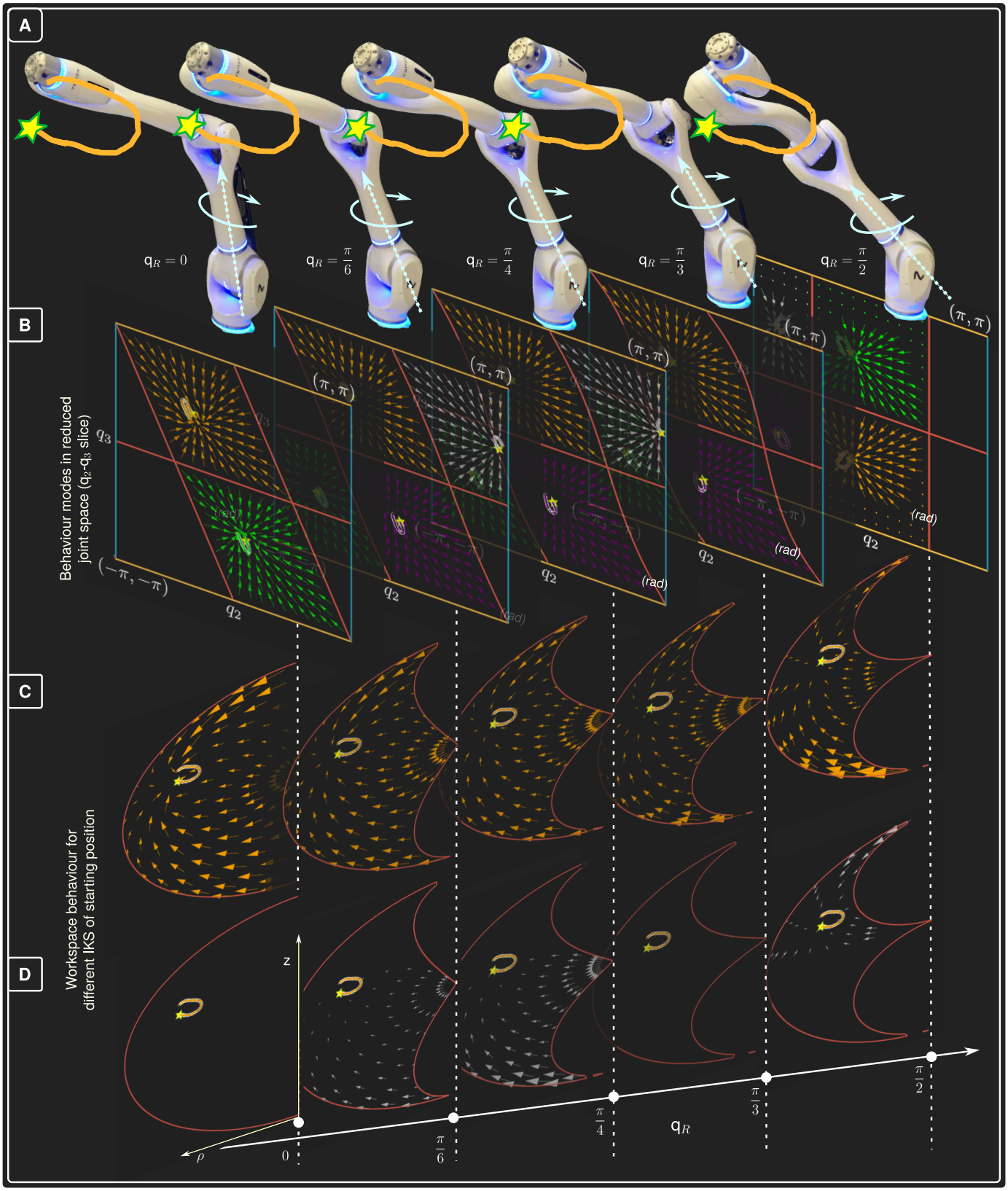
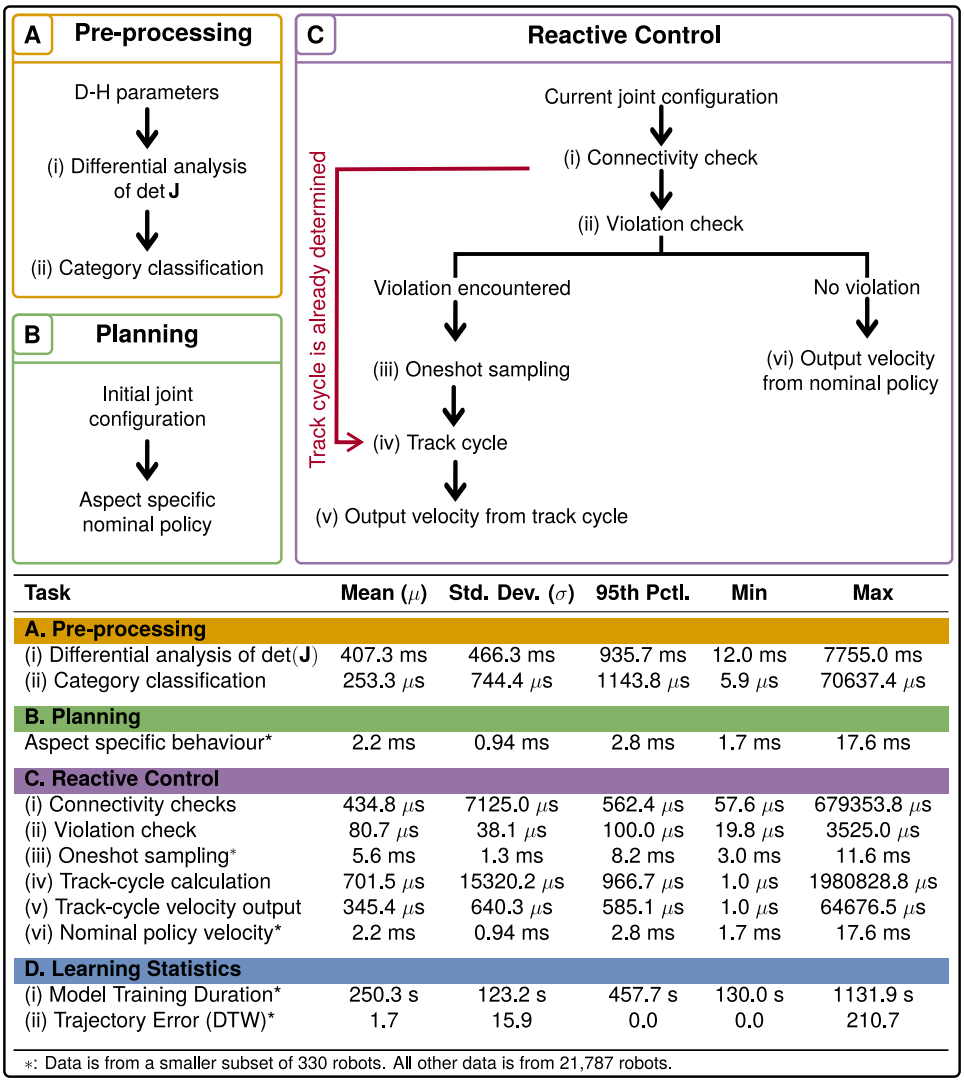


Figure 5: **Behaviour transfer to redundant robots through redundancy parameterization.** (**A**) Behaviour of the redundant $4R$ subchain of a wrist-removed 7-DoF wrist-partitioned robot is characterized by discretizing the redundant angle q_R. Five representative values 0–$\pi/2$ radians are considered. (**B**) Control policies for the equivalent non-redundant $3R$ subchains at these q_R values in the reduced joint space. (**C**) Feasible workspace shown in a ρ–z slice for each q_R, with the robot configuration in a specific joint space aspect highlighted in orange. (**D**) Resulting workspace behaviour when the robot starts from a different inverse-kinematic solution (IKS), shown in silver.



Task	Mean (μ)	Std. Dev. (σ)	95th Pctl.	Min	Max
A. Pre-processing					
(i) Differential analysis of det(**J**)	407.3 ms	466.3 ms	935.7 ms	12.0 ms	7755.0 ms
(ii) Category classification	253.3 μs	744.4 μs	1143.8 μs	5.9 μs	70637.4 μs
B. Planning					
Aspect specific behaviour*	2.2 ms	0.94 ms	2.8 ms	1.7 ms	17.6 ms
C. Reactive Control					
(i) Connectivity checks	434.8 μs	7125.0 μs	562.4 μs	57.6 μs	679353.8 μs
(ii) Violation check	80.7 μs	38.1 μs	100.0 μs	19.8 μs	3525.0 μs
(iii) Oneshot sampling*	5.6 ms	1.3 ms	8.2 ms	3.0 ms	11.6 ms
(iv) Track-cycle calculation	701.5 μs	15320.2 μs	966.7 μs	1.0 μs	1980828.8 μs
(v) Track-cycle velocity output	345.4 μs	640.3 μs	585.1 μs	1.0 μs	64676.5 μs
(vi) Nominal policy velocity*	2.2 ms	0.94 ms	2.8 ms	1.7 ms	17.6 ms
D. Learning Statistics					
(i) Model Training Duration*	250.3 s	123.2 s	457.7 s	130.0 s	1131.9 s
(ii) Trajectory Error (DTW)*	1.7	15.9	0.0	0.0	210.7

$*$: Data is from a smaller subset of 330 robots. All other data is from 21,787 robots.

Figure 6: **End-to-end policy execution on a single robot.** (**A**) Pre-processing: an offline one-time step to determine the robot's kinematic structure, and perform categorisation using topological-diffferential properties. (**B**) Planning: given an initial joint configuration, the framework identifies the corresponding aspect and the suitable joint space control policy. (**C**) Reactive control: an online loop combining connectivity checks, constraint-violation monitoring, nominal policy execution, and track-cycle computations to keep the robot within feasible regions. Reported wall-clock timings summarize the computational cost of each component, highlighting that the one-time offline pre-processing is very fast, and the online planning and reactive control steps are compatible with real-time execution.

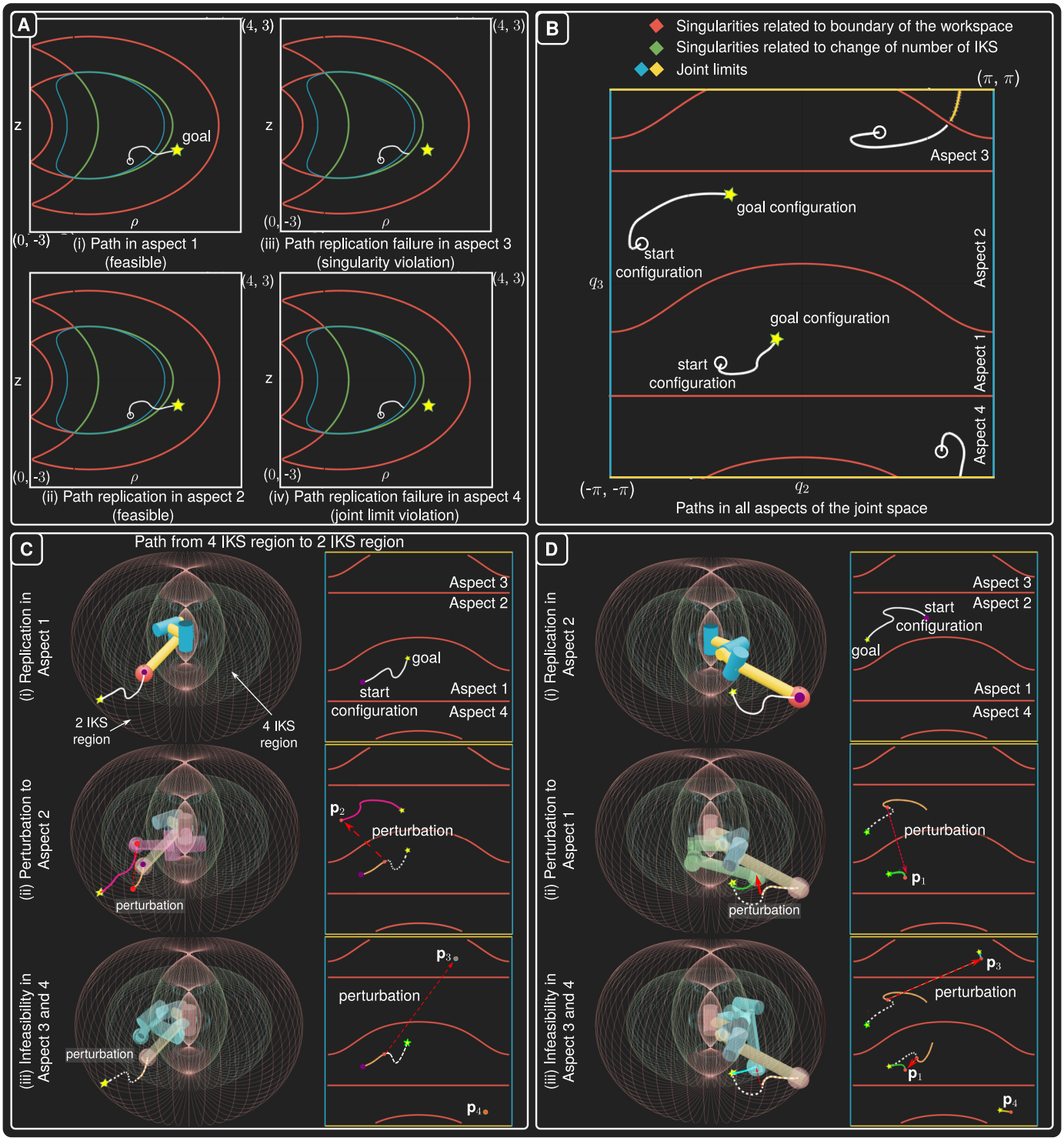


Figure 7: **Certified execution for non-redundant robots.** The figure analyzes the workspace from Figure 3A(i) on a Category I 3*R* robot. The start position admits four inverse-kinematic solutions (IKS), and the goal admits two. (**A**) Workspace trajectories for each aspect in a ρ–z slice of the workspace: (i-ii) Aspects 1 and 2 reach the goal without violations; (iii) Aspect 3 encounters a singular configuration; (iv) Aspect 4 violates the third-joint limit. (**B**) Corresponding joint space trajectories for the four aspects, highlighting constraint violations in Aspects 3 and 4. (**C**) Robot behaviour after learning the control policy and embedding Kinematic Intelligence: (i-ii) Starting from a feasible aspect or being perturbed to a feasible configuration $\mathbf{p}_2$, the policy achieves violation-free convergence to the goal; (iii) If the robot is perturbed to or starts from an infeasible configuration ($\mathbf{p}_3$, $\mathbf{p}_4$ respectively), the motion halts to avoid constraint violations. (**D**) Behaviour when the policy is learned from the reversed demonstration, where the start has two IKS and the goal has four: (i-iii) The learned policy reaches the goal from each configuration, including perturbations to configurations $\mathbf{p}_1$, $\mathbf{p}_3$, and starting from $\mathbf{p}_4$.

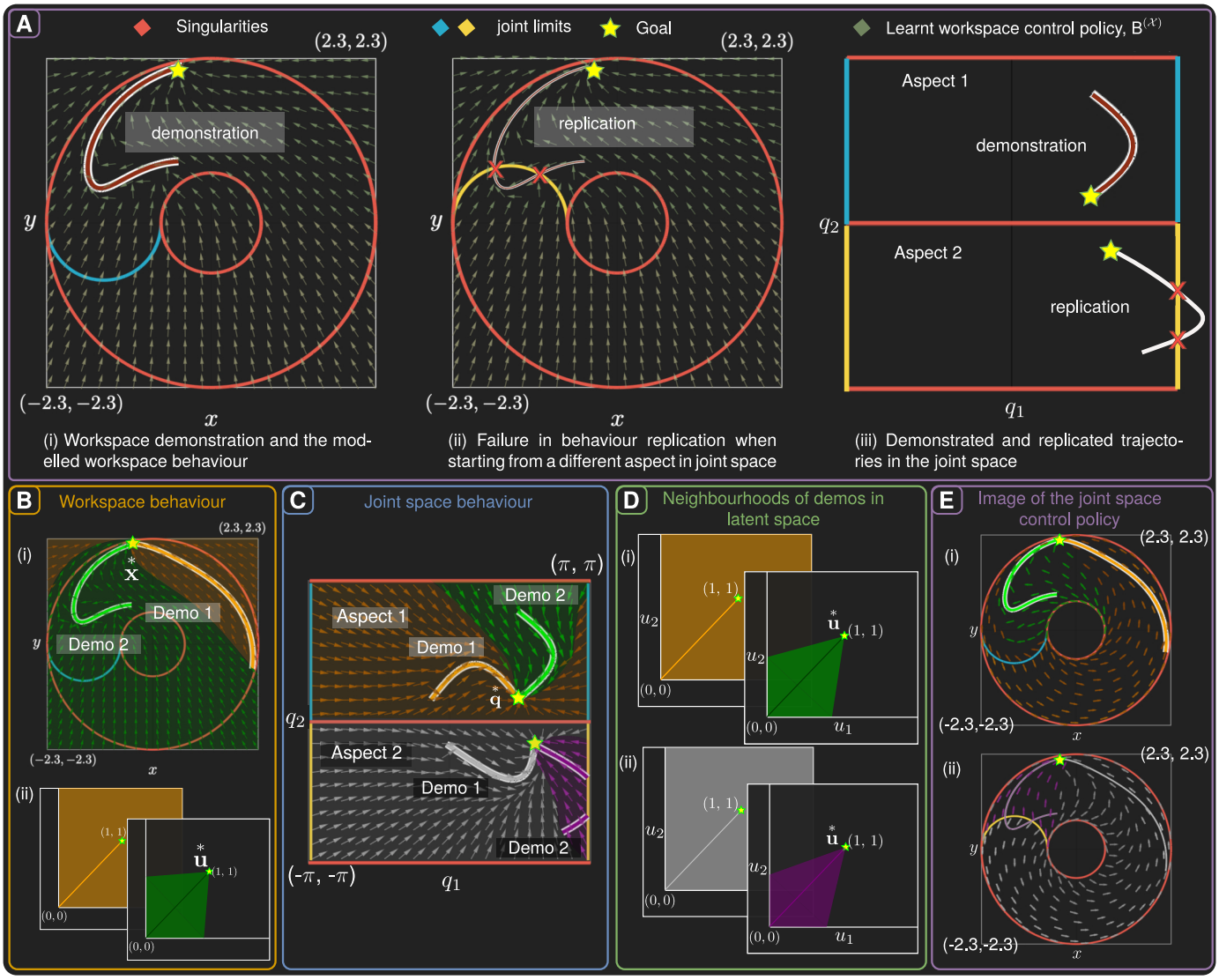


Figure 8: **Learning and replicating demonstrated behaviour for a $2R$ robot.** (**A**) Modelling the workspace behaviour and replication from different IKS: (i) workspace trajectory when robot is in Aspect 1; (ii) workspace replication when robot is in Aspect 2 leading to joint-limit violation; (iii) joint space representation of replications. (**B**) Learning workspace behaviour from multiple demonstrations: (i) dynamical-system embeddings and resulting workspace behaviour; (ii) latent-space representations with their cones of influence. (**C**) Corresponding joint space trajectories, trimmed for constraint compliance, and the resulting aspect-specific behaviour. (**D**) Latent embeddings of the dynamical systems in the aspect-specific behaviours, with their respective cones of influence (i-ii). (**E**) Resulting workspace behaviour when executing the joint space control policy from initial configuration: (i) in Aspect 1; (ii) Aspect 2.

Supplementary Materials for

Demonstrate Once, Execute on Many: Kinematic Intelligence for Cross-robot Skill Transfer

Sthithpragya Gupta*†, Durgesh Salunkhe Scientist†, Aude Billard

*Corresponding author. Email: sthithpragya.gupta@epfl.ch

†These authors contributed equally to this work.

This PDF file includes:

Materials and Methods

Figs. S1 to S22

Tables S1 to S3

Captions for Movies S1 to S6

Other Supplementary Materials for this manuscript:

Movies S1 to S6

Materials and Methods

Glossary

$\mathcal{Q}$ Joint space of the robot

$\mathcal{X}$ Workspace of the robot

EE-position End-effector position

$\mathbf{J}(\mathbf{q})$ Jacobian of the forward kinematic map at $\mathbf{q}$

$\det \mathbf{J}(\mathbf{q})$ Determinant of Jacobian at configuration $\mathbf{q}$

IKS Inverse kinematic solutions

$\mathcal{S}$ Singularities in joint space

t_i Rational parameterization of q_i, where $t_i = \tan \frac{q_i}{2}$

$\mathcal{B}$ Branch of a singularity, $\mathcal{B} \subset \mathcal{S}$

$\mathcal{U}$ Latent space that encodes the demonstration as a structured latent embedding

$\{\mathbf{X}\}$ Set of K expert demonstrations in the robot's workspace

$\{\mathbf{Q}\}$ Set of K_Q demonstrations in joint space

$\mathrm{B}^{(\mathcal{Q})}$ Behaviour learned in joint space

$\mathrm{B}^{(\mathcal{A}_i)}$ Aspect-specific behaviour in joint space

$\mathrm{B}^{(\mathcal{X})}$ Behaviour learned in workspace

$\overset{*}{\mathbf{u}}$ Latent embedding of the attractor in latent space

$\overset{*}{\mathbf{x}}$ Demonstrated workspace goal position

$\{\overset{*}{\mathbf{q}}\}$ Attractors in $\mathcal{Q}$, i.e., IK solutions of $\overset{*}{\mathbf{x}}$

$\{A_i\}_{i=1}^{I}$ Aspects forming a disjoint partition of $\mathcal{Q}$

Preliminaries

On the kinematics front, Section *Singularities in serial robots* presents definitions relevant to the kinematics of serial robots, and their singularities. Section *Homotopy classes* leverages these definitions and lays the foundation of homotopy-based classification of the robots on basis of their singularities. On the behaviour modelling front, Section *Oneshot embedding of a demonstration* presents the method to embed a demonstration as a stable control policy.

This paper uses original D-H parameters (*39*) to minimally represent a serial robot architecture. The subspace of $SE(3)$ formed by the reachable poses of a given nR robot is called the workspace and is denoted as workspace $\mathcal{X}$. The joint configuration of a robot is denoted as $\mathbf{q}$ and is a point in the joint space, joint space $\mathcal{Q}$, where $\mathbb{T}^n$ is an n-torus. Let $\mathbf{w}$ be the end-effector pose in $SE(3)$ corresponding to $\mathbf{q}$. The mapping between joint space $\mathcal{Q}$ and workspace $\mathcal{X}$, denoted by $\text{FK} : \mathcal{Q} \to \mathcal{X}$, defines the forward kinematics function $\text{FK} : \mathbf{q} \mapsto \mathbf{x}$.

Singularities in serial robots

The singularities $\mathcal{S}$ is the locus of critical points of FK in joint space $\mathcal{Q}$ that correspond to the configurations in the joint space where the Jacobian $\mathbf{J}(\mathbf{q})$ loses rank, i.e., when Jacobian determinant $\det \mathbf{J}(\mathbf{q})$ is zero. The critical values are the images of the critical points in $\mathcal{W}$. (*40*) showed that the roots of the univariate polynomial used to solve the inverse kinematics of a 3R robot have a multiplicity greater than or equal to two. A similar conclusion can be drawn for nonredundant nR robots, as the robot either loses or gains multiple inverse kinematic solutions (IKS) upon crossing the locus of critical values.

Aspect: (*28*) defines an aspect as the largest singularity-free connected region in the joint space. Each aspect is bounded only by singularities in the joint space, and this definition does not incorporate the robot's joint limits. However, it has been noted that the kinematic properties of the robot are significantly affected by its joint limits (*36*). To facilitate real-world application of our framework, we extend the definition of aspect to be the largest singularity-free region in the joint space bounded by singularities, joint limits or a combination of both. The joint space is parameterized in q_2 and q_3 only as the singularities of the 3R robot are independent of q_1 (*41*). The pink region is bounded by only singularities, the green region by singularities and joint limits of q_2 only, the blue region by singularities and joint limits of q_3 only, and the yellow region is bounded by singularities as well as joint limits of both q_2 and q_3.

For a structured analysis of $3R$ robots, as proposed by (*42*), the properties of robot singularities can be

leveraged to yield the following two categories:

Generic 3*R robot*: A robot whose singularities do not intersect.

Non-Generic robot: A 3*R* robot is nongeneric if it satisfies either of the following two conditions (*27*):

1. singularities $\mathcal{S}$ contains rank 1 critical points
2. singularities $\mathcal{S}$ contains rank 2 critical points, $\mathbf{q}_i^s$, satisfying:

$$\frac{\partial \det \mathbf{J}(\mathbf{q}^s)}{\partial \mathbf{q}_i^s} = 0, i = 2, 3$$

In this article, we will exclude the robots that satisfy the first condition of nongenericity as they have either reduced mobility or have singularities that map to an isolated critical value in the workspace (*43*).

Cusp: A cusp is a point in the locus of critical values of a 3R robot that satisfies the following conditions (*44*):

$$\begin{aligned} &M(t_3, R, z) = 0 \\ &\frac{\partial M}{\partial t_3}(t_3, R, z) = 0 \\ &\frac{\partial^2 M}{\partial t_3^2}(t_3, R, z) = 0 \\ &\frac{\partial^3 M}{\partial t_3^3}(t_3, R, z) \neq 0 \end{aligned} \tag{S1}$$

where, for an end effector position, $R = x^2 + y^2 + z^2$, and rational parameterization of q_i, first=$t_i = \tan \frac{q_i}{2}$, text=t_i. The function, $M(t_3, R, z)$, is a polynomial of degree four in t_3. This polynomial is obtained by eliminating the joint variables t_1 and t_2 from the forward kinematic function (refer to (*45*)).

Cuspidal robots: Robots whose joint space has at least one aspect with multiple IKS are defined as cuspidal robots (*44*).

Horizontal (resp. vertical) turning points: The joint configurations $\mathbf{q}$ that satisfy:

$$\begin{aligned} &\det \mathbf{J}(\mathbf{q}) = 0 \\ &\gamma \frac{\partial \det \mathbf{J}}{\partial q_2} = 0 (resp. 1), \text{ for } \gamma \neq 0 \\ &\gamma \frac{\partial \det \mathbf{J}}{\partial q_3} = 1 (resp. 0), \text{ for } \gamma \neq 0 \end{aligned} \tag{S2}$$

Homotopy classes

The classification of 3R serial robots based on the number and class of topology of the smooth manifolds of $\mathcal{S}$ was presented in (*41*). Later, the work was extended to classify 3R robots based on cuspidality by (*29*) and (*30*). As the Jacobian determinant $\det \mathbf{J}(\mathbf{q})$ for a 3R robot depends on only q_2 and q_3, singularities are a union of 1-dimensional closed curves, *loops*. The joint space spanned by q_2 and q_3 is a $\mathbb{T}^2$. In the 2-dimensional Euclidean plane, $\mathbb{R}^2$, all loops are homeomorphic to a point, but this does not hold for loops on $\mathbb{T}^2$. The number of homotopy classes depends on the different ways of encircling the generators of the $\mathbb{T}^2$ (*29*). Let n_2 and n_3 be two non-negative integers, then a homotopy class (n_2, n_3) suggests that a loop in $\mathcal{S}$ encircles the q_2- (respectively q_3-) generator n_2 (respectively n_3) times. The most important results from (*29*) and (*30*) that will be used for the presented work are related to the analysis of loops on $\mathbb{T}^2$ by utilizing the degree of t_2 and t_3 in Jacobian determinant $\det \mathbf{J}(\mathbf{q})$. The rational parameterization of Jacobian determinant $\det \mathbf{J}(\mathbf{q})$ for a 3R robot in terms of t_2 and t_3 is given as

$$\det \mathbf{J}(q_2, q_3) \rightarrow p(t_2^2, t_3^4).$$

This suggests that for any given value of t_2 (resp. t_3), we can have at most four (resp. two) singular joint configurations. In a non-generic robot, we can also have singularities for all values of t_2 (resp. t_3) for a given value of t_3 (resp. t_2). This often appears as a factored term in Jacobian determinant $\det \mathbf{J}(\mathbf{q})$ (e.g., $\det \mathbf{J} = \cos(q_3) \cdot \mathrm{j}(q_2, q_3)$). It is shown in (*29, 30*) that $n_2 \leq 2$ and $n_3 < 2$. So, a loop on $\mathbb{T}^2$ is homeomorphic to one of the five homotopy classes viz. $(0, 0)$, $(0, 1)$, $(1, 0)$, $(1, 1)$, $(2, 1)$. The previous homotopy-based classification does not consider *nongeneric* 3R robots. We extend the classification of singularities to *nongeneric* 3R robots by considering the conditions for nongenericity. This is of practical importance as most of the industrial robots with wrist-partitioned architecture have *nongeneric* positional $3R$ chain.

Oneshot embedding of a demonstration

One principled approach to behavior modeling is to represent a demonstrated trajectory as the flow of a stable dynamical system (DS). Once learned, this DS functions as a control policy that inherently exhibits robustness to spatial and temporal perturbations. By incrementally building a library of stable control policies, the system gradually accumulates knowledge of the overall demonstrated task. Each policy captures a locally stable behavior derived from a demonstration, and as more demonstrations are added, the library grows to reflect a more complete understanding of the task, while maintaining stability throughout. As proposed in (*33*), we present here the oneshot mechanism to embed the demonstration as a

globally asymptotically stable DS. In doing so, the demonstration is mapped to a stable control policy that ensures global convergence from any initial configuration in the task space.

We begin by considering a goal-directed human demonstration in the workspace $\mathbf{X} = \{\mathbf{x}_i \in \mathcal{X}\}_{i=1}^{M}$ consisting of a sequence of M demonstrated positions, achieving a zero-velocity final state. It is assumed that the demonstration is non self-intersecting. The objective is to model the workspace dynamical system $\dot{\mathbf{x}} = \mathrm{g}(\mathbf{x})$ that accurately models the demonstrated trajectory. The stable attractor of the dynamical system in the workspace $\overset{*}{\mathbf{x}}$ corresponds to the final configuration in the provided demonstration. Instead of modelling the dynamical system directly in the workspace, we learn a diffeomorphic transformation to an auxiliary space, referred to as the latent space $\sqcap$, where the dynamics are analytically defined and globally stable. This transformation allows us to induce an equivalent dynamical system in the workspace, while preserving the desired stability properties through the diffeomorphic mapping. (*33*) proposes a method to generate a (quasi)linear embedding $\mathbf{U} = \{\mathbf{u}_i \in \mathcal{U}\}_{i=1}^{M}$ of the workspace demonstration in a latent space of same dimensionality. The latent embedding depends only on the dimensionality of workspace $\mathcal{X}$, and the number of demonstrated points. Consequently, the method is applicable to generating latent embeddings for any non self-intersecting demonstrations without lack of generality. The globally asymptotically stable DS governing the latent space, given by

$$\dot{\mathbf{u}} = \mathbf{k} \circ \sqrt{\mathbf{1}_{\mathbf{n}}^{\mathbf{T}} - \mathbf{u}^2} \tag{S3}$$

where $k \in \mathbb{R}_{+}^{n}$ is constant, uniquely determined while generating the latent embedding. The attractor for the latent space DS $\overset{*}{\mathbf{u}} = \mathbf{1}_n^T$ serves as the latent embedding of workspace goal $\overset{*}{\mathbf{x}}$. A diffeomorphic mapping $\psi : \mathcal{U} \rightarrow \mathcal{X}$ from the latent space to the workspace is learned to align the latent embedding $\mathbf{U}$ with the demonstrated trajectory $\mathbf{X}$, where $\mathbf{X} = \psi(\mathbf{U})$ Since ψ is a diffeomorphism, the latent space DS from Equation S3 and the workspace DS $\dot{\mathbf{x}} = \mathrm{g}(\mathbf{x})$ are topologically equivalent, preserving the global asymptotic stability of the equilibrium. The workspace DS can be reformulated as

$$\dot{\mathbf{x}} = \mathrm{g}(\mathbf{x}) = \frac{\partial\, \psi(\mathbf{u})}{\partial \mathbf{u}} \dot{\mathbf{u}} \Big|_{\mathbf{u}=\psi^{-1}(\mathbf{x})} \tag{S4}$$

wherein by substituting from Equation S3, we have the control policy $\mathrm{g}(\mathbf{x})$. The latent embedding is designed such that the resulting workspace DS $\mathrm{g}(\mathbf{x})$ ensures an accurate replication of the demonstrated positions at the demonstrated velocities when starting from the original starting position. Furthermore, the formulation in Equation S4 ensures global asymptotic convergence at the goal position $\mathrm{B}^{(\mathcal{X})}$ for all starting

positions in workspace $\mathcal{X}$ as

$$\lim_{t\to\infty} \mathbf{x}_t = \lim_{t\to\infty} \psi(\mathbf{u}_t) = \psi\big(\lim_{t\to\infty} \mathbf{u}_t\big) = \psi(\mathbf{1}_{\mathbf{n}}^{\mathbf{T}}) = \overset{*}{\mathbf{x}} \tag{S5}$$

In doing so, we have embedded a demonstration $\mathbf{X}$ as a temporally invariant and robust policy $\mathrm{g}(\mathbf{x})$, fully specified by pair (ψ, k), where ψ defines the diffeomorphic transformation and $k \in \mathbb{R}_+^n$ is a scalar vector.

The same procedure can also be used to model the behaviour in joint space, from a demonstration $\mathbf{Q} = \{\mathbf{q}_i \in \mathcal{Q}\}$ with a goal joint configuration at $\overset{*}{\mathbf{q}}$, where $\overset{*}{\mathbf{x}} = \mathrm{FK}(\overset{*}{\mathbf{q}})$. The formulated joint space DS $\dot{\mathbf{q}} = \mathrm{g}(\mathbf{q})$ models the behaviour in joint space, and can be uniquely represented by the tuple $(\psi, \mathbf{k})$. The DS embedding of the demonstration is parameter-efficient and robust to perturbations, suitable for robot manipulation as demonstrated in (*33*). It must be noted that neither the workspace DS, nor the joint space DS intrinsically model the robot's kinematic constraints such as joint limits and singularities yet.

Learning the kinematics-aware control policy in the proposed framework (refer to Figure S1) is informed by a three-front analysis of the user-robot setup - (A) Modelling the user intent; (B) Identifying robot-specific kinematic constraints; (C) Modelling the robot control policy.

Modelling the User Behaviour

This section presents the method of modelling the user intent by modelling demonstrated workspace behaviour. We also discuss the challenges posed by the robot's kinematic constraints during replication of the demonstrated behaviour in the workspace.

We consider a workspace demonstration $\mathbf{X}$ provided for a $2R$ robot, that serves as an instance of the task specific behaviour to be modelled by the robot. We assume that the provided demonstration (Figure 8A (i)) does not encounter any singular configuration. The objective is to model the demonstrated behaviour across the workspace. Following the oneshot method presented in Section *Oneshot embedding of a demonstration*, we model the workspace behaviour as a globally asymptotically stable DS in the workspace $\dot{\mathbf{x}} = \mathrm{g}^{(\mathcal{X})}(\mathbf{x})$. It is also possible to have multiple workspace demonstrations $\{\mathbf{X_i}\}_{i=1}^{K}$, and use them model the workspace behaviour $\mathrm{B}^{(\mathcal{X})}$ as $\{\mathrm{g}_i^{(\mathcal{X})}\}_{i=1}^{K}$, and is presented in Section *Modelling the Robot Control Policy*.

Figure 8A depicts the vector field in the workspace resulting from the control policy $\dot{\mathbf{x}} = \mathrm{g}^{(\mathcal{X})}(\mathbf{x})$, learnt from a single workspace demonstration. There exist two joint configurations for each workspace position. When starting from the original workspace start position, with the robot in the original joint configuration, the learnt behaviour $\mathrm{g}^{(\mathcal{X})}$ is able to generate an identical replication of the demonstration. However, the modelled workspace behaviour $\mathrm{B}^{(\mathcal{X})}$ does not inherently embed the kinematic constraints imposed due to joint limits, singularity partitions, and (or) holes (workspace regions with no IKS) in workspace $\mathcal{X}$. As a result, the learnt control policy is not robust to perturbations. When deviating significantly from the original positions, even though the joint space aspect may be retained, the learnt behaviour can result in singularity and (or) joint limit violations (Figure 8A (i)).

When starting from the original workspace position, with the joint configuration in another aspect, the learnt workspace behaviour leads to the joint limit violation for the first joint (Figure 8A (ii)). In such a case, the demonstration itself cannot be replicated. Consequently, enforcing kinematic constraints is complicated by the multiplicity of IKS, their distribution across different aspects, and the individual constraints bounding the aspect.

Such behavioural ambiguities are even more pronounced for $3R$ robots. Figure 7 depicts the workspace demonstration on a robot, where the starting position has four IKS, whereas the goal position workspace goal position $\overset{*}{\mathbf{x}}$ has only two IKS. In such cases, the original workspace demonstration cannot be replicated for two of the four possible joint configurations without encountering kinematic violations (Figure 7C (i-iii)). Thus, it is crucial to detect such configurations as there is no possibility of reaching the goal position without violation.

Replicating the user behaviour while being robust to perturbations, and respecting kinematic constraints cannot be directly addressed via $g^{(\mathcal{X})}$ in workspace $\mathcal{X}$. While a mechanism to respect kinematic constraints may be devised to operate in conjunction with $g^{(\mathcal{X})}$, a key issue that remains is that the locus of critical values - singularities in workspace - is hard to derive, and may include nodes, and cusps. Furthermore, the joint limits in workspace do not form a linear variety.

To overcome the shortcomings associated with behavioural modelling in workspace $\mathcal{X}$, we instead focus on modelling the behaviour in joint space joint space $\mathcal{Q}$. Modelling in joint space offers an additional advantage when working with 3R robots. For $3R$ manipulators, as discussed in Section , the singularities are independent of q_1, allowing for singularity-free behaviour replication in the reduced space parameterized by (q_2, q_3) without any loss of information on singularity. Consequently, the joint limits of q_1 can be handled independently, ensuring constraint compliance without altering the behaviour in (q_2, q_3)-space.

The joint space behaviour, hereby represented as $g^{(\mathcal{Q})}$, will be modelled as a library of joint space DSs, and incrementally generalised (refer to Section *Modelling the Robot Control Policy*).For now, we can consider $\dot{\mathbf{q}} = g^{(\mathcal{Q})}(\mathbf{q})$ to be the control policy in the joint space, learnt using any one of the IKS of the workspace demonstration.

In the proposed framework, the modelled workspace behaviour serves as a proxy of the user intent, whereas the model of joint space behaviour embedded with kinematic constraints, serves as the robot control policy. In doing so, the resulting robot behaviour mirrors the user intent while respecting kinematic constraints.

Robot Categorisation

To comprehensively embed kinematic constraints in the control policy of the robot from the *user–robot* setup, and enable extension to multiple robots, we propose an exhaustive homotopy-based classification of the possible singularities for noncuspidal $3R$ robots. This classification is instrumental to deriving a stable control law that respect's the robot's kinematic constraints (in Section 1). As a result, the framework is robot-agnostic when learning user behaviour, while still generating robot-specific feasible behaviours during execution.

Singularities of noncuspidal $3R$ robots

If the trajectory obtained from $\mathrm{g}^{(Q)}$ crosses singularities or joint limits before reaching the goal configuration, we propose a method (refer to Section 1) to navigate along the constraint boundary until reaching a feasible joint configuration that lies on the generated trajectory. Once the trajectory re-enters the feasible aspect, it remains constraint-compliant for the rest of the motion, ultimately reaching the goal configuration without further violations. This approach ensures that the trajectory generalizes the expert demonstrated behaviour while adhering to kinematic constraints. Understanding the properties of singularities is essential, as the behaviour near singularities must be deterministic to guarantee convergence to a specific feasible joint configuration. To achieve this, we begin by exhaustively analysing the singularities for noncuspidal $3R$ robots.

For noncuspidal $3R$ robots, the mapping $r : (q_2, q_3) \mapsto (t_2, t_3)$ is a bijection for $q_{2,3} \in (-\pi, \pi)$ (refer Section). As the number of solutions for a fixed value of t_2 (resp. t_3) is at most four (resp. two), a joint path along a q_2- (resp. q_3-) generator may intersect the generic loops at most four (resp. two) times. A similar bound applies for nongeneric robots except for the configurations satisfying the nongenericity condition. At such configurations, the path along the generator coincides with the singularity, and we get infinite solutions.

Topological and differential properties

Using the information related to the degrees of each variable in Jacobian determinant $\det \mathbf{J}(\mathbf{q})$, singularities $\mathcal{S}$ can be classified as the union of branches that form loops on $\mathbb{T}^2$. For example, the singularities $\mathcal{S}$ in Figure 2A (ii-iii) can be classified as $2(1,0) + 2(0,1)$ with a total of four branches, two of them encircle q_2 generator and the remaining two encircle the q_3 generator. The classification of singularities $\mathcal{S}$ can be

made in multiple ways depending on the purpose of the analysis. (*29*) and (*30*) investigated the number of aspects for a given singularities $\mathcal{S}$ and thus used the enumeration of loops to establish equivalence among 3R robots. In the presented work, we aim to leverage the properties to design robust behaviour near singularity surfaces and, thus, classify the robots based on the encircling nature of the branches and the number of horizontal/vertical turning points on them. This is done as two homeomorphic singularities may require different treatment for the behaviour near the singularity.

Using Equation S2, the upper bound of the turning points can be calculated. (*30*) showed that, at most, eight real horizontal turning points can exist for a generic robot. A similar analysis can be done for vertical turning points. Classification of robots based on the turning points in addition to encircling characteristics have not been reported earlier to the best of the authors' knowledge.

Proposition 1. *The singularity expression of a generic 3R robot has at most three factors.*

Proof. The degrees of t_2 and t_3 in the Jacobian determinant $\det \mathbf{J}(\mathbf{q})$ are known to be 2 and 4, respectively. This allows for at most two intersections along a generator of q_3 and at most four intersections along the generator of q_2. Consequently, the joint space can be partitioned into at most eight aspects when all singularities are aligned with the torus generators formed by q_2 and q_3. Since each factor of $\det \mathbf{J}(\mathbf{q})$ divides the torus into at least two distinct regions, and a maximum of eight such regions is possible, the number of distinct factors is bounded above by three ($2^3 = 8$). An example robot exhibiting this structure is illustrated in Figure S3. □

Each factor f_i of Jacobian determinant $\det \mathbf{J}(\mathbf{q})$ may have one, two or four closed loops on the $\mathbb{T}^2$ referred to as branches of the factor.

Proposition 2. *The vertical turning points for any 3R robot are either at most twelve finite points or infinite points in* $\mathbb{R}$.

Proof. By eliminating t_2 from the rational parameterization of the singularity surface $\det \mathbf{J}(\mathbf{q}) = 0$ and the turning condition $\gamma \, \partial \det \mathbf{J} / \partial q_3 = 0$, we obtain a 14th-degree polynomial in t_3. This polynomial factors as $(t_3^2 + 1) \cdot p_2(t_3)$, where the first factor has only complex roots. Disregarding this, the remaining real polynomial is of degree 12, implying at most 12 real vertical turning points for any non-degenerate branch. If the branch is degenerate, then Equation S2 admits infinitely many solutions. □

Proposition 3. *The horizontal turning points for any branch of singularities of a 3R robot are either zero, two, or infinite.*

Proof. We prove this by considering the possible topologies of singularity branches $\mathcal{S}$.

Case $(n, 1)$*:* These branches encircle the q_3-generator once. Suppose, for contradiction, that there exist

two horizontal turning points $\mathbf{h}_1, \mathbf{h}_2$ on such a branch. Then, for any intermediate q_3 value between $\mathbf{h}_1$ and $\mathbf{h}_2$, a path along the q_2-generator would intersect the branch at least three times, contradicting the known upper bound of two intersections along q_2. Hence, no such horizontal turning points can exist on branches of class $(n, 1)$. A similar contradiction arises for any number greater than two.

Case $(n, 0)$*:* These include closed loops (for $n = 0$) and branches encircling the q_2-generator. By the Weierstraß theorem, generic branches in this class, such as $(0, 0)$ or $(1, 0)$, must have at least two horizontal turning points. If a third point $\mathbf{h}_3$ existed, then for some nearby value along q_2, a path would intersect the branch at least three times, violating the same upper bound. An exception occurs in nongeneric $(1, 0)$ branches where $\det \mathbf{J}(\mathbf{q})$ factors such that one factor is of the form $\cos(q_3 + k_1) + k_2$. In that case, there could be infinitely many horizontal turning points.

Thus, for any branch, the number of horizontal turning points is either zero, two, or infinite. □

The singular configurations of the robot can be characterised by the branch types of the factors present in the determinant of the Jacobian (*29, 30*). For instance, if the factor has two branches of type $(1, 0)$, the factor description can be expressed as $2(1, 0)$. If there are two factors, one with description $2(1, 0)$ and another with description $2(0, 1)$, the singularity is collectively described as $2(1, 0) + 2(0, 1)$

Robot categories

For a $2R$ robot, a single type of singularity $2(1, 0)$ exists and is invariant of the link lengths. For a $3R$ robot, eight homotopy classes of generic robots are known (refer to (*29, 30*)). The presented work extends the classes to nongeneric noncuspidal robots and further classifies the singularities based on the number of turning points. (*46*) conjectured that the homotopy class $3(0, 0)$ and $4(0, 0)$ do not exist (refer to Conjecture 4). The proof of this conjecture has not yet been presented. Still, as the proposed work focuses on only noncuspidal robots, we present an extension to the proposition from (*46*) to exclude the singularities with homotopy class $n(0, 0), n \geq 2$.

Proposition 4. *Any* $3R$ *robot belonging to the* $n(0, 0)$ *homotopy class with* $n > 1$ *must be cuspidal.*

Proof. Ignoring joint limits, we invoke Proposition 4 from Salunkhe (*46*), which states: *A* $3R$ *robot in the* $n(0, 0)$ *homotopy class with* $n > 1$ *must be a quaternary robot*, i.e., a robot that admits at least one pose in the workspace with four inverse kinematic solutions (IKS).

Since the Jacobian determinant $\det \mathbf{J}(\mathbf{q})$ is a C^∞ function, neighboring aspects must have opposite signs of $\det \mathbf{J}(\mathbf{q})$. Therefore, any regular configuration must have an equal number of IKS with positive and negative signs of the determinant.

In the $n(0,0)$ class, there are $(n+1)$ aspects, n interior aspects formed by $(0,0)$ loops, all with the same sign of $\det \mathbf{J}(\mathbf{q})$, and one unique exterior aspect with the opposite sign. Without loss of generality, assume the exterior aspect has a positive determinant. Then, for $n > 1$, the robot has four IKS, of which two must have positive and two negative signs. However, only one aspect has positive sign, forcing both positive IKS to lie within it. Hence, the robot is cuspidal, it contains multiple IKS of the same sign within the same aspect.

Previous works enumerated each branch of singularities $\mathcal{S}$ as (n_1, n_2); we extend it by adding the information on the number of horizontal and vertical turning points. Each branch of the singularity is enumerated as $(n_1, n_2)[n_3, n_4]$, which suggests that the given branch encircles the q_2- (resp. q_3-) generator n_1 (resp. n_2) times, and has n_3 (resp. n_4) horizontal (resp. vertical) turning points wherein $n_1 \leq 2, n_2 \leq 1, n_3 \leq 8$ or $\infty, n_4 \leq 12$ or ∞. Each singularity can now be represented as a union of these branches, and thus a singularity can be written as $\Sigma_{i=1}^{k}(n_{1i}, n_{2i})[n_{3i}, n_{4i}]$, where i denotes each branch of the singularity. For example, the robot singularity in Figure S2A is denoted as $2(1,0)[2,0] + 2(1,0)[\infty, 0]$ as it has two branches of type $(1,0)[2,0]$ and two of $(1,0)[\infty, 0]$.

Using the aforementioned novel enumeration of the branches, Propositions 4, and 5, the singularities of noncuspidal $3R$ robot can be classified into six categories:

1. Non-loop, non-intersecting branches with ONLY horizontal turning points
2. Non-loop, non-intersecting branches with NO horizontal turning points
3. Non-loop, intersecting branches
4. Non-loop, non-intersecting branches with folds
5. Loop non-intersecting branch
6. Intersecting branches including a loop

Category I: Non-loop, non-intersecting branches with only horizontal turning points: The singularity of robots in this category is made out of the union of branches that only have horizontal turning points, and do not intersect. Few types of singularities exhibited by robots in this category: $2(1,0)[2,0] + 2(1,0)[\infty, 0]$ (refer to Figure S2(A)), $4(1,0)[2,0]$.

Category II: Non-loop, non-intersecting branches with no horizontal turning points: The singularity of

robots in this category is made out of the union of branches that do not have any horizontal turning points, and the branches that do not intersect. Few types of singularities exhibited by robots in this category: $(0,1)[0,4]+(0,1)[0,2]$ (refer to Figure S2(B)), $(1,1)[0,2]+(1,1)[0,0]$ (refer to Figure S2(E)), $2(0,1)[0,a>0], 2(1,1)[0,a>0]$, where a is the finite number of vertical turning points on a branch of singularity.

It can be seen that the joint limits further divide the joint space, creating more partitions, and methods different from *Category I* are needed to be employed to investigate connectivity between two configurations (as discussed in Section 12).

Category III: Non-loop, intersecting branches:

The singularity of robots in this category is made out of the union of branches that do not belong to the $(0,0)$ homotopy class and the branches that intersect. Few types of singularities exhibited by robots in this category: $2(1,0)[\infty,0]+2(0,1)[0,*]$ (refer to Figure S2(F)), $4(1,0)[\infty,0]+2(0,1)[0,\infty]$ (refer to Figure S3, $2(1,0)[\infty,0]+2(1,1)[0,0]$ (refer to Figure S2(H)). This category consists of non-generic robots only, and the $(1,0)[\infty,0]$ branch comes from a factored Jacobian determinant $\det\mathbf{J}(\mathbf{q})$ and divides joint space $\mathcal{Q}$ in at least four aspects. The singularities in Figure 2A (ii) is one such example of $2(1,0)[\infty,0]+2(0,1)[0,2]$. We also show an example of a robot with eight aspects (without joint limits), which is the maximum limit of an aspect without joint limits for *any* $3R$ robot (refer to Figure S3). Interestingly, the intersecting singularities must have $(1,0)[\infty,0]$ branch as the maximum number of intersections a path can have along the q_2-generator that is not a singularity itself is two.

Category IV: Non-loop, Non-intersecting branches with folds: The singularity of robots in this category is made out of the union of branches that belong to the $(1,0)[2,*]$ homotopy class with at least one branch with $(1,0)[2,>0]$ homotopy classification, and the branches do not intersect. Here, a is a non-zero definite number of vertical turning points. These singularities are the most challenging when defining behaviour near the boundaries, as they have branches that encircle the q_2 generator; however, for some regions, they locally exhibit loop-like behaviour. The sense of direction may not remain constant while traversing between any two configurations on the branch. For such singularities, the sense of direction has to be isolated in regions by identifying the turning points along the branch. Similar to the loop singularities, the relative position of the configurations in an aspect may change due to the branches intersecting with joint limits, which further complicates the motion along a branch of singularity of this class.

The theoretical upper bound for the vertical turning points on a branch is twelve, but we could only find examples with two and four vertical turning points on a branch. Figure S8 shows an example of robot classes with $(1,0)[2,0] + (1,0)[2,2]$ singularities. Figure S2 shows singularities where one branch (bottom) has two horizontal and two vertical turning points and encircles the q_2- generator. Figure S8 shows singularities where one branch (top) has two horizontal and two vertical turning points and encircles the q_2- generator, while the other branch (bottom) has four vertical turning points.

Category V: Loop non-intersecting branch:

The singularity of robots in this category is made out of a single branch of the $(0,0)$ homotopy class. Few types of singularities exhibited by robots in this category: $(0,0)[2,2]$ and $(0,0)[2,6]$(refer to Figure S2C). This branch is guaranteed to have at least two horizontal and vertical turning points. The motion along the loop boundaries differs from the above categories. In this category, the sense of motion is defined in a clockwise or counterclockwise direction. We note that the robot shown in Figure S2C is a cuspidal robot as we could not find a noncuspidal robot belonging to this category, although no proof exists to suggest the nonexistence of noncuspidal robots belonging to this category.

Category VI: Intersecting branches including a loop: The singularity of robots in this category is made out of the union of branches that belong to the $(0,0)[2,*]$ and $(1,0)[\infty,0]$ types, and the branches intersect. This class has only one type of nongeneric robot with $2(1,0)[\infty,0] + (0,0)[2,2]$ class of singularities. The $(1,0)[\infty,0]$ branch comes from a factored $\det \mathbf{J}(\mathbf{q})$ and divides $\mathcal{J}$ in at least four aspects. This case requires the consideration of both intersecting (Category III) and loop (Category V) singularities. An example illustration of this type of robot is given in Figure S2(G), where the loop is complete in the $(-\pi,\pi)$ range and in Figure S7, where the loop is split due to joint limits.

Algorithm 1. Branch types in factor f_i

Solve $\mathrm{f} = 0, \partial \mathrm{f}/\partial q_2 = 0 \Rightarrow Q_h$ (horizontal turning points)

Solve $\mathrm{f} = 0, \partial \mathrm{f}/\partial q_3 = 0 \Rightarrow Q_v$ (vertical turning points)

If $|Q_h| > 0$ and $Q_v = \emptyset$:

Return $(1,0)[\geq 2,0]$

Else if $Q_h = \emptyset$ and $|Q_v| > 0$:

Check if branch encircles the q_2-generator

If $\exists\, \mathbf{q} \in Q_v$ s.t. path $\mathbf{q}(q_2)$ has no solution:

Return $(0,1)[0,*]$

Else: Return $(1,1)[0,*]$

Else if $|Q_h| > 0$ and $|Q_v| > 0$:

If $|Q_h| = 2$: Return $(0,0)[2,\geq 2]$

Else: Return $(1,0)[2,*]$

Else:

Compute $J = \{\mathbf{q} \mid \mathrm{f}(\mathbf{q}(q_2 = \pi)) = 0\}$

If $|J| > 2$: Return $(2,1)[0,0]$

Else: Return $(1,1)[0,0]$

To perform the categorisation of a robot into one of the six categories, we present Algorithm 1 that analyses all the branches associated with a single factor f_i, based on the encirclement of the generators, combined with the number of horizontal turning points Q_h, and vertical turning points Q_v. By repeating this for each factor of Jacobian determinant $\det \mathbf{J}(\mathbf{q})$ of a robot, we can comprehensively generate a description of the robot.

If there are nonzero horizontal and zero vertical turning points, the factor is a union of $(1,0)$ branches, as discussed in Proposition 4. If only vertical turning points exist, the factor cannot have a $(1,0)$ or a $(0,0)$ branch. Similarly, nonzero vertical turning points cannot exist in $(2,1)$ branch, and so the only possibility is that the factor is $2(0,1)$ or $2(1,1)$. If the given factor has no horizontal turning points, then the singularities may belong to either $(0,1)$ or $(1,1)$ or $(2,1)$. In this case, the fact that singularities with $(0,1)$ homotopy class does not encircle the q_2-generator is leveraged to identify such singularities.

In the case that both horizontal and vertical turning points exist, the only classes that can be factors of type $2(1,0)$ wherein at least one branch has non-zero vertical turning points or $(0,0)[2,*]$. In such cases, the classification can be done based on the number of horizontal turning points. If a factor is of a $(0,0)$ homotopy class, then it must have strictly two horizontal turning points as shown in Proposition 4, while a factor with description $2(1,0)$ homotopy class will strictly have four horizontal turning points. If there are no horizontal/vertical turning points, then the factor is either $2(1,1)$ or $2(2,1)$ class. As the branch with $2(2,1)$ class have two branches that encircle q_2-generator twice, the number of intersections of the branch with any path along the q_3-generator will be four. Thus, the singularity class of the factor can be determined by the number of roots of the factor at $q_2 = \pi$.

In this section, we have identified six categories that exhaustively classify generic and nongeneric $3R$ noncuspidal robots by using the homotopy-based classification with additional properties. This novel

categorisation is instrumental to autonomously embed kinematic constraints into the control policy as discussed in Sections 1, and 12.

Behaviour near Kinematic Constraints

The current joint space behaviour $\mathrm{g}^{(Q)}$, though stable and converges at the goal configuration $\overset{*}{\mathbf{q}}$, does not yet explicitly embed the kinematic constraints prior to convergence. For a given instantaneous joint configuration $\mathbf{q}$, the trajectory mapped by following the DS $\dot{\mathbf{q}} = \mathrm{g}^{(Q)}(\mathbf{q})$ has guaranteed convergence at the goal configuration $\overset{*}{\mathbf{q}}$. However, this trajectory may violate the joint limits or encounter singular configuration(s) enroute.

To avoid such scenarios, the modelled behaviour $\mathrm{g}^{(Q)}$ is selectively overlayed with a *near-boundary* control policy around the joint limits and singularities that enclose the aspect. Effectively, this policy ensures that the start configuration $\mathbf{q}$, the goal configuration $\overset{*}{\mathbf{q}}$, and the intermediate trajectory, are strictly within the bounds of a single aspect in the joint space when $\mathrm{g}^{(Q)}$ may lead to a violation. We present the near-boundary behaviour for noncuspidal $3R$ robots in Section 5.

Boundary regions for $2R$ robots

The kinematic constraints in the joint space of a $2R$ robot are due to the first joint's limits $q_1^{\times} = \{q_1 = \underline{q_1}, q_1 = \overline{q_1}\}$, a singularity boundary $S^{\times} = \{q_2 = 0\}$, and the second joint's limits $q_2^{\times} = \{q_2 = \underline{q_2}, q_2 = \overline{q_2}\}$. Each of the two aspects is the joint space are bound by these constraints.

We begin by defining regions $(\mathcal{B}_{q_1^{\times}}, \mathcal{B}_{q_2^{\times}})$ that encapsulate the neighbourhoods of the joint limits in joint space $\mathcal{Q}$

$$\begin{aligned} \mathcal{B}_{q_1^{\times}} &= \{\mathbf{q} \in \mathcal{Q} \mid \mathbf{q}(q_1) < \underline{q_1} + \epsilon_q \vee \mathbf{q}(q_1) > \overline{q_1} - \epsilon_q\} \\ \mathcal{B}_{q_2^{\times}} &= \{\mathbf{q} \in \mathcal{Q} \mid \mathbf{q}(q_2) < \underline{q_2} + \epsilon_q \vee \mathbf{q}(q_2) > \overline{q_2} - \epsilon_q\} \end{aligned} \tag{S6}$$

where $\epsilon_q > 0$ is the safety margin (in radians) around the joint limits. The joint limit safety margin is next used to calculate the singularity threshold $\epsilon_S = |\det \mathbf{J}(q_1 = 0, q_2 = \epsilon_q)|$, which is used to define the neighbourhood region of singularity $\mathcal{B}_{S^{\times}}$

$$\mathcal{B}_{S^{\times}} = \{\mathbf{q} \in \mathcal{Q} \mid \epsilon_S > |\det \mathbf{J}(\mathbf{q})|\}$$

Collectively, $\mathcal{B}_{q_1^{\times}}, \mathcal{B}_{q_2^{\times}}$, and $\mathcal{B}_{S^{\times}}$ encapsulate the regions of joint space $\mathcal{Q}$ in vicinity of the joint limits and singularities. Since each aspect in joint space $\mathcal{Q}$ is in itself bounded by the joint limits and singularities, the region near boundaries of the aspect belongs to a subset of $\mathcal{B}_{q_1^{\times}} \cup \mathcal{B}_{q_2^{\times}} \cup \mathcal{B}_{S^{\times}}$. While following $\mathrm{g}^{(Q)}$, if the robot assumes a configuration in the boundary region, the continued usage of $\mathrm{g}^{(Q)}$ may result in a joint trajectory that crosses singularities or joint limits before reaching the goal configuration.

Identifying boundary crossings for $2R$ robots

Consider a trajectory $\mathbf{Q}$ that can be generated by $\mathrm{g}^{(Q)}$ from a query point $\mathbf{q} \in \mathcal{A}$. Due to lack of embedded constraints, it is possible for this trajectory to cross the boundaries $(q_1^{\times}, q_2^{\times}, S^{\times})$ enclosing the current aspect. However, it must be noted that both the query $\mathbf{q}$ (start of trajectory) and the goal configuration $\overset{*}{\mathbf{q}}$ lie in the same aspect $\mathcal{A}$. This implies that if a trajectory crosses boundary(ies) and transitions into different aspect(s), it must eventually return to the original aspect and cross some boundary one last time to achieve convergence at the goal configuration.

Proposition 5. *Let $\mathbf{q}_t \in \mathcal{A}$ and $\overset{*}{\mathbf{q}} \in \mathcal{A}$ be the initial and goal configurations in a joint-space aspect $\mathcal{A} \subset \mathcal{Q}$, enclosed by the set of boundaries $\{q_1^{\times}, q_2^{\times}, S^{\times}\}$. Let $\mathbf{Q} = \{\mathbf{q}(t)\}_{t\geq 0}$ be a trajectory generated by the control policy $g^{(Q)}$. Then, regardless of how many times $\mathbf{Q}$ leaves and re-enters $\mathcal{A}$, there exists a time $T < \infty$ such that $\mathbf{q}(t) \in \mathcal{A}$ for all $t > T$, i.e., the trajectory must make a final crossing into $\mathcal{A}$ to converge to $\overset{*}{\mathbf{q}}$.*

Proof. The control policy $g^{(Q)}$ is defined as a globally asymptotically stable dynamical system with respect to the attractor $\overset{*}{\mathbf{q}} \in \mathcal{A}$. Thus, for any initial configuration $\mathbf{q}_t \in \mathcal{Q}$, the trajectory $\mathbf{Q}$ satisfies $\lim_{t\to\infty} \mathbf{q}(t) = \overset{*}{\mathbf{q}}$. Since the goal lies within $\mathcal{A}$, and the aspect is bounded by joint limits and singularity manifolds (i.e., the set $\mathcal{B}$), convergence requires that the trajectory eventually remains within $\mathcal{A}$. Therefore, there exists a finite time T such that $\mathbf{q}(t) \in \mathcal{A}$ for all $t > T$.

□

Consequently, the generated trajectory $\mathbf{Q}$ can be segregated into three segments - the initial constraint-compliant portion originating at $\mathbf{q} \in \mathcal{A}$ before the first boundary violation and exit from $\mathcal{A}$, the final constraint-compliant portion in $\mathcal{A}$ after the last violation resulting in convergence at goal, and the intermediate region which begins with an exit from $\mathcal{A}$ and terminates at the last reentry back into the this aspect. The intermediate portion of the trajectory is not constraint compliant, and may encounter multiple violations. For this region, we propose a control law that overlays $\mathrm{g}^{(Q)}$ beginning in the boundary region of the first exit from $\mathcal{A}$, to reactively navigate along the constraint boundary within the boundary region, until reaching the last reentry joint configuration. This approach ensures that the trajectory remains as faithful as possible to the expert behaviour, without exiting the aspect and adhering to the kinematic constraints.

Algorithm 2. First Exit and Last Entry configurations for $2R$ Robots

Input: Joint trajectory $\mathbf{Q}$

Detect singularity crossings

$Z \leftarrow \mathrm{sign}(\det \mathbf{J}(\mathbf{q} \in \mathbf{Q})) \cdot \mathrm{sign}(\det \mathbf{J}(\overset{*}{\mathbf{q}}_{\mathrm{goal}}))$

$\Rightarrow$ Assign +1 for configurations in same aspect as goal,

0 for singularities, and -1 for other aspects

Detect joint limit violations

$V \leftarrow \{i \mid \mathbf{Q}_i(q_1) \notin [\underline{q_1}, \overline{q_1}] \ \vee \ \mathbf{Q}_i(q_2) \notin [\underline{q_2}, \overline{q_2}]\}$

For all $i \in V$, set $Z_i \leftarrow -1$

Identify transition indices

first exit $\leftarrow \max\{i \mid \forall j < i, \ Z_j = +1\}$

last entry $\leftarrow \min\{i \mid \forall j > i, \ Z_j = +1\}$

Return: $(\mathbf{Q}_{\text{first exit}}, \ \mathbf{Q}_{\text{last entry}})$

To model the behaviour for the intermediate non-compliant segment of $\mathbf{Q}$, we reduce our analysis to only the first and last aspect boundary crossings - the first exit configuration from the current aspect ($\mathbf{q}_{\text{first exit}}$) and the last entry configuration ($\mathbf{q}_{\text{last entry}}$) - respectively marking the beginning and end of the non-compliant segment. While it is possible that crossing over from the current aspect and back into it occurs multiple times along $\mathbf{Q}$, considering only the first exit and last entry to determine the behaviour prevents limit cycles and self-intersections in the boundary region. The exit and entry points can be determined on the basis of discontinuity in aspect criteria i.e. $\text{sign}(\det \mathbf{J}(\mathbf{q}))$. The procedure to determine these joint configurations for $2R$ robots is presented in Algorithm 2.

Motion along boundaries for $2R$ robots

The behaviour to reach $\mathbf{q}_{\text{last entry}}$ from $\mathbf{q}_{\text{first exit}}$ involves traversing along the normal to the gradient of the boundaries. During this traversal, the boundary region of a configuration $\mathbf{q} \in \mathcal{Q}$ is identified as $\mathcal{B}(\mathbf{q}) \in \{\mathcal{B}_{q_1^\times}, \mathcal{B}_{q_2^\times}, \mathcal{B}_{S^\times}\}$. The overall behaviour depends on the combination of $\mathcal{B}(\mathbf{q}_{\text{last entry}})$ and $\mathcal{B}(\mathbf{q}_{\text{first exit}})$, as the predicted trajectory can cross the aspect boundary in multiple ways. For a given combination of $(\mathbf{q}_{\text{last entry}}, \mathbf{q}_{\text{first exit}})$ we define a track cycle $\text{C}(\mathbf{q}_{\text{last entry}})$ which associates joint configurations in each boundary region of the relevant aspect with a goal velocity direction $\mathbf{v}(\mathbf{q}) = \text{C}(\mathcal{B}(\mathbf{q}))$ and guides the robot safely to $\mathbf{q}_{\text{last entry}}$ configuration. The normal to the violation boundary $\mathbf{t}$ is determined using Algorithm 3. The near-boundary control policy $\text{g}^{(\mathcal{B})}(\mathbf{q}, \mathbf{q}_{\text{last entry}})$ that determines the joint velocity for a point $\mathbf{q}$ in the

boundary $\mathcal{B}(\mathbf{q})$ on an aspect is described by the DS

$$\dot{\mathbf{q}} = \mathrm{g}^{(\mathcal{B})}(\mathbf{q}, \mathbf{q}_{\text{last entry}}) = \|\mathbf{q}_{\text{last entry}} - \mathbf{q}\| \frac{\mathbf{v} \cdot \mathbf{t}^{\perp}}{\|\mathbf{v}\|} \mathbf{t}^{\perp} \tag{S7}$$

where $\mathbf{t}^{\perp}$ is the normal to $\mathbf{t}$.

Algorithm 3. Normal vector to the constraint boundary for $2R$ Robot

Input: Configuration $\mathbf{q}$ near aspect boundary

Boundary normal to $q_1^{\times}$

If $\underline{q_1} \leq \mathbf{q}(q_1) < \underline{q_1} + \epsilon_q$:

$\mathbf{t}_{q_1^{\times}} \leftarrow [1,\ 0]^T$

Else if $\overline{q_1} - \epsilon_q \leq \mathbf{q}(q_1) < \overline{q_1}$:

$\mathbf{t}_{q_1^{\times}} \leftarrow [-1,\ 0]^T$

Boundary normal to $q_2^{\times}$

If $\underline{q_2} \leq \mathbf{q}(q_2) < \underline{q_2} + \epsilon_q$:

$\mathbf{t}_{q_2^{\times}} \leftarrow [0,\ 1]^T$

Else if $\overline{q_2} - \epsilon_q \leq \mathbf{q}(q_2) < \overline{q_2}$:

$\mathbf{t}_{q_2^{\times}} \leftarrow [0,\ -1]^T$

Boundary normal to singularity surface $S^{\times}$

$\mathbf{t}_{S^{\times}} \leftarrow \mathrm{sign}(\det \mathbf{J}(\mathbf{q})) \cdot [\nabla_{q_1,q_2} \det \mathbf{J}(\mathbf{q})]^T$

Return: $(\mathbf{t}_{q_1^{\times}},\ \mathbf{t}_{q_2^{\times}},\ \mathbf{t}_{S^{\times}})$

The track cycle behaviour that connects $\mathbf{q}_{\text{first exit}}$ and $\mathbf{q}_{\text{last entry}}$ is determined using Algorithm 4. An example is presented in Figure S9, wherein the direction in red specifies the behaviour in $\mathcal{B}_{q_1^{\times}}$, while direction in blue specifies the behaviour in $\mathcal{B}_{S^{\times}}$. In Figure S9, the path $\mathbf{Q}$ generated by $\mathrm{g}^{(Q)}$, encounters the first violation at $\mathbf{q}_{\text{first exit}}$, $\mathcal{B}(\mathbf{q}_{\text{first exit}}) = \mathcal{B}_{S^{\times}}$. The last configuration in $\mathbf{Q}$ in violation of the kinematic constraints is $\mathbf{q}_{\text{last entry}}$ $\mathcal{B}(\mathbf{q}_{\text{last entry}}) = \mathcal{B}_{S^{\times}}$, and $\mathbf{q}_{\text{first exit}}(q_1) > \mathbf{q}_{\text{last entry}}(q_1)$. To generate the resulting trajectory, we start at $\mathbf{q} = \mathbf{q}_{\text{first exit}}$ and terminate at $\mathbf{q}_{\text{last entry}}$. At the start, $\mathbf{v}_{\mathbf{q}_{\text{first exit}}} = [\mathbf{q}_{\text{last entry}}(q_1) - \mathbf{q}_{\text{first exit}}(q_1), 0]^T$ forces the joint velocity to be along negative q_1 axis (blue). Progressively, the joint configuration, $\mathbf{q}$, approaches $\mathcal{B}_{q_1^{\times}}$. When $\mathbf{q} \in \mathcal{B}_{q_1^{\times}}$, $\mathbf{v} = [0, -\mathbf{t}_{\text{last entry}}(q_2)]^T$ forces the joint velocity along positive q_2 axis directed parallel to $q_1^{\times}$ while progressively approaching $\mathcal{B}_{q_2^{\times}}$ (blue). Lastly, when $\mathbf{q} \in \mathcal{B}_{q_2^{\times}}$, $\mathbf{v} = [\mathbf{q}_{\text{last entry}}(q_1) - \mathbf{q}(q_1), 0]^T$

forces the joint velocity along q_1 axis and approach $\mathbf{q}_{\text{last entry}}$ configuration (red). The overall behaviour encapuslated by $\mathrm{C}(\mathbf{q}_{\text{last entry}})$ is thus

$$(\mathbf{q}_{\text{first exit}} \in \mathcal{B}_{S^\times}) \rightarrow (\mathbf{q} \in \mathcal{B}_{q_1^\times}) \rightarrow (\mathbf{q} \in \mathcal{B}_{q_2^\times}) \rightarrow (\mathbf{q}_{\text{last entry}})$$

The track cycle $\mathrm{C}(\mathbf{q}_{\text{last entry}})$ determines the behaviour to reach $\mathbf{q}_{\text{last entry}}$ from any point in any of the boundary regions in the current aspect.

Algorithm 4. Algorithm: Track Cycle for a $2R$ Robot

Input: Configuration $\mathbf{q}$, aspect boundary normals

Output: Velocity direction vector $\mathbf{v}$ along tangential track cycle

Compute $\mathbf{t}_{\text{first exit}},\ \mathbf{t}_{\text{last entry}},\ \mathbf{t}$ using Algorithm 3

Case 1: $\mathbf{q}_{\text{last entry}} \in \mathcal{B}_{S^\times} \cup \mathcal{B}_{q_2^\times}$

If $\mathbf{q} \in \mathcal{B}_{q_1^\times}$:

$\mathbf{v} \leftarrow [0,\ -\mathbf{t}_{\text{last entry}}(q_2)]^T$

Else if $\mathbf{q} \in \mathcal{B}_{q_2^\times} \cup \mathcal{B}_S^\times$ and $\mathbf{t}(q_3)\mathbf{t}_{\text{last entry}}(q_3) < 0$:

$\mathbf{v} \leftarrow [\mathbf{q}_{\text{last entry}}(q_1) - \mathbf{q}_{\text{first exit}}(q_1),\ 0]^T$

Else if $\mathbf{q} \in \mathcal{B}_{q_2^\times} \cup \mathcal{B}_S^\times$ and $\mathbf{t}(q_3)\mathbf{t}_{\text{last entry}}(q_3) \geq 0$:

$\mathbf{v} \leftarrow [\mathbf{q}_{\text{last entry}}(q_1) - \mathbf{q}(q_1),\ 0]^T$

Case 2: $\mathbf{q}_{\text{last entry}} \in \mathcal{B}_{S^\times} \cup \mathcal{B}_{q_1^\times}$

If $\mathbf{q} \in \mathcal{B}_{q_2^\times} \cup \mathcal{B}_S^\times$:

$\mathbf{v} \leftarrow [-\mathbf{t}_{\text{last entry}}(q_1),\ 0]^T$

Else if $\mathbf{q} \in \mathcal{B}_{q_1^\times}$ and $\mathbf{t}(q_2)\mathbf{t}_{\text{last entry}}(q_2) < 0$:

$\mathbf{v} \leftarrow [0,\ \mathbf{q}_{\text{last entry}}(q_2) - \mathbf{q}_{\text{first exit}}(q_2)]^T$

Else if $\mathbf{q} \in \mathcal{B}_{q_1^\times}$ and $\mathbf{t}(q_2)\mathbf{t}_{\text{last entry}}(q_2) \geq 0$:

$\mathbf{v} \leftarrow [0,\ \mathbf{q}_{\text{last entry}}(q_2) - \mathbf{q}(q_2)]^T$

It is possible that a joint configuration may lie simultaneously in two boundary regions - namely $\mathcal{B}_{q_1^\times} \cap \mathcal{B}_{q_2^\times}$ and $\mathcal{B}_{q_1^\times} \cap \mathcal{B}_{s^\times}$. Such configurations have two potential candidates for the boundary normal $\mathbf{t}$. If $\mathbf{q}_{\text{last entry}}$ is one such configuration, then both normals qualify and any one can be used as $\mathbf{t}_{\text{last entry}}$ to determine the track cycle behaviour. Otherwise, the joint configurations in the overlapping regions,

constitute the transition from one boundary region to the next - as determined by the track cycle. The ideal goal velocity direction $\mathbf{v}_{\text{ideal}}$ is the direction which has a non-negative projection on all $\mathbf{t}$ candidates. The track cycle design ensures that there is a single direction that matches this criteria and maintains a non-repeating sequence of boundary regions to traverse, terminating at $\mathbf{q}_{\text{last entry}}$. The ideal boundary normal $\mathbf{t}_{\text{ideal}}$ is the boundary normal associated with this chosen direction. The resulting joint velocity is calculated using Algorithm 5.

Algorithm 5. Joint Velocity for $2R$ Robot in Overlapping Boundary Regions

Input: Configuration $\mathbf{q} \in \mathcal{B}_{q_1^\times} \cap \mathcal{B}_{q_2^\times}$ or $\mathbf{q} \in \mathcal{B}_{q_1^\times} \cap \mathcal{B}_{S^\times}$

Output: Desired joint velocity $\dot{\mathbf{q}}$

Call Algorithm 3 to compute all boundary normals:

$\{\mathbf{t}\} \leftarrow$ set of normals at $\mathbf{q}$

Determine all candidate velocity vectors:

$\{\mathbf{v}\} \leftarrow$ track-cycle velocities from classification $\mathrm{C}(\mathbf{q}_{\text{last entry}})$

Determine ideal goal velocity direction

$\mathbf{v}_{\text{ideal}} \leftarrow \{\mathbf{v} \in \{\mathbf{v}\} \mid \mathbf{v} \cdot \mathbf{t} \geq 0,\ \forall\, \mathbf{t} \in \{\mathbf{t}\}\}$

Determine ideal boundary normal

$\mathbf{t}_{\text{ideal}} \leftarrow$ boundary normal corresponding to $\mathbf{v}_{\text{ideal}}$

Return projected velocity

$$\dot{\mathbf{q}} = \|\mathbf{q}_{\text{last entry}} - \mathbf{q}\| \, \frac{\mathbf{v}_{\text{ideal}} \cdot \mathbf{t}_{\text{ideal}}{}^{\perp}}{\|\mathbf{v}\|} \, \mathbf{t}_{\text{ideal}}{}^{\perp}$$

The near-boundary control policy $\mathrm{g}^{(\mathcal{B})}(\mathbf{q}, \mathbf{q}_{\text{last entry}})$ from Equation S7 is also globally asymptotically stable at $\mathbf{q}_{\text{last entry}}$. Consequently, the track cycle models the behaviour across all boundary regions, while approaching $\mathbf{q}_{\text{last entry}}$ in a stable manner. An example track-cycle DS is shown in Figure S9.

Overall behaviour for $2R$

The overall behaviour in an aspect $\mathcal{A}$ of the joint space, is a resultant of the modelled behaviour $\mathrm{g}^{(\mathcal{Q})}(\mathbf{q})$ and the near-boundary control policy $\mathrm{g}^{(\mathcal{B})}(\mathbf{q}, \mathbf{q}_{\text{last entry}})$.

For a $2R$ robot, the joint velocity is regulated as

$$\dot{\mathbf{q}} = \begin{cases} \mathrm{g}^{(Q)}(\mathbf{q}) & \text{if } \mathbf{q} \notin \{\mathcal{B}_{q_1^\times}, \mathcal{B}_{q_2^\times}, \mathcal{B}_{S^\times}\} \\ \mathrm{g}^{(\mathcal{B})}(\mathbf{q}, \mathbf{q}_{\text{last entry}}) & \text{otherwise} \end{cases} \tag{S8}$$

where the track cycle $\mathrm{C}(\mathbf{q}_{\text{last entry}})$ is determined using Algorithm 4 as required.

Motion along boundaries for noncuspidal $3R$ robots

For $3R$ robots, it has been shown how the singularities can be studied in a reduced dimension parameterized by q_2 and q_3 only. Thus, the extension of near-boundary policy to $3R$ noncuspidal robots leverages the reduced space parameterisation (q_2, q_3). Violations due to first joint's limits do not lead to crossing over into another aspect, as the separation of aspects in joint space is independent of q_1. Thus, the joint velocities for second and third joints $\dot{\mathbf{q}}(q_2, q_3)$ is collectively determined by taking the tangent to the relevant factor of the singularity in the $q_2 - q_3$ slice. The behaviour near the first joint limit's violation $q_1^\times$ is presented in Section 9.

The process for navigating the near-boundary region is an extension to the case of $2R$ robot, and the procedure can be summarised as:

- Identify the first and last boundary crossings from the aspect in $q_2 - q_3$ slice
- Determine the track cycle $\mathrm{C}(\mathbf{q}_{\text{last entry}})$

A $2R$ robot always has exactly two aspects. However, for a $3R$ robot, this depends on a case-by-case basis. When not considering joint limits for defining aspects, there may be upto eight aspects in the joint space (refer to Proposition 2). However, by considering the joint limits, the joint space may get further divided into more aspects. As a result, robots having singularities with identical classifications, may vary in the number of aspects. Figure S11 presents the aspects for a $3R$ robot from Category III with singularity class $2(1,0)[\infty, 0] + 2(0,1)[0,2]$, in the $q_2 - q_3$ slice of the joint space containing nine aspects.

The aspect for $3R$ robots can be identified using the tuple composed of sign of each factor of Jacobian determinant $\det \mathbf{J}(\mathbf{q})$. For instance, if there are two factors

$$\mathcal{A}(\mathbf{q}) = (\mathrm{sign}(\mathrm{f}_1(\mathbf{q})), \mathrm{sign}(\mathrm{f}_2(\mathbf{q}))) \tag{S9}$$

Such factor-sign based identification, in conjunction with $q_2^\times$ and $q_3^\times$ joint limits, is used to identify the first exit from and last entry into the relevant aspect in the joint space (similar to Algorithm 2). We study each

factor independently to determine the near-boundary control policy $\mathrm{g}^{(C)}(\mathbf{q}, \mathbf{q}_{\text{last entry}})$. This is done so that the intersection points in singularities of nongeneric robots can be handled efficiently and the boundary near each branch can be designed with proper offsets.

Algorithm 6. Boundary Crossings for $3R$ Robots (Reduced Space)

Input: Trajectory $\mathbf{Q}$ in joint space, goal configuration $\overset{*}{\mathbf{q}}$

Output: First exit point and last entry point in $\mathbf{Q}$

Identify Singularity crossings

Assign:

$+1$ for configurations in same singularity-free region as $\overset{*}{\mathbf{q}}$

0 for singularities, and -1 otherwise

$Z_{\text{sim}} \leftarrow \text{sign}(\det \mathbf{J}(\mathbf{q} \in \mathbf{Q})) \cdot \text{sign}(\det \mathbf{J}(\overset{*}{\mathbf{q}}))$

Identify Joint limit violations

$\text{violation}(q_2) \leftarrow \{i \mid \mathbf{Q}[i](q_2) \notin [\underline{q_2},\ \overline{q_2}]\}$

$\text{violation}(q_3) \leftarrow \{i \mid \mathbf{Q}[i](q_3) \notin [\underline{q_3},\ \overline{q_3}]\}$

$\text{violations} \leftarrow \text{violation}(q_2) \cup \text{violation}(q_3)$

For all indices in violations: set $Z_{\text{sim}}[i] \leftarrow -1$

Find transition indices

$i_{\text{first exit}} \leftarrow \max\{i \mid \forall j < i,\ Z_{\text{sim}}[j] = +1\}$

$i_{\text{last entry}} \leftarrow \min\{i \mid \forall j > i,\ Z_{\text{sim}}[j] = +1\}$

Return: $(\mathbf{Q}[i_{\text{first exit}}],\ \mathbf{Q}[i_{\text{last entry}}])$

The behaviour to reach $\mathbf{q}_{\text{last entry}}$ from $\mathbf{q}_{\text{first exit}}$ involves $\dot{\mathbf{q}}$ along the tangent to the boundary. The normal to the boundary $\mathbf{t}$ is calculated in $q_2 - q_3$ slice, similar to Algorithm 3. However, as Jacobian determinant $\det \mathbf{J}(\mathbf{q})$ can have up to three factors, the normal to the singularity is calculated by considering the relevant factor of the singularity ($\nabla_{q_2,q_3} f(\mathbf{q})\ \text{sign}(f(\mathbf{q}))$). To determine the boundary region near singularity, we determine the singularity threshold as

$$\epsilon_S = \min\left\{\{\mathrm{f}_i\left(\mathbf{q}(q_2, q_3) + [0, \epsilon_S]^T\right), \mathrm{f}_i\left(\mathbf{q} + [\epsilon_S, 0]^T\right)\}\right\}$$

using all the factors f_i of $\det \mathbf{J}$. Here, $\mathbf{q}_h(q_2, q_3) \in Q_h \cup I_{SL}(q_2^{\times})$ represents the collection of horizontal turning points and the points of intersection between the singularity and $q_2^{\times}$ joint limits, whereas

$\mathbf{q}_v(q_2, q_3) \in Q_v \cup I_{SL}(q_3^{\times})$ represents the collection of vertical turning points and the points of intersection between the singularity and $q_3^{\times}$ joint limits. The boundary region around each factor is defined as

$$\mathcal{B}_{\mathrm{f}_i^{\times}} = \{\mathbf{q} \in \mathcal{Q} \mid \epsilon_S > |\,\mathrm{f}_i(\mathbf{q})|\}$$

Algorithm 7. Normal to the Boundary for $3R$ Robots

Input: Near-boundary region $\mathcal{B}_i$ containing the current joint configuration $\mathbf{q}_t$

Output: Normal direction to the constraint boundary

If $\mathcal{B}_i = \mathcal{B}_{q_2^{\times}}$:

  If $\mathbf{q}_t(q_2) = \underline{q_2}$:

    Return $[1,\ 0]^T$

  Else:

    Return $[-1,\ 0]^T$

Else if $\mathcal{B}_i = \mathcal{B}_{q_3^{\times}}$:

  If $\mathbf{q}_t(q_3) = \underline{q_3}$:

    Return $[0,\ 1]^T$

  Else:

    Return $[0,\ -1]^T$

Else (near singular configuration):

  $f \leftarrow$ factor of $\det \mathbf{J}$ associated with $\mathcal{B}_i$

  Return $\nabla_{q_2,q_3} f(\mathbf{q}_t) \cdot \mathrm{sign}(f(\mathbf{q}_t))$

Unlike $2R$ robots, the singularities in $3R$ robots vary depending on the robot architecture and need to be considered case-wise using the classification studied in Section 4. Consequently, multiple types of track cycles exist depending on the singularity class. Singularities with branches $(m, 0)[2, n]$, where $m \in 0, 1$ and $n \in [2, 12]$ have to be treated seperately. This is because the singularity branch either forms a closed loop or has a 'meander' like curve (refer to Figures S5 and S8). Other classes of singularities have branches that encircle at least one generator and only have horizontal or vertical turning points on a branch. An example of such a singularity with its aspects colored and the boundaries near the singularity and the joint limit violations marked is shown in Figure S11.

The track cycle behaviour that connects $\mathbf{q}_{\text{first exit}}$ and $\mathbf{q}_{\text{last entry}}$ is determined using Algorithm 8 for robots from Categories 1 through 3. As an example, let us consider a $3R$ robot with singularity class $2(1,0)[2,0]+2(1,0)[\infty,0]$ (refer to Figure S12). The Jacobian determinant $\det \mathbf{J}(\mathbf{q})$ for this robot has two factors. The trajectory $\mathbf{Q}$ starting at $\mathbf{q}_0$ has the goal configuration, $\overset{*}{\mathbf{q}}$ and is generated by a control policy $\mathrm{g}^{(Q)}$. The case where $\mathcal{B}(\mathbf{q}_{\text{first exit}}) = \mathcal{B}(\mathbf{q}_{\text{last entry}}) = \mathcal{B}_{f_h^\times}$. Figure S12 illustrates an example corresponding to this case. For cases with vertical turning points (Category II and Category III), the configuration branch is identified with the horizontal component of the normal.

Algorithm 8. Track Cycle in Joint Space for Robots from Categories 1–3

Input: Joint configuration $\mathbf{q}$ and boundary normals $\mathbf{t}$

Determine boundary normals

Compute $\mathbf{t},\ \mathbf{t}_{\text{first exit}},\ \mathbf{t}_{\text{last entry}}$ at current, first exit, and last entry configurations

Category I: Non-loop, non-intersecting singularities with only horizontal turning points

Let $\mathcal{B}_{\mathrm{f}_h^\times}$ denote the boundary region of such factors.

Case 1a: If $\mathbf{q}_{\text{last entry}} \in \mathcal{B}_{\mathrm{f}_h^\times} \cup \mathcal{B}_{q_3^\times}$:

If $\mathbf{q} \in \mathcal{B}_{q_2^\times}$:

$\mathbf{v} \leftarrow [0,\ -\mathbf{t}_{\text{last entry}}(q_3)]^T$

Else if $\mathbf{q} \in \mathcal{B}_{q_3^\times} \cup \mathcal{B}_{\mathrm{f}_h^\times},\ \mathbf{t}(q_3)\mathbf{t}_{\text{last entry}}(q_3) < 0$:

$\mathbf{v} \leftarrow [\mathbf{q}_{\text{last entry}}(q_3) - \mathbf{q}_{\text{first exit}}(q_3),\ 0]^T$

Else if $\mathbf{t}(q_2)\mathbf{t}_{\text{last entry}}(q_2) > 0$:

$\mathbf{v} \leftarrow [\mathbf{q}_{\text{last entry}}(q_2) - \mathbf{q}(q_2),\ 0]^T$

Case 1b: If $\mathbf{q}_{\text{last entry}} \in \mathcal{B}_{q_2^\times}$:

If $\mathbf{q} \in \mathcal{B}_{q_3^\times} \cup \mathcal{B}_{\mathrm{f}_h^\times}$:

$\mathbf{v} \leftarrow [-\mathbf{t}_{\text{last entry}}(q_2),\ 0]^T$

Else if $\mathbf{t}(q_2)\mathbf{t}_{\text{last entry}}(q_2) < 0$:

$\mathbf{v} \leftarrow [0,\ \mathbf{q}_{\text{last entry}}(q_3) - \mathbf{q}_{\text{first exit}}(q_3)]^T$

Else if $\mathbf{t}(q_2)\mathbf{t}_{\text{last entry}}(q_2) > 0$:

$\mathbf{v} \leftarrow [0,\ \mathbf{q}_{\text{last entry}}(q_3) - \mathbf{q}(q_3)]^T$

Category II: Non-loop, non-intersecting singularities with no horizontal turning points

Let $\mathcal{B}_{\mathrm{f}^\times}$ denote the boundary region of the single factor.

Case 2a: If $\mathbf{q}_{\text{last entry}} \in \mathcal{B}_{\text{f}^\times} \cup \mathcal{B}_{q_2^\times}$:

If $\mathbf{q} \in \mathcal{B}_{q_3^\times}$:

$\mathbf{v} \leftarrow [-\mathbf{t}_{\text{last entry}}(q_2),\ 0]^T$

Else if $\mathbf{t}(q_2)\mathbf{t}_{\text{last entry}}(q_2) < 0$:

$\mathbf{v} \leftarrow [0,\ \mathbf{q}_{\text{last entry}}(q_3) - \mathbf{q}_{\text{first exit}}(q_3)]^T$

Else if $\mathbf{t}(q_2)\mathbf{t}_{\text{last entry}}(q_2) > 0$:

$\mathbf{v} \leftarrow [0,\ \mathbf{q}_{\text{last entry}}(q_3) - \mathbf{q}(q_3)]^T$

Case 2b: If $\mathbf{q}_{\text{last entry}} \in \mathcal{B}_{q_3^\times}$:

If $\mathbf{q} \in \mathcal{B}_{q_2^\times} \cup \mathcal{B}_{\text{f}^\times}$:

$\mathbf{v} \leftarrow [0,\ -\mathbf{t}_{\text{last entry}}(q_3)]^T$

Else if $\mathbf{t}(q_3)\mathbf{t}_{\text{last entry}}(q_3) < 0$:

$\mathbf{v} \leftarrow [\mathbf{q}_{\text{last entry}}(q_3) - \mathbf{q}_{\text{first exit}}(q_3),\ 0]^T$

Else if $\mathbf{t}(q_3)\mathbf{t}_{\text{last entry}}(q_3) > 0$:

$\mathbf{v} \leftarrow [\mathbf{q}_{\text{last entry}}(q_2) - \mathbf{q}(q_2),\ 0]^T$

Category III: Non-loop, intersecting singularities

Let up to two factors be horizontal-type with region $\mathcal{B}_{f_h^\times}$, and one vertical-type with region $\mathcal{B}_{f_v^\times}$.

Case 3a: If $\mathbf{q}_{\text{last entry}} \in \mathcal{B}_{f_v^\times} \cup \mathcal{B}_{q_2^\times}$:

If $\mathbf{q} \in \mathcal{B}_{q_3^\times} \cup \mathcal{B}_{f_h^\times}$:

$\mathbf{v} \leftarrow [-\mathbf{t}_{\text{last entry}}(q_2),\ 0]^T$

Else if $\mathbf{t}(q_2)\mathbf{t}_{\text{last entry}}(q_2) < 0$:

$\mathbf{v} \leftarrow [0,\ \mathbf{q}_{\text{last entry}}(q_3) - \mathbf{q}_{\text{first exit}}(q_3)]^T$

Else if $\mathbf{t}(q_2)\mathbf{t}_{\text{last entry}}(q_2) > 0$:

$\mathbf{v} \leftarrow [0,\ \mathbf{q}_{\text{last entry}}(q_3) - \mathbf{q}(q_3)]^T$

Case 3b: If $\mathbf{q}_{\text{last entry}} \in \mathcal{B}_{f_h^\times} \cup \mathcal{B}_{q_3^\times}$:

If $\mathbf{q} \in \mathcal{B}_{q_2^\times} \cup \mathcal{B}_{f_v^\times}$:

$\mathbf{v} \leftarrow [0,\ -\mathbf{t}_{\text{last entry}}(q_3)]^T$

Else if $\mathbf{t}(q_3)\mathbf{t}_{\text{last entry}}(q_3) < 0$:

$\mathbf{v} \leftarrow [\mathbf{q}_{\text{last entry}}(q_2) - \mathbf{q}_{\text{first exit}}(q_2),\ 0]^T$

Else if $\mathbf{t}(q_3)\mathbf{t}_{\text{last entry}}(q_3) > 0$:

$\mathbf{v} \leftarrow [\mathbf{q}_{\text{last entry}}(q_2) - \mathbf{q}(q_2),\ 0]^T$

Return v

The trajectory generated by motion control law violates kinematic constraints at $\mathbf{q}_{\text{first exit}}$ and $\mathbf{q}_{\text{last entry}}$. The path between these two points is generated from the track cycle in Algorithm 8. As we $\mathcal{B}(\mathbf{q}_{\text{first exit}}) = \mathcal{B}_{f_1^\times}$, the velocity generated at this configuration is along the direction of $[\mathbf{q}_{\text{last entry}}(q_2) - \mathbf{q}_{\text{first exit}}(q_2), 0]^T$, in this case generates a leftward velocity direction. Progressively, the joint configuration, $\mathbf{q}$, approaches $\mathcal{B}_{q_2^\times}$. When $\mathbf{q} \in \mathcal{B}_{q_2^\times}$, $\mathbf{v} = [0, -\mathbf{t}_{\text{last entry}}(q_2)]^T$ forces the joint velocity along positive q_2 axis directed parallel to $q_2^\times$ while progressively approaching $\mathbf{q} \in \mathcal{B}_{f_1^\times}$ such that $\mathbf{t}_\mathbf{q}.\mathbf{t}_{\text{last entry}} \geq 0$. This forces the joint velocity along the singularity branch towards $\mathbf{q}_{\text{last entry}}$ as seen from the last row of the corresponding table in Algorithm 8. The overall behaviour encapuslated by $\text{C}(\mathbf{q}_{\text{last entry}})$ is thus

$$(\mathbf{q}_{\text{first exit}} \in \mathcal{B}_{f_1^\times}, \mathbf{t}(q_2)\mathbf{t}_{\text{last entry}}(q_2) < 0) \rightarrow (\mathbf{q} \in \mathcal{B}_{q_2^\times})$$
$$\rightarrow (\mathbf{q} \in \mathcal{B}_{f_1^\times}, \mathbf{t}(q_2)\mathbf{t}_{\text{last entry}}(q_2) > 0) \rightarrow (\mathbf{q}_{\text{last entry}})$$

The track cycle $\text{C}(\mathbf{q}_{\text{last entry}})$ determines the behaviour to reach $\mathbf{q}_{\text{last entry}}$ from any point in any of the boundary regions in the current aspect. To model the joint velocity during the track cycle behaviour between $\mathbf{q}_{\text{first exit}}$ and $\mathbf{q}_{\text{last entry}}$, we introduce a track-cycle DS. The joint velocity in the $q_2 - q_3$ slice for a point $\mathbf{q}$ in the boundary $\mathcal{B}(\mathbf{q})$ is determined as

$$\dot{\mathbf{q}}(q_2, q_3) = \|\mathbf{q}_{\text{last entry}} - \mathbf{q}\| \frac{\mathbf{v}(q_2, q_3) \cdot \mathbf{t}^\perp}{\|\mathbf{v}\|} \mathbf{t}^\perp \tag{S10}$$

where $\mathbf{t}^\perp$ is the normal to $\mathbf{t}$. If $\mathbf{q}_{\text{last entry}}$ lies simultaneously in multiple near-boundary regions, any region (and associated boundary normal $\mathbf{t}_{\text{last entry}}$ can be used to determine the track cycle.

Algorithm 9. Joint Velocity for $3R$ Robot During Overlapping Boundary Regions

Input: Current configuration $\mathbf{q}$ in overlapping edge region

Output: Joint velocity $\dot{\mathbf{q}}$ to reach $\mathbf{q}_{\text{last entry}}$

Identify overlap region

$\mathbf{q} \in E_{1_2^\times} \cap E_{1_3^\times}$ or $\mathbf{q} \in E_{1_2^\times} \cap E_{\text{f}_i^\times}$

Extract boundary edge set

$\{E(\mathbf{q})\} \leftarrow$ either $\{E_{1_2^\times}, E_{1_3^\times}\}$ or $\{E_{1_2^\times}, E_{\text{f}_i^\times}\}$

For each edge in $\{E(\mathbf{q})\}$

Call Algorithm 3 $\Rightarrow \{\mathbf{t}\}$ (set of boundary normals)

Retrieve corresponding track-cycle velocities

$\{\mathbf{v}\} \leftarrow$ velocity candidates from transition logic $\mathcal{T}$

Determine ideal goal velocity direction

$\mathbf{v}_{\text{ideal}} \leftarrow \{\mathbf{v} \in \{\mathbf{v}\} \mid \mathbf{v} \cdot \mathbf{t} \geq 0,\ \forall\, \mathbf{t} \in \{\mathbf{t}\}\}$

Determine ideal boundary normal

$\mathbf{t}_{\text{ideal}} \leftarrow$ normal corresponding to $\mathbf{v}_{\text{ideal}}$

$\mathbf{t}_{\text{ideal}} \leftarrow \text{argmin}_{\mathbf{t}\in\{\mathbf{t}\}}\ (\mathbf{t} \cdot \mathbf{v}_{\text{ideal}})$

Return projected joint velocity

$$\dot{\mathbf{q}} \leftarrow \|\mathbf{q}_{\text{last entry}} - \mathbf{q}\|\ \frac{\mathbf{v}_{\text{ideal}} \cdot {\mathbf{t}_{\text{ideal}}}^{\perp}}{\|\mathbf{v}\|}\ {\mathbf{t}_{\text{ideal}}}^{\perp}$$

Return $\dot{\mathbf{q}}$

The near-boundary control policy is globally asymptotically stable at $\mathbf{q}_{\text{last entry}}$ in the $q_2 - q_3$ slice. Consequently, the track cycle models the behaviour across all boundary regions, while approaching $\mathbf{q}_{\text{last entry}}$ in a stable manner. The method to determine the track cycles for robots from the remaining categories - Categories 4 through 6 - require different considerations and are presented at the end of this section.

When there are violations only in the $q_2 - q_3$ slice, the first joint's velocity is overlayed in accordance with the near-boundary control policy as

$$\begin{aligned}\dot{\mathbf{q}}(q_1) &= \mathrm{g}^{(\mathcal{B})}(\mathbf{q}, \mathbf{q}_{\text{last entry}}) \\ &= \text{sign}(\mathbf{q}_{\text{last entry}}(q_1) - \mathbf{q}(q_1)) \cdot \|\mathbf{q}_{\text{last entry}} - \mathbf{q}\|\end{aligned} \tag{S11}$$

Equation S11 ensures that the robot converges at the $\mathbf{q}_{\text{last entry}}$ across all axis in the joint space, before proceeding with further replication towards $\overset{*}{\mathbf{q}}$. In the scenario where $\mathbf{q}_{\text{last entry}}(q_1)$ exceeds the first joint's limit, it is reset to the nearest permissible joint value i.e. $\overline{q_1} - \epsilon_q$ or $\underline{q_1} + \epsilon_q$.

Behaviour near first joint's boundary

We now extend the near-boundary control policy to joint configurations in the boundary of first joint's limits $\mathcal{B}_{q_1^{\times}}$, when that is the only type of violation. In such scenario, the velocity of the first joint is overlayed as

$$\dot{\mathbf{q}}(q_1) = 0$$

which prevents violations along first joint's limits. For the behaviour of second and third joints, the predicted path $\mathbf{Q}$ is used to determine the first joint configuration satisfying $q_1^{\times}$. This point $\mathbf{q}_{\text{target}} \in \mathcal{Q}$ is used to overlay the joint velocities as

$$\begin{aligned}\dot{\mathbf{q}}(q_2, q_3) &= \mathrm{g}^{(\mathcal{B})}(\mathbf{q}, \mathbf{q}_{\text{target}}) \\ &= \text{sign}(\mathbf{q}_{\text{target}}(q_2, q_3) - \mathbf{q}(q_2, q_3)) \cdot \|\mathbf{q}_{\text{target}} - \mathbf{q}\|\end{aligned} \tag{S12}$$

Equation S12 ensures that the robot converges at the $\mathbf{q}_{\text{target}}$ across all axis in the joint space, before proceeding with further replication towards $\overset{*}{\mathbf{q}}$.

Overall behaviour for $3R$

The overall behaviour, resulting due to the control policy learned from demonstration $\mathrm{g}^{(\mathcal{Q})}$ for the feasible joint configurations of an aspect $\mathcal{A}$, as well as the near-boundary control policy $\mathrm{g}^{(\mathcal{B})}$ for configurations near the aspect's boundaries has been summarised in Table S1. It must be noted that it is not possible to have violation in both in the $q_2 - q_3$ slice, and along q_1 simultaneously since a violation in $q_2 - q_3$ slice and the resulting near boundary behaviour will prevent any further

The combination of the these policies ensures that kinematic constraints are embedded into the joint velocity during the replication of behaviour modelled from a demonstration.

When the initial joint configuration $\mathbf{q}$ is in a near-boundary region, the trajectory $\mathbf{Q}$ generated by $\mathrm{g}^{(\mathcal{Q})}$ starting at $\mathbf{q}$ and converging at $\overset{*}{\mathbf{q}}$, is used to determine the track cycle and subsequently the near-boundary behaviour $\mathrm{g}^{(\mathcal{B})}(\mathbf{q}, \mathbf{q}_{\text{last entry}})$. The method for generating $\mathbf{Q}$ is elaborated in Section 12. Unlike point-by-point integration methods, our approach allows the full trajectory to be computed in a single step, enabling efficient and rapid validation.

Algorithms for track cycle

The method to determine the track cycle for robots from Categories 1 through 3 was presented in Algorithm 8. In this section, we present the remainder of algorithms to determine the track cycle for noncuspidal $3R$ robots from Categories 4 through 6. The boundary normals $\mathbf{t}, \mathbf{t}_{\text{first exit}}, \mathbf{t}_{\text{last entry}}$ at the current joint configuration, first exit, and last entry configurations are determined using Algorithm 7 as the first step of determining the track cycle.

Robots from Category IV

There exists a single factor f in the expression of Jacobian determinant $\det \mathbf{J}(\mathbf{q})$ containing 2 branches of type $(1, 0)$, and at least one of the branches has finite vertical turning points The horizontal tuning points are represented as Q_h, and the vertical ones as Q_v. I_{SL} is the set of configurations at intersection between the singularity and joint limits in the $q_2 - q_3$ slice. It is possible that the branches may intersect with $q_3^{\times}$, and more than four times with $q_2^{\times}$. In such cases, the horizontal or vertical turning points no longer lie on a continuous branch in the $q_2 - q_3$ slice. For instance in Figure S5, the branch looks like it has been split or separated by the joint limits. In such cases, we translate the $q_2 - q_3$ slice along q_2 and (or) q_3 axis such that the horizontal and vertical turning points on the branches lie along a continuous path. To achieve this, the relevant points on each branch Q_h, Q_v, I_{SL} are translated using Algorithm 20. To determine which of the two branches each of these joint configurations lie on, we use the criteria

$$B(\mathbf{q}) = \begin{cases} +1 & \text{if } \operatorname{sign}(\mathrm{f}(\mathbf{q} + [0, \epsilon_q]^T)) > 0 \\ -1 & \text{if } \operatorname{sign}(\mathrm{f}(\mathbf{q} + [0, \epsilon_q]^T)) < 0 \end{cases} \tag{S13}$$

Joint configurations from (Q_h, Q_v, I_{SL}) are next sorted two sets of joint configurations Q^+ and Q^- viz:
$Q^+ \leftarrow$ Configurations with $B = +1$ along the branch in ascending order of q_2 values, in order of occurrence
$Q^- \leftarrow$ Configurations with $B = -1$ along the branch in ascending order of q_2 values, in order of occurrence

Configurations from Q^+ and Q^- are translated back to the original $q_2 - q_3$ slice, in case Algorithm 20 was previously used. To specify the behaviour near constraints, we define bounding boxes B_i^+ for $i \in [1, 2 \cdots |Q^+| - 1]$, made using Q_i^+ and Q_{i+1}^+ as diagonal vertices along the first branch. For the second branch, B_i^- for $i \in [1, 2 \cdots |Q^-| - 1]$ are similarly defined. To identify which bounding box does a joint configuration $\mathbf{q} \in \mathcal{B}_{\mathrm{f}^{\times}}$, we calculate all the projections of $\mathbf{q}$ on f along q_2 and q_3, and identify the projection closest as $\mathbf{q}_{\text{proj}}$. The bounding box containing $\mathbf{q}_{\text{proj}}$ also holds $\mathbf{q}$. In case of multiple suitable boxes, anyone can be selected. To move near the branch rightward, from a box B_i to B_{i+1} on either branches, the velocity vector $\mathbf{v}$ gets two candidate values $[Q_{i+1}(q_2) - \mathbf{q}(q_2), 0]^T$ and $[0, Q_{i+1}(q_3) - \mathbf{q}(q_3)]^T$. The suitable velocity is later determined using Algorithm 9. For leftward traversal, we instead use Q_{i-1} to determine $\mathbf{v}$. Figure S13 illustrates the track cycle in $q_2 - q_3$ slice for a robot from Category IV. The method to determine the track cycle is presented in Algorithm 10.

Algorithm 10. Track Cycle in Joint Space for Robots from Category IV

Input: Current configuration $\mathbf{q}$, entry/exit points $\mathbf{q}_{\text{first exit}}$, $\mathbf{q}_{\text{last entry}}$

Determine branches for $\mathbf{q}_{\text{first exit}}$ and $\mathbf{q}_{\text{last entry}}$

Identify type of violation at both points

Case 1: $\mathbf{q}_{\text{last entry}} \in B_j^+$

If $\mathbf{q} \in \mathcal{B}_{q_2^\times}$:

$\mathbf{v} \leftarrow [0,\ I_{SL}^+(q_3) - I_{SL}^-(q_3)]^T$

If $\mathbf{q} \in B_k^+$:

If $k < j$: Perform rightward traversal

If $k > j$: Perform leftward traversal

If $k = j$: $\mathbf{v} \leftarrow [\mathbf{q}_{\text{last entry}}(q_2) - \mathbf{q}(q_2),\ 0]^T$

If $\mathbf{q} \in B^-$

If $[\mathbf{q}_{\text{last entry}} - \mathbf{q}_{\text{first exit}}](q_2) \geq 0$:

Rightward traversal

Else: Leftward traversal

If $\mathbf{q} \in \mathcal{B}_{q_3^\times}$:

If $\mathbf{q}$ does not lie between relevant I_{SL}^+ points:

$\mathbf{v} \leftarrow [\mathbf{q}_{\text{last entry}}(q_2) - \mathbf{q}_{\text{first exit}}(q_2),\ 0]^T$

Else: Use B_k^+ that contains relevant I_{SL}^+

Case 2: $\mathbf{q}_{\text{last entry}} \in B_j^-$

If $\mathbf{q} \in \mathcal{B}_{q_2^\times}$:

$\mathbf{v} \leftarrow [0,\ I_{SL}^-(q_3) - I_{SL}^+(q_3)]^T$

If $\mathbf{q} \in B^+$:

If $[\mathbf{q}_{\text{last entry}} - \mathbf{q}_{\text{first exit}}](q_2) \geq 0$:

Rightward traversal

Else Leftward traversal

If $\mathbf{q} \in B_k^-$:

If $k < j$: Rightward traversal

If $k > j$: Leftward traversal

If $k = j$: $\mathbf{v} \leftarrow [\mathbf{q}_{\text{last entry}}(q_2) - \mathbf{q}(q_2),\ 0]^T$

If $\mathbf{q} \in \mathcal{B}_{q_3^\times}$:

If $\mathbf{q}$ does not lie between relevant I_{SL}^- (I_{SL}^- points with $q_3 = \mathbf{q}(q_3)$:

$\mathbf{v} \leftarrow [\mathbf{q}_{\text{last entry}}(q_2) - \mathbf{q}_{\text{first exit}}(q_2),\ 0]^T$

Else: Use B_k^- that contains relevant I_{SL}^-

Case 3: $\mathbf{q}_{\text{last entry}} \in \mathcal{B}_{q_2^\times}$

If $\mathbf{t}(q_2)\mathbf{t}_{\text{last entry}}(q_2) < 0$:

$\mathbf{v} \leftarrow [0,\ \mathbf{q}_{\text{last entry}}(q_3) - \mathbf{q}_{\text{first exit}}(q_3)]^T$

Else:

$\mathbf{v} \leftarrow [0,\ \mathbf{q}_{\text{last entry}}(q_3) - \mathbf{q}(q_3)]^T$

If $\mathbf{q} \in \mathcal{B}_{q_3^\times}$ or in B^+ or B^-:

Use direction based on whether $\mathbf{q}_{\text{last entry}}(q_2) = \overline{q_2}$ or $\underline{q_2}$

(rightward or leftward traversal)

Case 4: $\mathbf{q}_{\text{last entry}} \in \mathcal{B}_{q_3^\times}$

If $\mathbf{q}_{\text{last entry}}$ not between relevant I_{SL}^- or I_{SL}^+:

If $\mathbf{q} \in \mathcal{B}_{q_2^\times}$: $\mathbf{v} \leftarrow [0,\ -\mathbf{t}_{\text{last entry}}(q_3)]^T$

If $\mathbf{q} \in \mathcal{B}_{q_3^\times}$: $\mathbf{v} \leftarrow [\mathbf{q}_{\text{last entry}}(q_2) - \mathbf{q}(q_2),\ 0]^T$

If $\mathbf{q} \in \{B^+, B^-\}$ and $\Delta q_2 \geq 0$: rightward; else: leftward

If $\mathbf{q}_{\text{last entry}}$ lies between relevant I_{SL}^+:

If $\mathbf{q} \in B^-$ and $\Delta q_2 \geq 0$: rightward; else: leftward

If $\mathbf{q} \in B^+$: use velocity from B_k^+ containing any I_{SL}^+

If $\mathbf{q}_{\text{last entry}}$ lies between relevant I_{SL}^-:

If $\mathbf{q} \in B^+$ and $\Delta q_2 \geq 0$: rightward; else: leftward

If $\mathbf{q} \in B^-$: use velocity from B_k^+ containing any I_{SL}^+

Robots from Category V

There will be single factor in the expression of Jacobian determinant $\det \mathbf{J}(\mathbf{q})$ containing a single branch of type $(0, 0)$. The branch will have only two horizontal turning points, and a finite number of vertical turning points. Q_h, Q_v, and I_{SL} represent the horizontal tuning points, vertical tuning points, and intersection between the singularity and joint limits in the $q_2 - q_3$ slice respectively. Similar to Category IV, it is possible that the horizontal and(or) vertical turning points do not lie on a continuous branch in the $q_2 - q_3$ slice. In such cases, we translate the $q_2 - q_3$ slice along q_2 and (or) q_3 axis such that the horizontal and

vertical turning points on the branches lie along a continuous path. To achieve this, the points Q_h, Q_v, I_{SL} are translated using Algorithm 21.

To specify the behaviour, it is required to differentiate between the interior of the loop created by the branch, and its exterior. To do so, we identify the horizontal turning point $\mathbf{q}_h$ with least q_3 value. A configuration $\mathbf{q}$ is inside the loop if $\text{sign}(\text{f}(\mathbf{q})) = \text{sign}(\text{f}(\mathbf{q_h} + [0, \epsilon_q]^T))$, and outside otherwise.

$Q \leftarrow \{Q_1, Q_2 \cdots Q_n\}$ ordered set of configurations from (Q_h, Q_v, I_{SL}) along the $(0, 0)$ branch in clockwise fashion, starting from the horizontal turning point with least q_3

We once again define bounding boxes B_i for $i \in [1, 2 \cdots |Q|]$, made using Q_i and $Q_{i+1 \bmod n}$ as diagonal vertices. These bounding boxes are adjacent with a common edge vertex between subsequent boxes, distributed along the loop like branch. To identify which bounding box does a joint configuration $\mathbf{q} \in \mathcal{B}_{\text{f}^\times}$, we once again calculate all the projections of $\mathbf{q}$ on f along q_2 and q_3, and identify the projection closest as $\mathbf{q}_{\text{proj}}$. The box containing $\mathbf{q}_{\text{proj}}$ also contains $\mathbf{q}$. To specify behaviour near the constraints, we define two directions - clockwise and anticlockwise as:

For a point inside the loop, in the bounding box B_i:

- $\mathbf{v}_C = \text{track}(Q_{i+1})$
- $\mathbf{v}_{\text{AC}} = \text{track}(Q_{i-1})$

In $\mathcal{B}_{q_2^\times}$:

- $\mathbf{v}_C = [0, -\text{sign}(q_2)]^T$
- $\mathbf{v}_{\text{AC}} = [0, \text{sign}(q_2)]^T$

In $\mathcal{B}_{q_3^\times}$

- $\mathbf{v}_C = [\text{sign}(q_3), 0]^T$
- $\mathbf{v}_{\text{AC}} = -[\text{sign}(q_3), 0]^T$

track(Q_i) returns two velocity vectors from $[0, Q_i(q_3) - \mathbf{q}(q_3)]^T$ and $[Q_i(q_2) - \mathbf{q}(q_2), 0]^T$ to traverse from $\mathbf{q}$ to $\mathbf{q}_{\text{last entry}}$, and the suitable velocity is later determined using Algorithm 9. Figure S14 illustrates the track cycle in $q_2 - q_3$ slice for a robot from Category V. The method to determine the track cycle is presented in Algorithm 11.

Algorithm 11. Track cycle in joint space for robots from Category V

Input: $\mathbf{q}_{\text{last entry}}, \mathbf{q}_{\text{first exit}}$

Determine if $\mathbf{q}_{\text{last entry}}$ (and $\mathbf{q}_{\text{first exit}}$) lie inside or outside the loop

Determine type of violation at these indices

Case 1: $\mathbf{q}_{\text{last entry}} \in B_j$ lies **inside** the loop

In case $\mathbf{q}$ lies near joint limit, identify the bounding box for $\mathbf{q}$ by using the nearest I_{SL} while traversing in clockwis

Select bounding box containing the relevant I_{SL} vertex

If $\mathbf{q} \in B_k$:

If $k < j$: $\mathbf{v} \leftarrow \mathbf{v}_C$

If $k > j$: $\mathbf{v} \leftarrow \mathbf{v}_{\text{AC}}$

If $k = j$: $\mathbf{v} \leftarrow \text{track}(\mathbf{q}_{\text{last entry}})$

Case 2: $\mathbf{q}_{\text{last entry}}$ lies **outside** the loop and $I_{SL} = \emptyset$

Subcase: $\mathbf{q}_{\text{last entry}} \in \mathcal{B}_{S^\times}$

If $\mathbf{q} \in B_k$:

$k < j$: $\mathbf{v} \leftarrow \mathbf{v}_C$

$k > j$: $\mathbf{v} \leftarrow \mathbf{v}_{\text{AC}}$

$k = j$: $\mathbf{v} \leftarrow \text{track}(\mathbf{q}_{\text{last entry}})$

If $\mathbf{q} \in \mathcal{B}_{q_2^\times}$:

If $\mathbf{t}(q_2)\mathbf{t}_{\text{last entry}}(q_2) < 0$:

$\mathbf{v} \leftarrow (\mathbf{q}_{\text{first exit}}(q_3) - \mathbf{q}_{\text{last entry}}(q_3))\mathbf{v}_C$

Else:

$\mathbf{v} \leftarrow (\mathbf{q}_{\text{last entry}}(q_3) - \mathbf{q}(q_3))\mathbf{v}_C$

If $\mathbf{q} \in \mathcal{B}_{q_3^\times}$:

$\mathbf{v} \leftarrow (\mathbf{q}_{\text{last entry}}(q_2) - \mathbf{q}_{\text{first exit}}(q_2))\mathbf{v}_C$

Subcase: $\mathbf{q}_{\text{last entry}} \in \mathcal{B}_{q_2^\times}$

If $\mathbf{q} \in \mathcal{B}_{q_3^\times}$:

$\mathbf{v} \leftarrow [-\mathbf{t}_{\text{last entry}}(q_2),\ 0]^T$

If $\mathbf{t}(q_2)\mathbf{t}_{\text{last entry}}(q_2) < 0$:

$\mathbf{v} \leftarrow [0,\ \mathbf{q}_{\text{last entry}}(q_3) - \mathbf{q}_{\text{first exit}}(q_3)]^T$

Else:

$\mathbf{v} \leftarrow [0,\ \mathbf{q}_{\text{last entry}}(q_3) - \mathbf{q}(q_3)]^T$

Subcase: $\mathbf{q}_{\text{last entry}} \in \mathcal{B}_{q_3^\times}$

If $\mathbf{q} \in \mathcal{B}_{q_2^\times}$:

$\mathbf{v} \leftarrow [0,\ -\mathbf{t}_{\text{last entry}}(q_3)]^T$

If $\mathbf{t}(q_3)\mathbf{t}_{\text{last entry}}(q_3) < 0$:

$\mathbf{v} \leftarrow [\mathbf{q}_{\text{last entry}}(q_2) - \mathbf{q}_{\text{first exit}}(q_2),\ 0]^T$

Else:

$\mathbf{v} \leftarrow [\mathbf{q}_{\text{last entry}}(q_2) - \mathbf{q}(q_2),\ 0]^T$

Case 3: $\mathbf{q}_{\text{last entry}}$ lies **outside** the loop and $I_{SL} \neq \emptyset$

If $\mathbf{q} \in B_k \in \mathcal{B}_{S^\times}$:

If $k < j$: $\mathbf{v} \leftarrow \mathbf{v}_C$

If $k > j$: $\mathbf{v} \leftarrow \mathbf{v}_{\text{AC}}$

If $k = j$: $\mathbf{v} \leftarrow \text{track}(\mathbf{q}_{\text{last entry}})$

If $B_k \in \{\mathcal{B}_{q_2^\times}, \mathcal{B}_{q_3^\times}\}$:

If $k < j$: $\mathbf{v} \leftarrow \mathbf{v}_{\text{AC}}$

If $k > j$: $\mathbf{v} \leftarrow \mathbf{v}_C$

If $k = j$:

If $\mathbf{t} \cdot \mathbf{t}_{\text{last entry}} > 0$: $\mathbf{v} \leftarrow \mathbf{q}_{\text{last entry}} - \mathbf{q}$

If $\mathbf{t} \cdot \mathbf{t}_{\text{last entry}} = 0$: $\mathbf{v} \leftarrow -\mathbf{t}_{\text{last entry}}$

If $\mathbf{t} \cdot \mathbf{t}_{\text{last entry}} < 0$: revise $k \leftarrow n + 1$ and determine direction

---

Robots from Category VI

There will be two factors, one of which is of type $(0, 0)$ with a single branch. The other factor that exist can only be of type $2(1, 0)[\infty, 0]$. The factor of type $(0, 0)$ is represented using f_1, and f_2 represents the factor corresponding to the $(1, 0)$ branches, Q_h, and Q_v represent the horizontal and vertical turning

points associated with f_1, and I_{SL} represents the intersections between f_1 and $q_2^{\times}, q_3^{\times}$. I_{SS} represents the intersections between f_1 and f_2. $Q \leftarrow \{Q_1, Q_2 \cdots Q_n\}$ ordered set of configurations from $(Q_h, Q_v, I_{SL}, I_{SS})$ along the $(0,0)$ branch in clockwise fashion, starting from the horizontal turning point with least q_3. Using Q, bounding boxes are defined as before. The interior/exterior of the $(0,0)$ branch, and the method of determining the bounding box for a joint configuration $\mathbf{q} \in \mathcal{B}_{\mathrm{f}^{\times}}$ is identical to Appendix 10. $\mathcal{B}_{\mathrm{f}_2^{\times}}$ is the boundary region of f_2, and treated identical to $\mathcal{B}_{q_3^{\times}}$. The method to determine the track cycle is presented in Algorithm 12.

Algorithm 12. Track cycle in joint space for robots from Category VI

Input: $\mathbf{q}_{\text{last entry}}$

Output: Tangential joint velocity $\mathbf{v}$

If $\mathbf{q}_{\text{last entry}}$ lies inside the loop:

$\mathbf{v} \leftarrow$ Call Algorithm 11

Else if $\mathbf{q}_{\text{last entry}}$ lies outside the loop:

Let Q_k be the region containing $\mathbf{q}$

If $Q_k \in \mathcal{B}_{S^{\times}}$:

If $k > j$:

If I_{SS} lies clockwise between j and k: $\mathbf{v} \leftarrow \mathbf{v}_{\mathrm{AC}}$

Else: $\mathbf{v} \leftarrow \mathbf{v}_{\mathrm{C}}$

If $k < j$:

If I_{SS} lies clockwise between j and k: $\mathbf{v} \leftarrow \mathbf{v}_{\mathrm{C}}$

Else: $\mathbf{v} \leftarrow \mathbf{v}_{\mathrm{AC}}$

If $k = j$: $\mathbf{v} \leftarrow \mathrm{track}(\mathbf{q}_{\text{last entry}})$

Else if $Q_k \in \{\mathcal{B}_{q_2^{\times}}, \mathcal{B}_{q_3^{\times}}\}$:

If $k < j$:

If I_{SS} lies clockwise between j and k: $\mathbf{v} \leftarrow \mathbf{v}_{\mathrm{C}}$

Else: $\mathbf{v} \leftarrow \mathbf{v}_{\mathrm{AC}}$

If $k > j$:

If I_{SS} lies clockwise between j and k: $\mathbf{v} \leftarrow \mathbf{v}_{\mathrm{AC}}$

Else: $\mathbf{v} \leftarrow \mathbf{v}_{\mathrm{C}}$

If $k = j$:

If $\mathbf{t} \cdot \mathbf{t}_{\text{last entry}} > 0$: $\mathbf{v} \leftarrow \mathbf{q}_{\text{last entry}} - \mathbf{q}$

If $\mathbf{t} \cdot \mathbf{t}_{\text{last entry}} = 0$: $\mathbf{v} \leftarrow -\mathbf{t}_{\text{last entry}}$

If $\mathbf{t} \cdot \mathbf{t}_{\text{last entry}} < 0$: revise $k \leftarrow n + 1$ and determine direction

---

Modelling the Robot Control Policy

So far, we have discussed the robot behaviour while considering a single demonstration in one of the aspects of the joint space. In this section, we consider the joint space behaviour resulting from multiple demonstrations in multiple aspects. We present the different elements of the framework that allows us to generalise the behaviour of a demonstrated task from multiple demonstrations in workspace workspace $\mathcal{X}$, to a robust and reactive control law across all the aspects in joint space $\mathcal{Q}$, while also embedding information about singularities and joint-limits

We begin by considering a set of K_Q demonstrations in joint space, denoted as $\{\mathbf{Q}\}$. These demonstrations are the joint space counterparts of the demonstrations in workspace $\mathcal{X}$. The proposed method first leverages the available $\{\mathbf{Q}\}$ to learn a preliminary behaviour in the joint space. Later in Section 21, the framework incrementally updates the preliminary behaviour in an offline manner, to embed additional knowledge about the task from workspace into the joint space.

Grouping joint demonstrations by aspects

While the workspace behaviour $\mathrm{g}^{(\mathcal{X})}$ had a single goal position workspace goal position $\overset{*}{\mathbf{x}}$, there exist I configurations in joint space $\mathcal{Q}$ that correspond to workspace goal position $\overset{*}{\mathbf{x}}$. This set of distinct joint configurations $\{\overset{*}{\mathbf{q}}_i \mid \mathrm{FK}(\overset{*}{\mathbf{q}}_i) = \overset{*}{\mathbf{x}}\}_{i=1}^{I}$, are the inverse kinematic solutions for workspace goal position $\overset{*}{\mathbf{x}}$. Each of $\overset{*}{\mathbf{q}}_i$ resides in a unique aspect $\mathcal{A}_i$. An individual goal joint configuration $\overset{*}{\mathbf{q}}_i$ may be known via the last point of the expert demonstration in joint space $\mathcal{Q}$, solved numerically or provided externally.

As there exists a single workspace goal position $\overset{*}{\mathbf{x}}$, each associated joint demonstration strictly terminates at one of the configurations in goal configurations $\{\overset{*}{\mathbf{q}}_i \mid \mathrm{FK}(\overset{*}{\mathbf{q}}_i) = \overset{*}{\mathbf{x}}\}_{i=1}^{I}$, wherein $\overset{*}{\mathbf{q}}_i \in \mathcal{A}_i$. Furthermore, as the workspace demonstrations $\{\mathbf{X}\}$ do not encounter any singular configuration, their joint space counterparts $\{\mathbf{Q}\}$ also remain free of singular configurations. Hence, if a joint demonstration $\mathbf{Q}$ terminates at $\overset{*}{\mathbf{q}}_i \in \mathcal{A}_i$, it can be said the entire trajectory $\mathbf{Q} \in \mathcal{A}_i$ only.

To proceed with behavioural modelling, all the joint demonstrations belonging to the same aspect $\mathcal{A}_i$ (and hence by extension having a common goal configuration $\overset{*}{\mathbf{q}}_i$) are grouped into a set $\{\mathbf{Q}\}_{\mathcal{A}_i}$. Thus, for a query joint configuration $\mathbf{q}$ that lies in aspect $\mathcal{A}(\mathbf{q})$, the relevant joint attractor fulfils $\mathcal{A}(\overset{*}{\mathbf{q}}_i) = \mathcal{A}(\mathbf{q})$. The grouping of joint demonstrations is performed on basis of the aspects of the respective joint attractors

$$\{\mathbf{Q}\}_{\mathcal{A}_i} = \{\mathbf{Q}_j \in \mathcal{Q} \mid \mathcal{A}(\mathbf{Q}_j) = \mathcal{A}_i\}$$
$$\forall\, i \in \{1, \ldots, I\},\ \forall\, j \in \{1, \ldots, K_Q\}$$

A $2R$ robot always has exactly two aspects. The aspect of a non-singular query point $\mathbf{q}$ can be determined as $\mathcal{A}(\mathbf{q}) = \text{sign}(\det \mathbf{Q}(\mathbf{q}))$. On the other hand, a $3R$ robot may have upto eight aspects of varying nature which depends on the specific architecture of the robot. The aspect can be identified using the tuple composed of sign of each factor of Jacobian determinant $\det \mathbf{J}(\mathbf{q})$. For instance, if there are two factors

$$\mathcal{A}(\mathbf{q}) = (\text{sign}(\text{f}_1(\mathbf{q})), \text{sign}(\text{f}_2(\mathbf{q}))) \tag{S14}$$

Figure 8C illustrates the two aspects in the joint space of a $2R$ robot. Each aspect contains two joint trajectories corresponding to the two workspace demonstrations.

Validation of joint demonstrations

As presented in Section , it stands that some of the joint demonstrations, corresponding to the workspace demonstration, may be infeasible. Such trajectories must be identified to prevent acquiring infeasible behaviour.

To identify such trajectories, we perform two checks.

First check: For each joint space trajectory $\mathbf{Q}_i \in \{\mathbf{Q}\}$, check for joint limit violations along the trajectory

$$\exists \mathbf{q} = [q_1, q_2, q_3]^T \in \mathbf{Q}_i \text{ such that } q_1 \notin [\underline{q_1} + \epsilon_q, \overline{q_1} - \epsilon_q]$$

or $q_2 \notin [\underline{q_2} + \epsilon_q, \overline{q_2} - \epsilon_q]$ or $q_3 \notin [\underline{q_3} + \epsilon_q, \overline{q_3} - \epsilon_q]$.

Second check: Check for singularity violations in the trajectory

$$\exists \mathbf{q} \in \mathbf{Q}_i \text{ such that any for any factor } \text{f}_i \text{ of } \det \mathbf{J},$$

$|\text{f}_i(\mathbf{q}_t)| < \epsilon_S$ where $\epsilon_S > 0$ is the safety threshold around singular configurations (refer Section 5).

Any trajectory $\mathbf{Q}_i$ which fails either of the two checks has violated the kinematic constraints. Instead of discarding such trajectories, we trim the trajectories to only include the terminal violation free portion of the trajectory that starts after the last violation in the trajectory and converges at the respective $\overset{*}{\mathbf{q}}$. In doing so, all the trajectories from $\{\mathbf{Q}\}$ are trimmed on an as-needed basis, and are hence validated to be free of infeasible configurations.

The second joint trajectory in Aspect 2 of Figure 8C is trimmed due to violation of first joint's limit prior to learning.

Modelling aspect-specific behaviour

Previously (Section 1), a single demonstration was used to model aspect behaviour. We now extend this to multiple demonstrations within the same aspect. Consider the aspect-specific set of grouped K joint trajectories $\{\mathbf{Q}\}_{\mathcal{A}_i}$ for modelling the behaviour in $\mathcal{A}_i$. These trajectories have a common goal configuration $\overset{*}{\mathbf{q}}_i$ where $\mathcal{A}(\overset{*}{\mathbf{q}}) = \mathcal{A}_i$.

Since this section focuses on modelling a single aspect's behaviour, we hereby drop the subscript i to refer to a specific aspect in this Section. To model the aspect-specific joint behaviour that converges at $\overset{*}{\mathbf{q}} \in \mathcal{A}$, we embed each associated demonstration as a dynamical system by following the procedure in Section . The library of these K DSs - $\{\mathrm{g}_j^{(Q)} \leftrightarrow (\psi_j, \mathbf{k}_j)\}_{j=1}^{K}$ - embed the instances of demonstrated behavior in $\mathcal{A}$ to be replicated (refer Equation). Corresponding to each embedding, we partition the aspect $\mathcal{A}$ into K deformed conical regions $\{\mathcal{R}_i\}_{i=1}^{K}$, where each region shares a common vertex at $\overset{*}{\mathbf{q}}$. $\mathcal{R}_i$ represents a region of $\mathcal{A}$ that makes up the *neighbourhood* of the demonstration $\mathbf{Q}_i$, and is exclusively governed by associated DS embedding $\mathrm{g}_i^{(Q)}$

$$\dot{\mathbf{q}} = \mathrm{g}_i^{(Q)}(\mathbf{q}) \text{ if } \mathbf{q} \in \mathcal{R}_i$$

The overall partitioning of aspect $\mathcal{A}$ follows

$$\mathcal{A} = \bigcup_{i=1}^{K} \mathcal{R}_i, \quad \mathcal{R}_i \cap \mathcal{R}_j \neq \emptyset \text{ for some } i \neq j. \tag{S15}$$

There may exist overlap between some or all of the K different partitions. To disambiguate the selection of relevant DS in such cases, we assign a priority $P(\mathcal{R}_i) = i$ to each $\mathcal{R}_i$ based on the chronology of the demonstration that corresponds to the relevant embedding $\mathrm{g}_i^{(Q)}$. This makes the first demonstration of the least priority, and subsequent demonstrations get assigned progressively higher priorities. For any point $\mathbf{q} \in \mathcal{A}$, the assigned region $\mathcal{R}(\mathbf{q})$ is determined by:

$$R(\mathbf{q}) = \{\mathcal{R}_i \mid \mathbf{q} \in \mathcal{R}_i, P(\mathcal{R}_i) = \max_{\mathcal{R}_j, \mathbf{q} \in \mathcal{R}_j} P(\mathcal{R}_j)\} \tag{S16}$$

This formulation ensures that if multiple regions overlap at a point $\mathbf{q}$, the region (and by extension the associated DS embedding $\mathrm{g}_i^{(Q)}$) with the highest priority takes precedence. The velocity at a point $\mathbf{q} \in \mathcal{A}$ is determined as

$$\dot{\mathbf{q}} = \mathrm{g}_i^{(Q)}(\mathbf{q}) \text{ where } i = \arg\max\{P(\mathcal{R}_i) \mid \mathbf{q} \in \mathcal{R}_i\} \tag{S17}$$

It must be noted that the priority order may also be composed by the expert, or even randomly allocated, especially if the chronology of demonstrations is unknown, or the demonstrations are provided simultaneously.

Since all DS embeddings are globally asymptotically stable (Equation S5) and share a common attractor $\overset{*}{\mathbf{q}}$, there exists at least one highest-priority embedding $\mathrm{g}_i^{(Q)}$ for all $\mathbf{q} \in \mathcal{A}$ ensuring convergence at the attractor

$$P(\mathcal{R}(\mathbf{q})) = \max_{1 \le i \le K} P(\mathcal{R}_i) \implies$$

$$\lim_{t\to\infty} \mathbf{q} = \lim_{t\to\infty} \psi_i\big(\psi_i^{-1}(\mathbf{q})\big) \tag{S18}$$

$$= \psi_i\Big(\lim_{t\to\infty} \psi_i^{-1}(\mathbf{q})\Big) \tag{S19}$$

$$= \psi_i(\mathbf{1}_n^T) = \overset{*}{\mathbf{q}}, \tag{S20}$$

Partitioning in the latent space

The neighbourhood of a demonstration $\mathbf{Q}_i$ in aspect $\mathcal{A}$, given by region $\mathcal{R}_i$, is smoothly mapped to a region $\mathcal{L}_i = \psi_i^{-1}(\mathcal{R}_i)$ around the latent embedding of the demonstration $\psi_i^{-1}(\mathbf{Q}_i)$ in latent space $\sqcap$. As the topology and differentiable structure of the neighbourhood region is preserved between $\mathcal{R}_i \subset \mathcal{A}$ and $\mathcal{L}_i \subset \mathcal{U}$, we can instead define the neighbourhood in latent space $\sqcap$. Doing so allows us to leverage the (quasi)linear structure in latent space $\sqcap$, to determine the non-linear neighbourhood region in joint space $\mathcal{Q}$. In latent space $\sqcap$, we define the neighbourhood $\mathcal{L}_i$ as a straight volume cone with an opening angle λ_i, whose axis is aligned with the vector $\mathbf{v} = \overset{*}{\mathbf{u}} - \mathbf{0}$ along the latent embedding of the demonstration, and its vertex is at $\overset{*}{\mathbf{u}}$. Here, $\overset{*}{\mathbf{u}} = \mathbf{1}_n^T$ is the latent embedding of $\overset{*}{\mathbf{q}} \in \mathcal{A}$ and $\mathbf{0}$ is the latent embedding of the first point in demonstration $\mathbf{Q}_i$. The cone is defined as the collection of joint space points

$$\mathcal{L}_i = \{\angle(\overset{*}{\mathbf{u}} - \psi_i^{-1}(\mathbf{q}), \mathbf{v}) \le \lambda_i\} \; \forall \; \mathbf{q} \in \mathcal{A} \tag{S21}$$

and represents the set of all points in latent space $\mathcal{U}$ for DS embedding $\mathrm{g}_i^{(Q)}$, for which the angle between the direction $\psi_i^{-1}(\mathbf{q}) - \overset{*}{\mathbf{q}}$ and the axis $\mathbf{v}$ does not exceed λ_i. The latent embedding of the demonstration $\mathbf{Q}_i$ lies along the axis of this *cone of influence*, ensuring that the cone encapsulates a neighbourhood around the demonstration in latent space $\sqcap$. Figure 8D illustrates the cones of influence, in the latent space associated with each DS embedding, for every aspect. In $\mathcal{A}$, $\mathcal{R}_i = \psi_i(\mathcal{L}_i)$ defines the neighbourhood. Consequently, Equation S17 can be reformulated as:

$$\dot{\mathbf{q}} = \mathrm{g}_i^{(Q)}(\mathbf{q}), \; i = \arg\max\{P(\mathcal{L}_i) \mid \psi_i^{-1}(\mathbf{q}) \in \mathcal{L}_i\} \tag{S22}$$

Calculating the partitions

Each cone $\mathcal{L}_i$ is uniquely parametrised by its opening angle $\lambda_i \in [\epsilon_\lambda, \pi]$, where ϵ_λ is the minimal angular value (in radians) for the opening angle. Higher the opening angle, wider the cone, and hence more region in latent space $\sqcap$ and joint space $\mathcal{Q}$ which is governed by $\mathrm{g}_i^{(Q)}$. As a result, these angles determine the overall influence of each of the learnt embeddings $\mathrm{g}_i^{(Q)}$ over the resulting behaviour in joint space. The first (and lowest priority) embedding is assigned a cone angle of π, effectively spanning the entire aspect. As more (and higher priority) embeddings are available, they get assigned progressively lower values. Maximising the neighbourhood region of each embedding, will lead to a better overall generalisation of the expert demonstrations in $\mathcal{A}$. However, it is also crucial to minimise the overlap among the partitions $\mathcal{R}_i$, to prevent an unrelated higher priority embedding $\mathrm{g}_H^{(Q)}$ from incorrectly overlaying the the desired dynamics which were to be ideally generated using a lower priority embedding $\mathrm{g}_L^{(Q)}$. To identify and subsequently eliminate such overlaps, we measure the influence of a DS learnt using a higher priority demonstration $\mathrm{g}_H^{(Q)}$ over a lower priority demonstration $\mathbf{Q}_L \in \mathcal{A}$. For a faithful behaviour replication, at least no joint configuration from $\mathbf{Q}_l$ should lie in the influence cone of $\mathrm{g}_H^{(Q)}$. Thus, the cone angle for $\mathrm{g}_H^{(Q)}$ is set as the maximum permissible value such that no joint configurations from a lower priority demonstration $\mathbf{Q}_l$ lie in it.

To do so, we define the set of projection angles $\Theta_H(\mathbf{Q}_l)$ for embedding $\mathrm{g}_H^{(Q)}$ as:

$$\Theta_H(\mathbf{Q}_l) = \{\angle(\psi_H^{-1}(\mathbf{q}_j) - \mathbf{1}_n^T, \mathbf{1}_n^T - \mathbf{0}_n^T)\}_{j=1}^{M_l} \tag{S23}$$

where M_l is the number of demonstrated points in $\mathbf{Q}_l$, and $\psi_H^{-1}(\mathbf{q}_j)\ \forall\ \mathbf{q}_j \in \mathbf{Q}_l$ denotes the latent embedding of $\mathbf{Q}_l$ generated in the latent space associated with $\mathrm{g}_H^{(Q)}$. Furthermore, $\Theta_H(\mathbf{Q}_l)$ is the set of the angles subtended by the vectors between the aforementioned latent embedding of demonstrated points $\psi_i^{-1}(\mathbf{Q}_l)$ and latent space's origin, and the vector between the origin and the attractor ($\mathbf{1}_n^T$) for some lower priority demonstration $\mathbf{Q}_l$. Points in $\mathbf{Q}_l$ that have higher projection angles represent more significant deviation of the demonstration, in contrast to those with lower projection angles.

Assuming ascending order of priority based on chronology of demonstrations, the opening angles $\lambda_i, i \in \{1, 2 \cdots K\}$ that prevent a lower priority demonstration being overlayed by a higher priority demonstrations DS can be determined as:

$$\begin{aligned}
&\lambda_1 = \pi \text{ (least priority demonstration for entire aspect)} \\
&\lambda_i = \min(\{\Theta_i(\mathbf{Q}_l)\}_{l=1}^{i-1}) \text{ for } 2 \leq i \leq K \\
&\text{if } \lambda_i < \epsilon_\lambda, \quad \lambda_i \leftarrow \epsilon_\lambda
\end{aligned} \tag{S24}$$

The set of these opening angles, collectively referred to as $\Lambda = \{\lambda_i\}_{i=1}^{K}$, ensures that the cone of influence of some high priority demonstration $\mathcal{L}_H$ does not overlap with the latent embedding of any of the lower priority demonstrations $\{\psi_H^{-1}(\mathbf{Q}_l)\}_{l=1}^{H-1}$. The cone angle for the first demonstration is set to π and encompasses the entire aspect. However, as soon as a second demonstration (with higher priority) is available, a region of the aspect is now governed by this second demonstration's embedded DS. The cone angles get updated as new demonstrations become available (refer to Section *Trajectory generation*). Enforcing the lower bound of ϵ_λ on the cone angles ensures that in case there are intersections between two distinct demonstrations (in workspace or their joint space counterparts), the cone angle in Equation S24 does not assume a negligible value. If $\lambda_i \approx 0$ for demonstration $\mathbf{Q}_i$, the learnt DS embedding would have no influence over the joint space behaviour, except for a very narrow volume of joint space around $\mathbf{Q}_i$. Having a lower bound of $\epsilon_\lambda > 0$ prevents this. Though this may lead to a higher priority DS embedding abruptly switching the dynamics of the trajectory being generated by a lower priority DS if left unchecked, the extension to prevent this during robot execution is presented in Section *Trajectory generation*.

The composition of the priority ordering of the embeddings P, the opening angles Λ, and the learnt DS embeddings $\{\mathrm{g}_i^{(Q)}\}_{i=1}^{K}$, comprehensively describe the behaviour in the aspect $\mathcal{A}$ of the joint space $\mathrm{B}^{\mathcal{A}}$ as

$$\begin{aligned} &\mathrm{B}^{(\mathcal{A})} = \{P, \Lambda, \{\mathrm{g}_i^{(Q)} \leftrightarrow (\psi_i, \mathbf{k}_i)\}\} \\ &\dot{\mathbf{q}} = \mathrm{B}^{(\mathcal{A})}(\mathbf{q}) = \mathrm{g}_i^{(Q)}(\mathbf{q}) \text{ where} \\ &R(\mathbf{q}) = \mathcal{R}_i,\ P(\mathcal{R}_i) = \max_{1 \le j \le K} P(R_j) \end{aligned} \tag{S25}$$

Incremental learning

When a new demonstration $\mathbf{Q}_{K+1}$ becomes available, only the corresponding embedding $(\psi_{K+1}, \mathbf{k}_{K+1})$ is learned and the list of embeddings gets updated. The set of opening angles Λ are also iteratively modified to avoid overlap with the new demonstration

$$\begin{aligned} &\lambda_i \leftarrow \min(\lambda_i, \Theta_i(\mathcal{Y}_{K+1}))\ \forall\ i \in \{1, 2 \cdots K-1\} \\ &\lambda_K \leftarrow \min(\Theta^K(\mathcal{Y}_{K+1})) \\ &\lambda_{K+1} \leftarrow \frac{1}{K}\sum_{i=1}^{K} \lambda_i \end{aligned} \tag{S26}$$

Lastly, the priority order of the associated partition is updated $P(L_{K+1}) = K + 1$

Overall joint space behaviour

Post sorting all the joint demonstrations, each set aspect-specific set of demonstrations $\{\mathbf{Q}\}_{A_i}$ is used to learn the respective aspect-specific behaviour $\mathrm{B}^{(\mathcal{A}_i)}$ (refer Equation S25) Thus, for a query joint configuration $\mathbf{q}$, the dynamics are characterised by aspect containing $\mathbf{q}$. The aspect-specific behaviours collectively describe the behaviour in joint space $\mathcal{Q}$ hereby referred to as $\mathrm{B}^{(\mathcal{Q})} = \{\mathrm{B}^{(\mathcal{A}_1)}, \mathrm{B}^{(\mathcal{A}_2)} \cdots \mathrm{B}^{(\mathcal{A}_{N_\mathcal{A}})}\}$ where $N_\mathcal{A}$ is the number of aspects. For a $2R$ robot, there will be upto two functionals in $\mathrm{B}^{(\mathcal{Q})}$, whereas for a noncuspidal $3R$ robot, there can be upto eight. Figure 8(B-E) illustrates the process of modelling the behaviour from multiple demonstrations, both in the workspace and the joint space. The workspace behaviour $\mathrm{B}^{(\mathcal{X})}$ is learnt using two demonstrations. In the joint space, each of the two aspects also have two demonstrations each. The aspect specific behaviours $\mathrm{B}^{(\mathcal{A}_1)}$, and $\mathrm{B}^{(\mathcal{A}_2)}$ are learnt using the two demonstrations in each aspect. The corresponding cones of influence in the latent space, both for the workspace behaviour and aspect-specific behaviours, correspond to regions in workspace and joint space where a specific demonstration influences the behaviour.

Certified execution

To generate the trajectory from a query joint configuration $\mathbf{q}$, it is crucial to first determine the aspect which $\mathbf{q}$ belongs to and then utilise the relevant aspect-specific behaviour $\mathrm{B}^{(\mathcal{A}_i)}$. This is because $\mathrm{B}^{(Q)}$ is a multi-attractor system where the attractors or goal configurations are given by $\{\overset{*}{\mathbf{q}}_i \mid \mathrm{FK}(\overset{*}{\mathbf{q}}_i) = \overset{*}{\mathbf{x}}\}_{i=1}^{\mathcal{I}}$.

To formulate the selection process, we leverage the fact that each $\mathrm{B}^{(\mathcal{A}_i)}$ has a specific goal configuration $\overset{*}{\mathbf{q}}_i$ associated with it. Thus, each aspect specific behaviour can be identified via the goal joint space configuration it converges at. Now, instead of determining the aspect of query configuration $A(\mathbf{q})$, we can simply check which goal configuration $\overset{*}{\mathbf{q}}_i$ belongs to the same aspect as $\mathbf{q}$.

Connectivity analysis

Establishing a *connection* between the query and goal configuration without violating any aspect boundary uniquely determines the aspect-specific behaviour to choose. Recently, roadmap algorithms were proposed for connectivity analysis (*47*), which become challenging as they use real algebraic formulation and the rational parametrization of trigonometric terms at π radians is singular. Though this algorithm can be extended to consider joint limits, no work has been reported regarding it. In Algorithm 14, we propose a certified connectivity check algorithm that leverages the properties of singularities to determine the connectivity of the query and goal joint configurations, while also considering joint limits. Here, certified algorithm means an algorithm that provides worst-case and beyond worst-case performance guarantees (*48*). To assess the connectivity between query and goal, we consider each factor f_i (upto three) of the expression of Jacobian determinant $\det \mathbf{J}(\mathbf{q})$ independently. Each factor is a branch of the singularity, and a boolean *connectivity* with respect this factor between the query and goal is determined. If connectivity via each factor can be established, we can conclude that the goal and query are connected, and hence belong to a common aspect. The different types of factors, that can arise for noncuspidal $3R$ robots (covered by Category I through Category VI) as discussed in Section 4 can be characterised by the description of their branches (using Algorithm 1). A factor can have branches of one of the five types (refer to Table S2) viz;

1. $(1, 0)[\geq 2, 0]$ (e.g. Category I (row (A)), Category III (rows (F) and (H)), and Category V in Figure S2)

2. $(0, 1)[0, *]$ (e.g. Category II (row (B), and Category III (row (F)) in Figure S2)

3. $(0, 0)[2, \geq 2]$ (e.g. Category IV (row (C)), and Category V (row (G)) in Figure S2)

4. $(1,0)[2,>0]$ (refer to Category VI (row (D)) in Figure S2)

5. $(>0,1)[0,*]$ (refer to Category II (row (E)), and Category III (row (H)) in Figure S2)

Case (i): As two joint configurations in the same aspect should have the same sign of f_i, the connectivity algorithm verifies the signature of $\mathrm{f}_i(\mathbf{q}_A)$ and $\mathrm{f}_i(\mathbf{q}_B)$. Later, to determine the connectivity between two joint configurations $\mathbf{q}_A$ and $\mathbf{q}_B$ with the same sign of f_i of robots belonging to the first case, we consider the paths along the q_2 generator that passes through each horizontal turning point. If we have a nongeneric branch, the branch itself is the path we consider. As joint limits are linear paths, they are also easily included as a path for consideration. Both $\mathbf{q}_A$ and $\mathbf{q}_B$ are projected on each path. Let $\mathbf{p}_A(q_3)$ be the value of q_3 where the f_i value at $\mathbf{q}_A$ and its projection on the path through q_3 match. If the number of common q_3 values for $\mathbf{p}_A$ and $\mathbf{p}_B$ is non zero, then $\mathbf{q}_A$ and $\mathbf{q}_B$ are connected as presented in Algorithm 14 and illustration of an example case is shown in Figure S15.

Case (ii): When factors encircle the q_3 generator, the analysis differs as now we can have more than two vertical turning points. In such cases, we check the intersection of the path, $q_3 = \mathbf{q}_A(q_3)$ with the singularity as well as the joint limits, and verify the branches of the neighborhood points of intersection (say $\mathbf{q}_{A1}$ and $\mathbf{q}_{A2}$ for $\mathbf{q}_A$). As there can exist at most two $(1,0)$ branches, we can identify them with the direction of the gradient of Jacobian determinant $\det \mathbf{J}(\mathbf{q})$. The connectivity of the query and goal configuration is then decided by investigating the type of branches on which $\mathbf{q}_{A1}, \mathbf{q}_{A2}, \mathbf{q}_{B1}$ and $\mathbf{q}_{B2}$ lie on. The connectivity algorithm for this case is discussed in Algorithm 16, and illustration of an example case is shown in Figure S17.

Case (iii): For singularities with homotopy case $(0,0)$, we need to consider if the configurations are connected even after being split by joint limits. For this purpose, we consider a range of joint space with length 2π where the loop is not split. The horizontal and vertical turning points, the intersections of singularities with joint limit and the configurations in query are all translated along q_2 and q_3 to lie on or inside the loop in this new range. Once the loop closure is done, the connectivity of query configurations is investigated based on the shift required to place the query configurations inside the loop. The algorithm for connectivity check of such singularities is presented in Algorithm 17, and illustration of an example case is shown in Figure S18.

Case (iv): For singularities with homotopy case $(1, 0)[2, a > 0]$, we need to consider if the configurations are connected even after being split by joint limits and later using an algorithm similar to Case (i) to decide connectivity. In this case, the knowledge of f_i is leveraged to know that there can be at most one branch that is split by q_2 joint limits as well as q_3 joint limits. The algorithm for connectivity check of such singularities is presented in Algorithm 18, and illustration of an example case is shown in Figure S16.

Case (v): For singularities with homotopy case $(1, 1)$ and $(2, 1)$, we leverage the fact that the singularities encircle both q_2 and q_3 generators. This implies that each aspect is necessarily bounded by singularities as well as joint limits. The joint limits for each aspect are unique, and thus, the range of joint limits is used to identify the aspect and, thus, the connectivity between two configurations. The algorithm for connectivity check of such singularities is presented in Algorithm 19, and illustration of an example case is shown in Figure S19.

The singularities for all noncuspidal $3R$ robots, covered by Category I through Category VI, are a combination of the connectivity check applied to each type of factor occurring in the singularity. In doing so, the proposed framework can verify connectivity among the joint configurations in a deterministic manner with a constant time complexity, as the maximum number of steps in any algorithm is always finitely bounded. These properties of the connectivity check allow the query to be allocated a finite time and computation resource, which are important considerations for any real-time behavior.

The connectivity checks for the factor classes (refer Table S3 are presented at the end of this section.

Trajectory generation

The steps to generating the trajectory from a query joint configuration $\mathbf{q}$, while not violating the kinematic constraints, can be summarised as follows:

1. Determine suitable aspect-specific behaviour $\mathrm{B}^{(\mathcal{A})}$ from the known set of behaviours $\mathrm{B}^{(Q)} = \{\mathrm{B}^{(\mathcal{A}_1)}, \mathrm{B}^{(\mathcal{A}_2)} \cdots \mathrm{B}^{(\mathcal{A}_{N_{\mathcal{A}}})}\}$
2. Determine suitable joint DS embedding $\mathrm{g}^{(Q)}$ from the embeddings in aspect-behaviour $\mathrm{B}^{(\mathcal{A})} = \{P, \Lambda, \{\mathrm{g}_i^{(Q)} \leftrightarrow (\psi_i, k_i)\}\}$
3. Formulate near-boundary behaviour near violation boundaries if and when needed

We begin by identifying the correct goal configuration, and the aspect-specific behaviour $\mathrm{B}^{(\mathcal{A})}$ to be used, using the relevant connectivity check (refer to Table S3). Next, the relevant DS embedding $\mathrm{g}^{(Q)}$ is picked on the basis of the priority P and the cones of influence Λ using Equation S22.

Note that when determining the relevant embedding, we measure the angle subtended between the vectors $\overset{*}{\mathbf{u}} - \psi_i^{-1}(\mathbf{q})$ and $\overset{*}{\mathbf{u}} - \mathbf{0}$ (Equation S21) to check which embeddings' cones of influence does $\mathbf{q}$ reside in, and then promptly select the highest priority embeddings from amongst them. However, if there was an intersection between the distinct user demonstrations, merely prioritising the highest priority embedding may lead to an abrupt switching in the dynamics at the point of intersection. To avoid such situations, if the angle subtended for the suitable DS embeddings is lower than the minimal cone angle ϵ_λ, we retain the embedding that was being used instead of simply selecting the highest priority embedding.

$\mathrm{B}^{(Q)}$ models the behaviour in each aspect of joint space Q, such that stable convergence at the relevant attractor $\overset{*}{\mathbf{q}}$ is guaranteed. The learnt DS embeddings are globally asymptotically stable, and invariant to spatial or temporal perturbations. However, if due to a perturbation or otherwise, the instantaneous joint configuration $\mathbf{q}$ is detected to be in a near-boundary region, the near-boundary control law $\mathrm{g}^{(\mathcal{B})}$ (Table S1) is formulated and overlays the learnt behaviour. To determine the near-boundary behaviour, the projected trajectory $\mathbf{Q}$ starting from $\mathbf{q}$, encountering violations and converging at the goal configuration, is required to determine the $\mathbf{q}_{\text{last entry}}$ and $\mathbf{q}_{\text{first exit}}$ configurations in the current aspect.

Oneshot trajectory generation for violation detection

Utilising the properties of the latent embedding and the learnt diffeomorphic mapping, permits the entire trajectory $\mathbf{Q}$ to be evaluated in batch, without recursive integration. First, the latent embedding for the projected trajectory $\mathbf{U}$ is generated using a predefined temporal formulation. Next, the diffeomorphic transformation ψ is used to transform the embedding into the joint space trajectory $\mathbf{Q}$ for use in violation detection.

For an initial point $\mathbf{q}$, the identified DS embedding $\mathrm{g}^{(Q)} \leftrightarrow (\psi, \mathbf{k})$ provides

$$\begin{aligned} \mathbf{u}_{\text{start}} &= \psi^{-1}(\mathbf{q}) \\ \acute{\mathbf{u}}_{\text{start}} &= \mathbf{1} + \|\mathbf{1} - \mathbf{u}_{\text{start}}\| \frac{\mathbf{1} - \mathbf{0}}{\|\mathbf{1} - \mathbf{0}\|} \end{aligned} \tag{S27}$$

where $\mathbf{u}_{\text{start}}$ is the latent embedding of $\mathbf{q}$, and $\acute{\mathbf{u}}_{\text{start}}$ is its projection along the latent embedding of the original demonstration. The demonstration's embedding is a quasilinear distribution along the vector from $\mathbf{0}$ (start) to $\mathbf{1}$ (goal). From Appendix H (*33*), when $\|\mathbf{1} - \mathbf{u}_{\text{start}}\| \leq \|\mathbf{1}\|$, the latent embedding of the original

demonstration follows

$$\mathbf{u}_t = \sin(\mathbf{k}t) \tag{S28}$$

The original formulation represents the constant $\mathbf{k} \in \mathrm{R}_+^n$ as $\dfrac{\mathbf{a}}{\Delta t}$ where $\mathbf{a} \in \mathrm{R}_+^n$, and $\Delta t > 0$ is the sampling interval associated with the demonstration.

By using $\acute{\mathbf{u}}_{\text{start}}$ and the temporal formulation of original demonstration's latent embedding (Equation S28), we can estimate the latent embedding between $\mathbf{u}_{\text{start}}$ and $\mathbf{1}$

$$\begin{aligned} t_0 &= \frac{\arcsin(\acute{\mathbf{u}}_{\text{start}})}{\mathbf{k}} \\ \acute{\mathbf{u}}_i &= \sin(\mathbf{k}(t_0 + i\Delta t)) \text{ for } i = \{0, 1, \cdots \frac{T}{\Delta t}\} \\ \mathbf{U} &= \{\mathbf{1} + \frac{\mathbf{u}_{\text{start}} - \mathbf{1}}{\|\mathbf{u}_{\text{start}} - \mathbf{1}\|}\|\acute{\mathbf{u}}_i - \mathbf{1}\|\}_{i=0}^{T/\Delta t} \end{aligned} \tag{S29}$$

where T is the total duration of the original demonstration, Δt is the sampling frequency, and $\mathbf{U}$ is the latent embedding of the projected trajectory.

In situations where $\|\mathbf{1} - \mathbf{u}_{\text{start}}\| > \|1\|$, we first identify the point $\mathbf{u}_0$ along the segment between $\mathbf{u}_{\text{start}}$ and $\|\mathbf{1}\|$, such that $\|\mathbf{1} - \mathbf{u}_0\| = \|\mathbf{1}\|$

$$\mathbf{u}_0 = \mathbf{1} + \frac{\mathbf{u}_{\text{start}} - \mathbf{1}}{\|\mathbf{u}_{\text{start}} - \mathbf{1}\|}\|\mathbf{1}\| \tag{S30}$$

The first portion of the latent embedding of the projected path $\mathbf{U}$ is uniformly distributed along the linear segment between $\mathbf{u}_{\text{start}}$ and $\mathbf{u}_0$. The remainder of embedding between $\mathbf{u}_0$ and the goal $\|\mathbf{1}\|$ is generated using Equation S29.

Finally, the joint space trajectory $\mathbf{Q}$ is generated using diffeomorphic transformation ψ as

$$\mathbf{Q} = \psi(\mathbf{U})$$

The resulting track cycle $\mathrm{C}(\mathbf{q}_{\text{last entry}})$ now maps the trajectory between $\mathbf{q}_{\text{first exit}}$ and $\mathbf{q}_{\text{last entry}}$ from $\mathbf{Q}$. Once in the vicinity of $\mathbf{q}_{\text{last entry}}$, the $\mathrm{B}^{(Q)}$ once again resumes control and eventually converges at $\overset{*}{\mathbf{q}}$

Resulting behaviour

The hybrid approach leveraging both $\mathrm{B}^{(Q)}$ and the near-boundary behaviour $\mathrm{C}(\mathbf{q}_{\text{last entry}})$ when required, ensures stable convergence at relevant attractor configuration, being robust to perturbations without trying to overshoot the joint limits or cross a singularity. The control law is presented in Algorithm 13.

Algorithm 13. Joint Velocity at Query Configuration q

Input: Query joint configuration $\mathbf{q}$

Output: Joint velocity $\dot{\mathbf{q}}$

Set of goal configurations:

$\{\overset{*}{\mathbf{q}}\} \leftarrow \{\overset{*}{\mathbf{q}}_1, \overset{*}{\mathbf{q}}_2, \ldots, \overset{*}{\mathbf{q}}_{N_{\mathcal{A}}}\}$

Determine aspect:

Identify $\mathcal{A}$ such that $\mathcal{A} \leftarrow \text{connectivity}(\mathbf{q}, \overset{*}{\mathbf{q}}_i)$ for all $\overset{*}{\mathbf{q}}_i \in \{\overset{*}{\mathbf{q}}\}$

If $\mathbf{q}$ is not connected to any $\overset{*}{\mathbf{q}}_i$:

Do not initiate motion

Else: Identify corresponding aspect-specific behaviour $\mathrm{B}^{(\mathcal{A})}$

Identify relevant embedding in $\mathrm{B}^{(\mathcal{A})}$**:**

$\mathrm{B}^{(\mathcal{A})} = \{P, \Lambda, \{\mathrm{g}_i^{(Q)} \leftrightarrow (\psi_i, \mathbf{k}_i)\}\}$

$\mathrm{g}^{(Q)} \leftarrow \mathrm{g}_i^{(Q)}(\mathbf{q})$,

where $i = \arg\max\{P(\mathcal{L}_i) \mid \psi_i^{-1}(\mathbf{q}) \in \mathcal{L}_i\}$

If $\mathbf{q}$ is not in any near-boundary region:

Use the learned behaviour $\mathrm{g}^{(Q)}$

Return $\dot{\mathbf{q}} = \mathrm{g}^{(Q)}(\mathbf{q})$

Else: Overlay with near-boundary control policy

If track cycle $\mathrm{C}(\mathbf{q}_{\text{last entry}})$ is not determined:

$\mathbf{Q} \leftarrow$ Oneshot formulation (Sec. 12)

Determine $\mathrm{C}(\mathbf{q}_{\text{last entry}})$ (Table S3)

Formulate $\mathrm{g}^{(\mathcal{B})}(\mathbf{q}, \mathbf{q}_{\text{last entry}})$

Note: $\mathrm{C}(\mathbf{q}_{\text{last entry}})$ is computed only once per region entry

and reused until convergence at $\mathbf{q}_{\text{last entry}}$

Return $\dot{\mathbf{q}} = \mathrm{g}^{(\mathcal{B})}(\mathbf{q}, \mathbf{q}_{\text{last entry}})$

Algorithms for connectivity check

We present the remainder of connectivity check algorithms presented in Section 12 (refer Table S3). The connectivity check is defined for each of the possible type of factors of expression of Jacobian determinant $\det \mathbf{J}(\mathbf{q})$. In the following body, we presents checks to assess connectivity between query joint configurations $\mathbf{q}_A$ and $\mathbf{q}_B$.

Case (i)

The check for Case (i), when the factor class is $2(1,0)[*,0]$, is presented below in Algorithm 14.

Algorithm 14. Case (i): factor with branch type $(1,0)[*,0]$

Input: Joint configurations $\mathbf{q}_A$, $\mathbf{q}_B$

Output: Boolean indicating connectivity without crossing singularities

Check for singularity separation:

If $\operatorname{sign}(\det \mathbf{J}(\mathbf{q}_A)) \neq \operatorname{sign}(\det \mathbf{J}(\mathbf{q}_B))$:

Points are separated by a singularity

Return: false

Make horizontal connection lines H:

$H \leftarrow \{h_i : q_3 = \mathbf{q}^i(q_3)\}$ for all $\mathbf{q}^i \in Q^h \cup \{\underline{q_3}, \overline{q_3}\}$

Make vertical connection lines V:

$V \leftarrow \{v_1 : q_2 = \overline{q_2}, v_2 : q_2 = \underline{q_2}, \}$

Project points orthogonally onto horizontal and vertical lines:

$P_H^{A,B}(q_2, q_3) \leftarrow \{q_2^{A,B}, \mathbf{q}^h(q_3)\}$

$P_V^{A,B}(q_2, q_3) \leftarrow \{\mathbf{q}^v(q_2), q_3^{A,B}\}$

For each $m \in \{A, B\}$**:**

Identify projections with same sign of Jacobian determinant

$Z_{H,V}^m \leftarrow \{\det \mathbf{J}(\mathbf{p}_i) \cdot \det \mathbf{J}(\mathbf{q}_m) \geq 0 \ \forall \ \mathbf{p}_i \in P_{H,V}^m\}$

Obtain valid indices using continuity check (Algorithm 15):

$I_H^m \leftarrow P_H^m(q_3), Z_H^m, \mathbf{q}_m(q_3)$

$I_V^m \leftarrow P_V^m(q_2), Z_V^m, \mathbf{q}_m(q_2)$

Identify common valid lines:

$H_C \leftarrow I_H^A \cap I_H^B$

$V_C \leftarrow I_V^A \cap I_V^B$

Return: $|H_C| \geq 1$ and $|V_C| \geq 1$

The continuity check in Algorithm 15 is used in conjunction with Algorithm 14 for factor with branches $(1, 0)$ in Case (i) Table S3.

Algorithm 15. Continuity check for Case (i)

Input: Projections $P(q_i)$, query value q_i^c, boolean mask Z from Algorithm 14

Output: Index ranges $\{a : b, d : c\}$ indicating connectivity windows

Sort projections and align booleans:

Let π be the permutation that sorts $P(q_i)$ in ascending order

$P_{\text{sorted}} \leftarrow P(q_i)[\pi]$

$Z_{\text{sorted}} \leftarrow Z[\pi]$

Identify ascending connectivity interval:

$a \leftarrow \min\{k \mid P_{\text{sorted}}[k] \geq q_i^c\}$

$b \leftarrow$ smallest index such that $(b \geq a) \wedge (Z_{\text{sorted}}[b+1] = \text{false})$

Identify descending connectivity interval:

$c \leftarrow \max\{k \mid P_{\text{sorted}}[k] < q_i^c\}$

$d \leftarrow$ largest index such that $(d < c) \wedge (Z_{\text{sorted}}[d-1] = \text{false})$

Return: $\{a : b, d : c\}$

Case (ii)

Factor f with branch type $(0, 1)$ and no horizontal tuning points, has two branches by encircling the q_3 generator twice. The set I_{SL} contains the points of intersection between the joint limits $q_2^{\times}, q_3^{\times}$ and the factor f. The joint space regions in vicinity of these branches are denoted as B_+, B_- To determine a joint configuration is in vicinity of which of the two branches, we can utilise the sign of the factor:

$$B = \begin{cases} +1 & \text{if } \operatorname{sign} \mathrm{f}(\mathbf{q} + [\epsilon_q, 0]^T) > 0 \\ -1 & \text{if } \operatorname{sign} \mathrm{f}(\mathbf{q} + [\epsilon_q, 0]^T) < 0 \end{cases} \tag{S31}$$

Note, a configuration lies on $q_2^{\times}$, the branch is assigned using any configuration in I_{SL}.

The check is presented in Algorithm 16. Figure S17 presents the check process in $q_2 - q_3$ slice for robot from Category II (B).

Algorithm 16. Case (ii): factor with branch type $(0, 1)[0, *]$

Input: Joint configurations $\mathbf{q}_A$ and $\mathbf{q}_B$

Output: Boolean indicating if configurations are connected

Singularity check:

If $\text{sign}(\det(\mathbf{J}(\mathbf{q}_A))) \neq \text{sign}(\det(\mathbf{J}(\mathbf{q}_B)))$ then

Returnfalse

Construct horizontal lines through $\mathbf{q}_A$ and $\mathbf{q}_B$:

$H_A \leftarrow \{\mathbf{q} \mid \text{f}(\mathbf{q}) = 0, \mathbf{q}(q_3) = \mathbf{q}_A(q_3)\} \cup \{[\underline{q_2}, \mathbf{q}_A(q_3)], [\overline{q_2}, \mathbf{q}_A(q_3)]\}$

$H_B \leftarrow \{\mathbf{q} \mid \text{f}(\mathbf{q}) = 0, \mathbf{q}(q_3) = \mathbf{q}_B(q_3)\} \cup \{[\underline{q_2}, \mathbf{q}_B(q_3)], [\overline{q_2}, \mathbf{q}_B(q_3)]\}$

Identify nearest neighbours:

$\mathbf{q}_{A1}, \mathbf{q}_{A2} \leftarrow$ neighbours of $\mathbf{q}_A$ along q_2 in H_A

$\mathbf{q}_{B1}, \mathbf{q}_{B2} \leftarrow$ neighbours of $\mathbf{q}_B$ along q_2 in H_B

Identify branches for each neighbour:

$B_{A1}, B_{A2} \leftarrow$ branches for $\mathbf{q}_{A1}$ and $\mathbf{q}_{A2}$

$B_{B1}, B_{B2} \leftarrow$ branches for $\mathbf{q}_{B1}$ and $\mathbf{q}_{B2}$

Note: If point lies on $q_2^{\times}$, assign branch as 0

Check connectivity:

If $B_{A1} \neq B_{A2} \wedge B_{B1} \neq B_{B2}$:

Return: true

If $(B_{A1} = B_{A2} \wedge B_{B1} \neq B_{B2}) \vee (B_{B1} = B_{B2} \wedge B_{A1} \neq B_{A2})$:

Return: false

If $B_{A1} = B_{A2} \wedge B_{B1} = B_{B2}$:

If $\mathbf{q}_A(q_3)$ and $\mathbf{q}_B(q_3)$ lie between the same I_{SL} pair along the $q_2 = \overline{q_2}$ or $\underline{q_2}$:

Return: true

Else:

Return: false

Return: false

Case (iii)

Factor f with class $(0, 0)$ has a single branch, forming a closed loop, without encircling any generator.

I_{SS} is the set of configurations at the intersection of f and another factor (if any) of the expression of Jacobian determinant $\det \mathbf{J}(\mathbf{q})$. To ensure that Q_h, Q_v, I_{SS}, I_{SL} lie on a continuous branch, not split by joint limits, we first translate these points using Algorithm 21. To specify the connectivity, it is once again required to differentiate between the interior of the loop created by the branch and its exterior. To do so, we identify the horizontal turning point $\mathbf{q}_h$ with the least q_3 value. A configuration $\mathbf{q}$ is inside the loop if $\text{sign}(\text{f}(\mathbf{q})) = \text{sign}(\text{f}(\mathbf{q_h} + [0, \epsilon_q]^T))$, and outside otherwise. The check is presented in Algorithm 17. Figure S18 presents the check process in $q_2 - q_3$ slice for robot from Category V (C).

Algorithm 17. Case (iii): factor with branch type $(0, 0)[2, *]$

Input: Joint configurations $\mathbf{q}_A$ and $\mathbf{q}_B$

Output: Boolean indicating connectivity inside a closed loop

Check singularity separation:

If $\text{f}(\mathbf{q}_A) \cdot \text{f}(\mathbf{q}_B) < 0$ then

Configurations are separated by singularity

Return: false

If both points lie outside the loop:

If $\det \mathbf{J}$ does not have $(1, 0)[\geq 0, \geq 0]$ factor:

Return true

Else:

Use Algorithm 21 to obtain Q_h, Q_v

Displace $\mathbf{q}_A$ and $\mathbf{q}_B$ to fit in the shifted span

Note: Only displacements of $\pm 2\pi$ *along* q_2 *or* q_3 *are allowed.*

Same translation rule as in Algorithm 21

If $\mathbf{q}_A$ and $\mathbf{q}_B$ in q_3 span of Q_h:

Check connectivity for $(1, 0)[\geq 0, \geq 0]$ factor using Algorithm 14

If Algorithm 14 is true:

Retrieve common horizontal saddle lines

Identify I_{SS} and I_{SL} on them (if any)

If $\exists \mathbf{q}' \in I_{SS}, I_{SL}$ on each saddle line such that

$min(\mathbf{q}_A(q2), \mathbf{q}_B(q2)) \leq \mathbf{q}' \leq max(\mathbf{q}_A(q2), \mathbf{q}_B(q2))$:

Return false

Else:

Return true

Else:

Return false

Else:

Return true

Else:

Use Algorithm 21 to obtain Q_h, Q_v

Displace $\mathbf{q}_A$ and $\mathbf{q}_B$ by $\mathbf{d}_A$ and $\mathbf{d}_B$ such that

$\mathbf{q}_A$ and $\mathbf{q}_B$ lie in the q_3 span of Q_h and q_2 span of Q_v

Note: Only displacements of $\pm 2\pi$ along q_2 or q_3 are allowed.

Same translation rule as in Algorithm 21

Check displacements:

If $\mathbf{d}_A = \mathbf{d}_B = 0 \ \vee$

$(\mathbf{d}_A(q_2) = \mathbf{d}_B(q_2) = 0 \ \wedge \ \mathbf{d}_A(q_3) = \mathbf{d}_B(q_3))$:

Return: true

Else If $\mathbf{d}_A(q_2) = \mathbf{d}_B(q_2) \neq 0 \wedge$

$\mathbf{d}_A(q_3) = \mathbf{d}_B(q_3)$:

Check intermediate singularities:

$I_{SL}^{\text{mid}} \leftarrow \{\mathbf{q}_{SL} \in I_{SL} \mid$

$\min(\mathbf{q}_A(q_3), \mathbf{q}_B(q_3)) < \mathbf{q}_{SL}(q_3) <$

$\max(\mathbf{q}_A(q_3), \mathbf{q}_B(q_3))\}$

For each intermediate singularity $\mathbf{q}_i$:

Let ϵ be a small distance in direction of $\mathbf{q}_A - \mathbf{q}_i$

If $\det(\mathbf{J}(\mathbf{q}_i + [\epsilon, 0]^T)) \cdot$

$\det(\mathbf{J}(\mathbf{q}_A)) < 0$:

Return: false

Else:

Return: false

Return: true

Case (iv)

Factor f with atleast one branch of type $(1,0)[2,>0]$ will create a *meander* or *fold* while encircling the q_2 generator.

The check is presented in Algorithm 18. Figure S16 presents the check process in $q_2 - q_3$ slice for a robot from Category IV, with factor class $2(1,0)[2,2]$

Algorithm 18. Case (iv): factor with a branch of type $(1,0)[2,>0]$

Input: Query configurations $\mathbf{q}_A$, $\mathbf{q}_B$

Check singularity separation:

If $\mathrm{f}(\mathbf{q}_A) \cdot \mathrm{f}(\mathbf{q}_B) < 0$ then

Configurations are separated by singularity

Return: false

Fold information retrieval:

Obtain (Q_h, Q_v) translated from Algorithm 20

Separate individual (1, 0) branches by separating the Q_v related to consecutive pairs of Q_h

Collect the bounding boxes corresponding to each branch (refer to section *Robots from Category IV*)

Reverse the branch closure.

Create a chain of connected bounding boxes

Collect series of connected bounding boxes by investigating common Q_h, Q_v or I_{SL}

A series $\mathcal{B}_{Si}$ is a union of multiple bounding boxes such that a point lying in any bounding box $\mathcal{B}_i \in \mathcal{B}_{Si}$ can reach any other point lying in $\mathcal{B}_j \in \mathcal{B}_{Si}$

Create a collection of union of series

Create two collections, $\mathcal{P}_{S_p}$ and $\mathcal{N}_{S_n}$

$\{Q_{\text{boundary}}\}$: Starting counter clockwise from (π, π), arrange the I_{SL} and I_{LL} points along the outer boundary of the joint space

if $\det(\mathbf{J}) > 0$ at (π, π), start with $\mathcal{P}_{S_1}$, otherwise with $\mathcal{N}_{S_1}$

For each $\mathbf{q}_n \in \{Q_{\text{boundary}}\}$:

make a bounding box, $\mathcal{B}_{new}$, with previous I_{SL} or I_{LL}

add $\mathcal{B}_{new}$ to $(\mathcal{P}_{S_p}\mathcal{N}_{S_n})$

If $\mathbf{q}_n \in I_{SL}$:

Identify the series $\mathcal{B}_{Si}|\mathbf{q}_n \in \mathcal{B}_i \in \mathcal{B}_{Si}$

Add $\mathcal{B}_{Si}$ to the collection $(\mathcal{P}_{S_p}/\mathcal{N}_{S_n})$

switch the collection $(\mathcal{P}_{S_p}/\mathcal{N}_{S_n} \rightarrow \mathcal{N}_{S_n}/\mathcal{P}_{S_p})$

$p \leftarrow p + 1 \quad (or\ n \leftarrow n + 1)$

Else: continue

Group the collections to form boundary of each distinct region

Make groups $\mathcal{G}_i, i > 0$ that is a union of $\mathcal{P}_{S_p} \forall p$

such that there exists a common bounding box $\mathcal{B}_k \in \mathcal{P}_{S_i} \bigcap \mathcal{P}_{S_j}$ where $i \neq j$

make similar groups for $\mathcal{N}_{S_j}$

Associate a query point with a bounding box

Project a query point $\mathbf{q}_A$ vertically and choose the closest point, $\mathbf{q}_{As}$ on the singularity ($\mathbf{q}_{As} \in \mathcal{S}$).

Project a query point $\mathbf{q}_B$ vertically and choose the closest point, $\mathbf{q}_{Bs}$ on the singularity ($\mathbf{q}_{Bs} \in \mathcal{S}$).

Identify the bounding box $\mathcal{B}_{Ai}$ such that ($\mathbf{q}_{As} \in \mathcal{B}_{Ai}$).

Identify the bounding box $\mathcal{B}_{Bi}$ such that ($\mathbf{q}_{Bs} \in \mathcal{B}_{Bi}$).

Decide on connectivity

Identify the series $\mathcal{B}_{SA}, \mathcal{B}_{SB}$ related to $\mathcal{B}_{Ai}$ and $\mathcal{B}_{Bi}$

if $\det(\mathbf{J}(\mathbf{q}_A)) > 0$:

identify $\mathcal{P}_{S_A}|\mathcal{B}_{SA} \in \mathcal{P}_{S_A}$ and $\mathcal{P}_{S_B}|\mathcal{B}_{SB} \in \mathcal{P}_{S_B}$

if $\exists \mathcal{G}_i | \mathcal{P}_{S_A} \in \mathcal{G}_i \wedge \mathcal{P}_{S_B} \in \mathcal{G}_i$:

Return True

else if $\det(\mathbf{J}(\mathbf{q}_A)) < 0$:

identify $\mathcal{N}_{S_A}|\mathcal{B}_{SA} \in \mathcal{N}_{S_A}$ and $\mathcal{N}_{S_B}|\mathcal{B}_{SB} \in \mathcal{N}_{S_B}$

if $\exists \mathcal{G}_i | \mathcal{N}_{S_A} \in \mathcal{G}_i \wedge \mathcal{N}_{S_B} \in \mathcal{G}_i$:

Return True

else:

Return False

Case (v)

Factor f with branch type $(1,1)[0,*]$, where each branch encircles both the q_2 and q_3 generators. The check is presented in Algorithm 19. Figure S19 presents the check process in $q_2 - q_3$ slice for a robot from Category II (E), with factor class $(1,1)[0,2] + (1,1)[0,0]$.

Algorithm 19. Case (V): factor with a branch of type $(1,1)$

Input: Configurations $\mathbf{q}_A$, $\mathbf{q}_B$

Output: Boolean indicating connectivity between the two points

$\mathbf{t}_{SL} \leftarrow$ normal at configuration $\mathbf{q} \in I_{SL}$

$I_{SL} \leftarrow I_{SL} \cup I_{LL}$

If $\mathbf{t}_{SL}(q_2) > 0$:

Arrange I_{SL} in clockwise order from $(\pi, -\pi)$

Else:

arrange from $(-\pi, -\pi)$

$I_{SL} \leftarrow I_{SL} \setminus I_{LL}$

$I_{SL}^{-} \leftarrow$ counterclockwise ordering of I_{SL}

Define aspect bounding:

$A_i \leftarrow \{I_{SL_i}, I_{SL_{i+1}}, I_{SL_i}^{-}, I_{SL_{i+1}}^{-}\}$

Identify aspect of $\mathbf{q}_A$:

$Q_{CH_A} \leftarrow \{\mathbf{q} \mid \det \mathbf{J}(\mathbf{q}_A(q_3)) = 0\} \cup \{(\overline{q_2}, \mathbf{q}_A(q_3)), (\underline{q_2}, \mathbf{q}_A(q_3))\}$

$Q_{CV_A} \leftarrow \{\mathbf{q} \mid \det \mathbf{J}(\mathbf{q}_A(q_2)) = 0\} \cup \{(\mathbf{q}_A(q_2), \overline{q_3}), (\mathbf{q}_A(q_2), \underline{q_3})\}$

$Q_{N_A} \leftarrow$ immediate neighbours of $\mathbf{q}_A$ in $Q_{CH_A} \cup Q_{CV_A}$

Expand search until joint limits are hit:

While no $\mathbf{q} \in Q_{N_A}$ lies on joint limits:

For each $\mathbf{q}_i \in Q_{N_A}$:

$Q_{CH_i} \leftarrow \{\mathbf{q} \mid \det \mathbf{J}(\mathbf{q}_i(q_3)) = 0\} \cup \{(\overline{q_2}, \mathbf{q}_i(q_3)), (\underline{q_2}, \mathbf{q}_i(q_3))\}$

$Q_{CV_i} \leftarrow \{\mathbf{q} \mid \det \mathbf{J}(\mathbf{q}_i(q_2)) = 0\} \cup \{(\mathbf{q}_i(q_2), \overline{q_3}), (\mathbf{q}_i(q_2), \underline{q_3})\}$

Add neighbors of $\mathbf{q}_i$ in $Q_{CH_i} \cup Q_{CV_i}$ to Q_{N_A}

Determine aspect of $\mathbf{q}_A$ as $\mathcal{A}_A$ by checking which A_i it lies in

Repeat Steps 7–9 for $\mathbf{q}_B$ to obtain $\mathcal{A}_B$

Compare aspects:

If $\mathcal{A}_A = \mathcal{A}_B$:

Return: true

Else:

Return: false

Branch Closure algorithms

For robots from Categories 4 through 6, it is possible that the branch(es) associated with the factor(s) may not form a continuous curve in the $q_2 - q_3$ slice bounded by the joint limits. In such cases, the horizontal or vertical turning points no longer lie on a continuous branch in the $q_2 - q_3$ slice. As a result, when moving along the constraint boundary between two turning points, joint-limits may be occur. This makes specifying a generalised behaviour near constraints challenging. To avoid this, we first translate the Q_h, Q_v, I_{SL}, I_{SS} configurations in the $q_2 - q_3$ slice such that no joint limit boundary is encountered between turning points. We refer to this translation process as *branch closure*. Next, the track cycle is determined. For determining the joint velocity, the Q_h, Q_v, I_{SL}, I_{SS} are translated back to the original slice bounded by joint limits.

We present the method for performing branch closure for a factor with a branch of type $(1, 0)[2, > 0]$ (Connectivity for Case (iv)) in Algorithm 20, and the method for factors with a branch of type $(0, 0)[]$ branch is presented in Algorithm 21

For factor with a branch type $(1, 0)[2, > 0]$

Algorithm 20. Branch Closure for factors with branch types $(1, 0)[*, > 0]$

Input: Q^h, Q^v, expression of $\det \mathbf{J}$, I_{SL}

Output: Ordered turning points maintaining continuity in 2π interval

Check vertical split and adjust turning points if needed:

If $\exists\, \mathbf{q}_{\mathrm{SL}} \in I_{\mathrm{SL}} \mid |\mathbf{q}_{\mathrm{SL}}(q_3)| = \pi$:

$\mathbf{q}_{sh} \leftarrow \arg\min_{\mathbf{q}_i \in Q^h} \mathbf{q}(q_3)$

$Q_h \leftarrow \{\mathbf{q}_i + (0, 2\pi) \mid \mathbf{q}_i \in Q^h, \mathbf{q}_i(q_3) \le \mathbf{q}_{sh}(q_3)\}$

$Q_v \leftarrow \{\mathbf{q}_i + (0, 2\pi) \mid \mathbf{q}_i \in Q^v, \mathbf{q}_i(q_3) \le \mathbf{q}_{sh}(q_3)\}$

$I_{\mathrm{SL}} \leftarrow \{\mathbf{q}_i + (0, 2\pi) \mid \mathbf{q}_i \in I_{\mathrm{SL}}, \mathbf{q}_i(q_3) \le \mathbf{q}_{sh}(q_3)\}$

Group turning points by branch:

$Q_h^+ \leftarrow \{\mathbf{q} \in Q_h \mid \det \mathbf{J}(\mathbf{q} + (0, \epsilon)) > 0\}$

$Q_h^- \leftarrow Q_h \setminus Q_h^+$

$Q_v^+ \leftarrow \{\mathbf{q} \in Q_v \mid \mathbf{q}(q_3) \in \mathrm{span}(Q_h^+(q_3))\}$

$Q_v^- \leftarrow Q_v \setminus Q_v^+$

$I_{\mathrm{SL}}^+ \leftarrow \{\mathbf{q} \in I_{\mathrm{SL}} \mid \mathbf{q}(q_3) \in \mathrm{span}(Q_h^+(q_3))\}$

$I_{\mathrm{SL}}^- \leftarrow \{\mathbf{q} \in I_{\mathrm{SL}} \mid \mathbf{q}(q_3) \in \mathrm{span}(Q_h^-(q_3))\}$

Check and adjust horizontal split:

For each $m \in \{+, -\}$:

If $|\{\mathbf{q}_{\mathrm{SL}} \in I_{\mathrm{SL}}^m \mid |\mathbf{q}_{\mathrm{SL}}(q_2)| = \pi\}| > 2$:

$\mathbf{q}_{sv} \leftarrow \arg\max_{\mathbf{q}_i \in Q_v^m} \mathbf{q}(q_2)$

$Q_h^m \leftarrow \{\mathbf{q} + (2\pi, 0) \text{ if } \mathbf{q}(q_2) < \mathbf{q}_{sv}(q_2), \text{ else } \mathbf{q} \mid \mathbf{q} \in Q_h^m\}$

$Q_v^m \leftarrow \{\mathbf{q} + (2\pi, 0) \text{ if } \mathbf{q}(q_2) < \mathbf{q}_{sv}(q_2), \text{ else } \mathbf{q} \mid \mathbf{q} \in Q_v^m\}$

Return: $(Q_h^+, Q_v^+, I_{\mathrm{SL}}^+), (Q_h^-, Q_v^-, I_{\mathrm{SL}}^-)$

For factor with a branch type $(0, 0)[2, > 0]$

Algorithm 21. Loop Closure for factors with branch types $(0, 0)[2, > 0]$

Input: Q^h, Q^v, factor f_i of $\det \mathbf{J}$, I_{SL}, I_{SS}

Output: Ordered saddle points maintaining continuity over 2π interval

Check vertical split and adjust saddle points if needed:

If $\exists \mathbf{q}_{\mathrm{SL}} \in I_{\mathrm{SL}}$ s.t. $|\mathbf{q}_{\mathrm{SL}}(q_3)| = \pi$:

$\mathbf{q}_{sh} \leftarrow \arg\min_{\mathbf{q}_i \in Q^h} \mathbf{q}(q_3)$

$Q_h \leftarrow \{\mathbf{q}_i + (0, 2\pi) \mid \mathbf{q}_i \in Q^h, \mathbf{q}_i(q_3) \leq \mathbf{q}_{sh}(q_3)\}$

$Q_v \leftarrow \{\mathbf{q}_i + (0, 2\pi) \mid \mathbf{q}_i \in Q^v, \mathbf{q}_i(q_3) \leq \mathbf{q}_{sh}(q_3)\}$

$I_{\mathrm{SL}} \leftarrow \{\mathbf{q}_i + (0, 2\pi) \mid \mathbf{q}_i \in I_{\mathrm{SL}}, \mathbf{q}_i(q_3) \leq \mathbf{q}_{sh}(q_3)\}$

$I_{\mathrm{SS}} \leftarrow \{\mathbf{q}_i + (0, 2\pi) \mid \mathbf{q}_i \in I_{\mathrm{SS}}, \mathbf{q}_i(q_3) \leq \mathbf{q}_{sh}(q_3)\}$

$Q_v^{\mathrm{sorted}} \leftarrow$ sort Q_v in ascending order of q_2

Identify the signature of $\det \mathbf{J}$ **inside the loop**

$\mathbf{q}_{H_t} = \arg\max_{\mathbf{q} \in Q_h} \mathbf{q}_3$

$j_{\mathrm{loop}} = sign(\det \mathbf{J}(\mathbf{q}_{H_t} - [0, \epsilon]))$

Check whether the loop wraps around the q_2**-generator:**

$Q_R \leftarrow \varnothing$

For $i \in \{1, 2, \ldots, |Q_v^{\mathrm{sorted}}| - 1\}$:

$\bar{q}_2^{(i)} \leftarrow \dfrac{\mathbf{q}_i(q_2) + \mathbf{q}_{i+1}(q_2)}{2}$

$N_i \leftarrow \#\{q_3 \in \mathbb{R} : \det \mathbf{J}(q_2 = \bar{q}_2^{(i)}) = 0\}$

If $N_i = 0$:

$Q_R \leftarrow \varnothing$

Break

Else:

let $q_3^{(1)}, q_3^{(2)}$ be the two real roots of $\det \mathbf{J}(q_2 = \bar{q}_2^{(i)}) = 0$

If $(q_3^{(1)} - \mathbf{q}_i(q_3))(q_3^{(2)} - \mathbf{q}_i(q_3)) > 0$:

$Q_R \leftarrow Q_R \cup \{\mathbf{q}_i\}$

$Q_R^{\mathrm{left}} = \{\mathbf{q} \in Q_R \mid sign(\det \mathbf{J}(\mathbf{p} - [\epsilon, 0]) = j_{\mathrm{loop}}\}$

$Q_R^{\mathrm{right}} = \{\mathbf{q} \in Q_R \mid sign(\det \mathbf{J}(\mathbf{p} + [\epsilon, 0]) = j_{\mathrm{loop}}\}$

Retain extremes of the loop

$Q_R^{\mathrm{ext}} = \{\mathbf{q} \in Q_R^{\mathrm{left}} \mid \mathbf{q}(q_2) = \max_{\mathbf{p} \in Q_R^{\mathrm{left}}} \mathbf{p}(q_2)\} \cup \{\mathbf{q} \in Q_R^{\mathrm{right}} \mid \mathbf{q}(q_2) = \min_{\mathbf{p} \in Q_R^{\mathrm{right}}} \mathbf{p}(q_2)\}$

Loop: For i in $\{1, 2, \ldots, |Q_v^{\mathrm{sorted}}| - 1\}$

If $f_i(\mathbf{q}_i) \cdot f_i(\mathbf{q}_{i+1}) < 0$:

If $\mathbf{q}_i \in Q_R^{\mathrm{ext}}$:

selection criteria: $\mathbf{q}(q_2) \leq \mathbf{q}_{i+1}(q_2)$ except $\mathbf{q}_i$

Else:

selection criteria: $\mathbf{q}(q_2) \leq \mathbf{q}_i(q_2)$

$Q_v^{\text{sorted}} \leftarrow \{\mathbf{q} + (2\pi, 0)$ if $\mathbf{q}$ satisfies selection criteria$\}$

$Q_h \leftarrow \{\mathbf{q} + (2\pi, 0)$ if $\mathbf{q}$ satisfies selection criteria$\}$

$I_{\text{SS}} \leftarrow \{\mathbf{q} + (2\pi, 0)$ if $\mathbf{q}$ satisfies selection criteria$\}$

$I_{\text{SL}} \leftarrow \{\mathbf{q} + (2\pi, 0)$ if $\mathbf{q}$ satisfies selection criteria$\}$

If $\exists \mathbf{q}_h \in Q^h$ s.t. $\mathbf{q}_h \notin \text{span}(Q_v^{\text{sorted}}(q_2))$:

$i \leftarrow 0$ (reset loop)

Renumber the shifted Q_v in ascending order of q_2

Else:

Return: $(Q_h, Q_v^{\text{sorted}}, I_{\text{SL}}, I_{\text{SS}})$

Return: $(Q_h, Q_v^{\text{sorted}}, I_{\text{SL}}, I_{\text{SS}})$

Incremental update by Workspace-to-Joint Space Transfer

In Section 12 we discussed the modelling of the demonstrated behaviour in an aspect of the joint space, using multiple demonstrations. The same procedure can also be used for modelling the workspace behaviour by leveraging multiple workspace demonstrations $\{\mathbf{X}\}_{i=1}^{W}$. The workspace contains a single goal position $\overset{*}{\mathbf{x}}$, and the workspace behaviour $\mathrm{B}^{(\mathcal{X})}$ can be modelled as $\mathrm{B}^{(\mathcal{X})} = \{P, \Lambda, \{\mathrm{g}_i^{(\mathcal{X})} \leftrightarrow (\psi_i, \mathbf{k}_i)\}_{i=1}^{W}\}$. The workspace behaviour also gets updated when novel workspace demonstrations are provided by the user (Section 12).

Using the available demonstration data, we have modelled the workspace behaviour $\mathrm{B}^{(\mathcal{X})}$ and the preliminary joint space $\mathrm{B}^{(\mathcal{Q})}$. The preliminary $\mathrm{B}^{(\mathcal{Q})}$ accurately replicates the demonstrated trajectories while handling perturbations and adhering to the robot's kinematic constraints. As a result, they provide a robust model of the behaviour across the joint space. However, to more effectively incorporate the embedded knowledge from user-demonstrated workspace behaviour, we introduce a generalization step.

Generalization is achieved through an iterative process that identifies joint-space trajectories contributing the most novel information. A trajectory is considered to improve generalization if a significant portion of its configurations lie outside the distribution of previously recorded demonstrations. In this section, we go over the steps to generalise the behaviour in joint space $\mathcal{Q}$

1. First, the joint space joint space $\mathcal{Q}$ is sampled to generate candidate configurations.

2. Next, from their workspace counterparts, *transfer* trajectories in workspace $\mathcal{X}$ are generated using workspace behavior $\mathrm{B}^{(\mathcal{X})}$. These trajectories are then inverted back into joint space.

3. A filtering step follows to detect and trim the *incorrect* transfer trajectories in each aspect of joint space $\mathcal{Q}$.

4. Identify the most novel transfer trajectory in each aspect and update $\mathrm{B}^{(\mathcal{Q})}$.

This step-by-step approach ensures a robust and accurate update of $\mathrm{B}^{(\mathcal{Q})}$, while maintaining that after each update the modelled behaviour remains stable, respects kinematic constraints, and is robust to perturbations.

Generating transfer trajectories

A uniform grid of points is generated in joint space $\mathcal{Q}$, ensuring even coverage across the range of motion for each joint. This grid provides a comprehensive set of candidate joint configurations for subsequent

evaluation and analysis. Let the ranges of the two joints be $[\underline{q_1}, \overline{q_1}]$ and $[\underline{q_2}, \overline{q_2}]$, each discretized into m_{q_1} and m_{q_1} points, respectively. The set of sampled joint configurations is defined as

$$\{\mathbf{Q}\}^{\text{sample}} = \{(q_1, q_2) \mid q_1 \in Q_1,\ q_2 \in Q_2\},$$

where

$$Q_1 = \{\underline{q_1} + \epsilon_q + i\Delta q_1\}_{i=0}^{m_{q_1}}, \quad \Delta q_1 = \frac{\overline{q_1} - \underline{q_1} - 2\epsilon_q}{n_1 - 1}$$
$$Q_2 = \{\underline{q_2} + \epsilon_q + i\Delta q_2\}_{i=0}^{m_{q_2}}, \quad \Delta q_2 = \frac{\overline{q_2} - \underline{q_2} - 2\epsilon_q}{n_2 - 1}$$

where $\epsilon_q > 0$ is user-specified safety margin (in radians) around the joint limits. $\{\mathbf{Q}\}^{\text{sample}}$ is next mapped to workspace $\mathcal{X}$ to yield a set of candidate end effectir positions

$$\{\mathbf{x}\}^{\text{sample}} = \{\mathbf{x}_i = \text{FK}(\mathbf{q}_i \in \{\mathbf{Q}\}^{\text{sample}})\}_{i=1}^{|\{\mathbf{Q}\}^{\text{sample}}|}$$

Since the FK is a many-to-one mapping, there exist some repeated entries in $\{\mathbf{x}\}^{\text{sample}}$ which are eliminated. The candidate end effectir positions are used to generate a set of *transfer* trajectories $\{\mathbf{X}\}^{\text{transfer}}$ where each trajectory

$$\mathbf{X}_i^{\text{transfer}} = \{\mathbf{x}_t \mid \mathbf{x}_{t=0} = \mathbf{x}_i,\ t \in [0, \infty),\ \lim_{t\to\infty} \mathbf{x}_t = \overset{*}{\mathbf{x}}\}$$

is generated in workspace $\mathcal{X}$ by the relevant DS embedding $\text{g}^{(\mathcal{X})}$, starting from the initial state $\mathbf{x}_i$. The relevant DS embedding is determined from the workspace behaviour $\text{B}^{(\mathcal{X})} = \{P, \Lambda, \{\text{g}_i^{(\mathcal{X})} \leftrightarrow (\psi_i, \mathbf{k}_i)\}\}$ similar to Equation S22, but for workspace DS embeddings.

These trajectories can now be inverted back into joint space $\mathcal{Q}$, generating a set of joint trajectories $\{\mathbf{Q}\}^{\text{transfer}}$ which encapsulate additional information about the user behaviour in workspace $\mathcal{X}$.

Some of the transfer trajectories in $\{\mathbf{X}\}^{\text{transfer}}$ may be infeasible (refer to Section). Thus, we perform the validation and trimming procedure (refer to Section 12) for each transfer trajectory in $\{\mathbf{Q}\}^{\text{transfer}}$.

Measuring trajectory novelty

Each new joint space trajectory in $\{\mathbf{Q}\}^{\text{transfer}}$ encapsulates some additional information that can contribute towards updating the overall behaviour $\text{B}^{(\mathcal{Q})}$. However, instead of learning the embeddings for all of the trajectories, we selectively prioritise learning those embeddings which encapsulate the most additional information. If a transfer trajectory has a significant inclusion of joint configurations that exhibit little overlap with the already learnt behaviour in $\text{B}^{(\mathcal{Q})}$, this transfer trajectory embeds more novel information as compared with a trajectory that shows significant overlap with $\text{B}^{(\mathcal{Q})}$.

Selectively prioritising the most novel demonstrations enables us to transfer behaviour from workspace $\mathcal{X}$ to joint space $\mathcal{Q}$, while minimising the embeddings to be learnt. Doing so keeps the model of $\mathrm{B}^{(\mathcal{Q})}$ parameter-efficient. The *novelty* metric for prioritising trajectories is based on their overlap with the known partitions in each aspect of joint space $\mathcal{Q}$. Lower overlap makes for a trajectory that encapsulates most additional behaviour for transfer.

To do so, first the aspect of a trajectory in $\{\mathbf{Q}\}^{\text{transfer}}$ is determined $\mathcal{A}_i = \mathcal{A}(\mathbf{Q}_i^{\text{transfer}})$. The aspect-specific behaviour $\mathrm{B}^{(\mathcal{A}_i)}$ from the learnt joint behaviour $\mathrm{B}^{(\mathcal{Q})}$ is queried. Next, from $\mathrm{B}^{(\mathcal{A}_i)} = \{P_i, \Lambda_i, \{\mathrm{g}^{(\mathcal{Q})} \leftrightarrow (\psi, \mathbf{k})\}_i\}$, the relevant DS embeddings $\{\mathrm{g}^{(\mathcal{Q})} \leftrightarrow (\psi, \mathbf{k})\}_i$ are retrieved to calculate the projection angles (refer Equation S23) $\{\Theta\}_i$ subtended by the transfer trajectory in the latent space of the known DS embeddings

$$\begin{aligned}
\Theta_j(\mathbf{Q}^{\text{transfer}}) &= \{\angle(\psi_j^{-1}(\mathbf{q}_t) - \mathbf{1}_n^T, \mathbf{1}_n^T - \mathbf{0}_n^T)\}_{t=1}^{|\mathbf{Q}^{\text{transfer}}|} \\
\{\Theta\}_i &= \{\Theta_j \forall\ \psi_j\ \in \{\mathrm{g}^{(\mathcal{Q})} \leftrightarrow (\psi, \mathbf{k})\}_i\} \\
\text{novelty}(\mathbf{Q}^{\text{transfer}}) &= \min_{1 \le j \le |\,\mathrm{B}^{(\mathcal{A}_i)}\,|} \left\{\{\Theta\}_j\right\}
\end{aligned} \tag{S32}$$

where $\mathbf{q}_t \in \mathbf{Q}^{\text{transfer}}$ is a joint configuration in the transfer trajectory. $\Theta_j(\mathbf{Q}^{\text{transfer}})$ contains the angles subtended by the latent embedding of transfer trajectory points generated using ψ_j. These angles correspond to the angular deviations between the latent embedding of the demonstration originally used to learn ψ_j, and the embeddings of points in the transfer trajectory ($\psi_j^{-1}(\mathbf{Q}^{\text{transfer}})$), computed in the latent space $\mathcal{U}$. $\{\Theta\}_i$ contains these angular deviations between the transfer trajectory $\mathbf{Q}^{\text{transfer}}$, and the original demonstrations associated with the DS embeddings of the identified aspect $\mathrm{B}^{(\mathcal{A}_i)}$. The $\text{novelty}(\mathbf{Q}^{\text{transfer}}) \in \mathbb{R}^+$ corresponds to the least angular deviation of between the transfer trajectory and all of the recorded demonstrations. The most novel transfer trajectory from amongst $\{\mathbf{Q}\}^{\text{transfer}}$ aims to have least overlap with existing demonstrations i.e. the maximum novelty value

$$\mathbf{Q}_{\text{novel}}^{\text{transfer}} = \text{argmax}\left\{\text{novelty}(\mathbf{Q}_i^{\text{transfer}})\ \forall\ \mathbf{Q}_i^{\text{transfer}} \in \{\mathbf{Q}\}^{\text{transfer}}\right\} \tag{S33}$$

The most novel trajectory in each aspect is used to update the respective aspect-specific behaviour $\mathrm{B}^{(\mathcal{A}_i)}$, thereby updating the control policy $\mathrm{B}^{(\mathcal{Q})}$

Issues with numerical path-planning strategies

Null-space-based numerical strategies are inherently limited to redundant manipulators. When the Jacobian has full row rank and the number of joints matches the task dimension (as in non-redundant 6R arms), the null space is trivial and joint velocities reduce to $\dot{\mathbf{q}} = \mathbf{J}^{-1}\dot{\mathbf{x}}$, leaving no degree of freedom to encode posture optimisation, joint-limit avoidance, or aspect selection. As a result, any skill-transfer scheme that relies on shaping behaviour via null-space motions can only be applied to redundant robots, and cannot endow non-redundant robots with the same level of constraint-aware adaptability.

Even for redundant robots, numerical null-space controllers suffer from fundamental robustness and certification issues. A typical strategy computes

$$\dot{\mathbf{q}} = \mathbf{J}^{\#}\dot{\mathbf{x}} + \mathbf{N}(\mathbf{q})\,\mathbf{z},$$

where $\mathbf{J}^{\#}$ is a (damped) pseudoinverse, $\mathbf{N}$ the null-space projector, and $\mathbf{z}$ encodes secondary objectives (e.g., joint-limit avoidance, manipulability maximisation). Because the controller reasons only locally and never explicitly encodes aspects, singularity curves, or voids in the workspace, small changes in initialization, gains, or integration step can lead to qualitatively different joint-space behaviours for the same demonstrated path.

These effects are illustrated in Figure S20. In Figure S20(A), a particular initialization and parameter choice yield a seemingly ideal outcome: the redundant angle q_R remains nearly constant, the end-effector follows the demonstrated path, and no constraints are visibly approached. In Figure S20(B), for the same robot and task, but with different initial IKS, the path is not completed, and the null-space evolution induces oscillations and shaking at the end of the trajectory, revealing sensitivity to the tuning of secondary objectives. In Figure S20(C), a different evolution of q_R steers the posture so that the robot tracks the workspace path while remaining close to singular manifolds along most of the motion; the task is accomplished, but the trajectory demonstrates erratic behaviour near constraints. Finally, in Figure S20(D), the null-space policy drives the arm into a configuration where two joint limits are reached, and the end-effector trajectory deviates from the demonstrated behaviour, resulting in an explicit failure.

More broadly, numerical planners operate on discretised waypoints and iterative optimisation or sampling between them. Even if all sampled configurations satisfy limits and avoid singularities, this only approximates the continuous path; detecting crossings of singularities, entries into IK holes, or aspect changes would require very dense sampling or additional symbolic checks that these methods do not natively provide. Consequently, they cannot distinguish genuine kinematic infeasibility (no start–goal con-

nection exists in any aspect or in any redundant posture) from failures caused purely by local optimisation artefacts (an unfortunate null-space direction, an incorrect IK branch choice, or initialization). This motivates our certified execution layer, which explicitly factorises $\det \mathbf{J}$, computes aspects and their boundaries, and performs connectivity checks in reduced joint-space slices, providing deterministic, aspect-aware decisions that are not subject to the numerical fragility illustrated in Fig. S20.

Computational performance

Benchmarking was performed on a PC equipped with an Intel i9-10900K CPU (3.70 GHz). To evaluate the categorisation procedure in Figure 6(A2), we generated the factorised expressions of $\det J$ for 21,787 distinct 3R robots.

In the preprocessing for our kinematic benchmarking, we discretize the design parameter space of the $3R$ manipulator and, for each candidate, first test cuspidality using the known necessary-and-sufficient condition; only noncuspidal robots are retained. For these, we start from the closed-form expression of the Jacobian determinant, and symbolically factorise $\det \mathbf{J}$ after simplifying the equation by substituting zero link lengths/offsets and any joint angles fixed at 0 or $\pi/2$ radians. We then compute horizontal and vertical turning points, as well as intersections of each factor with the boundary lines $q_{2,3} = \pm\pi$, using both half-angle tangent and cotangent substitutions to regularise the behaviour at 0 and $\pm\pi$.

For each robot, we assessed connectivity (C1) and constraint-violation (C2) checks using 100 joint-space configurations. These configurations were uniformly sampled on the q_2–q_3 slice with a fixed value for the first joint. To benchmark connectivity, conditioned on the robot's category, we computed the connectivity between each of the 100 sampled configurations and the reference configuration $[0, 0.5, 0.1]$. The average time required to determine connectivity across these 100 pairs was recorded.

For the near-constraint strategy (C4), we identified points lying close to violation boundaries within a common aspect and generated the corresponding near-boundary traversal strategy that respects all kinematic constraints. The time required to compute the control velocities for these near-boundary cases (C5) was also measured. These metrics together provide the average runtime per robot. The entire procedure was executed for all 22,000 robots, and the aggregated runtimes are reported in Figure 6.

To evaluate learning and replication of human-demonstrated behaviours, we used seven workspace demonstrations from the "Multi Models 1" subset of the LASA Handwriting Dataset (*38*). These demonstrations exhibit three sub-dynamics in the workspace and converge to a common attractor.

Figure 6C reports the time required to learn the aspect-specific behaviours for each robot; these tests were performed for approximately 500 robots. For each robot, we used the workspace demonstrations' starting points (mapped to joint space) to measure the time needed to (B) plan and identify the relevant joint primitive for replication and (C6) compute the corresponding joint velocities. We additionally evaluated the time taken to perform all-at-once sampling (C3) using the same starting points.

The average time to learn the model is presented in Figure 6(D1). Additionally, we computed the

DTW score (using FastDTW (*49*)) between each demonstration's joint-space trajectory and its replication generated by the learnt joint-space control policy and have reported in (D2).

Robustness

Our framework separates *structural* kinematic robustness (how the singularity layout and aspect structure respond to modelling error) from *execution*-level robustness (how the closed-loop controller responds to noise, friction, and compliance).

Structural robustness to kinematic calibration errors. For a *generic* manipulator, small perturbations of the D–H parameters do not change the local homotopy type of the singularity set: the number of factors of $\det \mathbf{J}$, their winding behaviour, and the induced aspect structure are preserved in a neighbourhood of the nominal design. Only larger perturbations that move the mechanism across the discriminant set in parameter space (e.g., from a generic to a non-generic architecture, or from noncuspidal to cuspidal) can change this topological structure. In our setting, we target industrial and collaborative robots whose calibration errors are typically a few percent and remain within the same class, so that the classification and connectivity results (robot category, factor types, aspects) remain valid under realistic parameter uncertainty.

Figure S21 illustrates this structural robustness on a representative 3R robot. The blue curve shows the nominal singularity, the red band corresponds to a safety margin, and the black curves show the singularities obtained by independently adding 5% noise to each D–H parameter that affects $\det \mathbf{J}$ (d_3, a_1, a_2, a_3, α_1, and α_2). In all cases, the perturbed singularities remain inside the red band and the qualitative structure in the region of interest is unchanged.

Perturbations of the D–H parameters that affect *joint orientations* (e.g., α_i) are known to be more delicate: even relatively small changes can move the mechanism from a non-generic configuration with intersecting singularity branches to a generic configuration where these branches no longer intersect, or even render a previously noncuspidal robot cuspidal. We explicitly explored such perturbations and observed that, for orientation errors up to 5%, a safety margin of order 10^{-3} on $\det \mathbf{J}$ still encloses the perturbed singularity curves in the joint-space region where execution takes place (see additional illustration in Fig. S22). Thus, even when the global singularity class changes, the local boundary used by our feasibility and connectivity checks can be conservatively padded so that the true singularity manifold remains inside the safety margin used by the controller.

Safety margins and execution-level disturbances. Execution-level uncertainties (encoder noise, unmodelled friction, moderate compliance) are handled by introducing a *safety margin* around analytically computed constraints. Instead of treating the exact singularity curve $\det \mathbf{J} = 0$ and joint-limit surfaces as hard boundaries, we define a padded near-constraint band: configurations with $|\det \mathbf{J}| < \varepsilon$ or within a user-defined distance to joint limits are treated as if they were on the constraint. The value of ε can be tuned per robot; in our examples, values on the order of 10^{-3} are sufficient to cover the observed shifts due to 5% D–H perturbations, including those on orientation parameters.

Robustness of DS-based execution and track cycles. Within each aspect, the joint-space motion is governed by a continuous, globally stable dynamical system (DS) that realises the learned task-space policy. As long as the state remains inside the padded feasible region, small perturbations due to friction, compliance, or sensor noise are naturally rejected: the DS drives the configuration back towards its nominal flow. When the configuration enters the near-constraint band (either because the nominal DS would approach a boundary or due to disturbances), the violation-check step detects this and switches to the corresponding track-cycle rule. Track cycles are tangent and category-specific: by construction, they follow the local singularity or joint-limit boundary while remaining on the feasible side of the constraint. Because the underlying singularity structure is structurally robust for generic robots and the safety margin absorbs the residual shifts (including those induced by moderate orientation errors), the track-cycle execution remains feasible and behaves consistently under realistic modelling and sensing uncertainties.

We conducted a large-scale study for $\sim$ 22,000 noncuspidal 3R designs, benchmarking classification, connectivity checks, violation checks, and track-cycle execution. All the steps in the process were completed without failures, and we provide the statistics underlying our timing table in Figure 5 of the main text. The corresponding benchmarking scripts and raw timing data are provided in the section 'Computational performance'. On all real robots considered (KUKA iiwa 7, Neura Maira M, and a 6-DoF Dyna arm with tight, non-symmetric joint limits), the factorisation, classification, and online checks succeeded without ambiguity. Each model was assigned to a unique noncuspidal category, all singular branches were recovered, and we did not observe numerical failures or inconsistent behaviour. For a generic robot, our feasibility/connectivity checks and track-cycle execution are structurally robust, since small perturbations of the D–H parameters do not alter the homotopy type of the singularity set. A user-defined safety margin around analytically computed constraints accommodates the resulting shifts in singularities

In summary, for generic robots, our feasibility and connectivity checks are structurally robust to

realistic calibration errors, and the combination of user-defined safety margins, stable DS-based execution, and boundary-aligned track cycles ensures that encoder noise, friction, moderate compliance, and small orientation perturbations are absorbed without compromising the certified behaviour.

Failure modes

To demonstrate different failure modes, we tested a robot from Category I with varying inverse kinematic solutions (IKS): four in some regions and two in others. Figure 7A (i-iv) of the main text presents the user demonstration; the path transitions from a region with four IKS to a region with only two IKS. When replicating this behaviour from different possible IKS of the starting workspace position, our method constructs aspect-specific behaviour models in joint space and proceeds with execution only when the current configuration lies in a feasible aspect (see Figure 7C (i-ii)); otherwise, the robot remains halted for safety (Figure 7C (iii)). When the demonstration transitions from a region with two IKS to a region with four IKS, only the terminal portion of the trajectory is used to learn the aspect-specific behaviour for the two newly gained feasible aspects. As a result, even when the robot is perturbed to a state where the complete path is unavailable, the control policy can still guide the robot to the goal without violating constraints, ensuring faithful and consistent reproduction of the intended motion.

In practice, certified execution exposes a set of well-defined, aspect-based failure modes. For non-redundant 6R robots, the positional subchain behaves as a standard $3R$ manipulator, and failures arise when (i) the goal is unreachable from the current aspect (no admissible path respecting joint limits and singularities exists), or (ii) perturbations drive the state towards joint limits or singularities. Before motion, we run an analytic connectivity check between the current and goal configurations; if no feasible connection exists in the current aspect, the controller refuses to move and reports the goal as unreachable. During execution, when the state enters a user-defined safety margin around constraints, the controller switches to the appropriate track cycle; if re-entry into the interior is not feasible while preserving feasibility, it halts and reports a boundary-induced failure rather than violating constraints.

The same logic extends to wrist-partitioned redundant $7R$ arms. Fixing the redundant angle q_R defines a family of $3R$ subchains, and the above aspect-based checks are applied per subchain. An additional failure mode arises when no value of q_R yields a feasible aspect connecting start and goal; in this case, the task is declared infeasible for that robot, as illustrated in Figure 5D of the main text, where some aspects at certain redundant angles are detected as infeasible, and the robot remains halted. In all cases, the system explicitly reports whether the goal is reachable from the current aspect (and, for 7R, the current q_R) or

whether execution stopped due to a boundary-induced or aspect-induced failure. The complete factor-wise feasibility framework and connectivity algorithms are detailed in the section *Certified Execution* of this supplementary text.

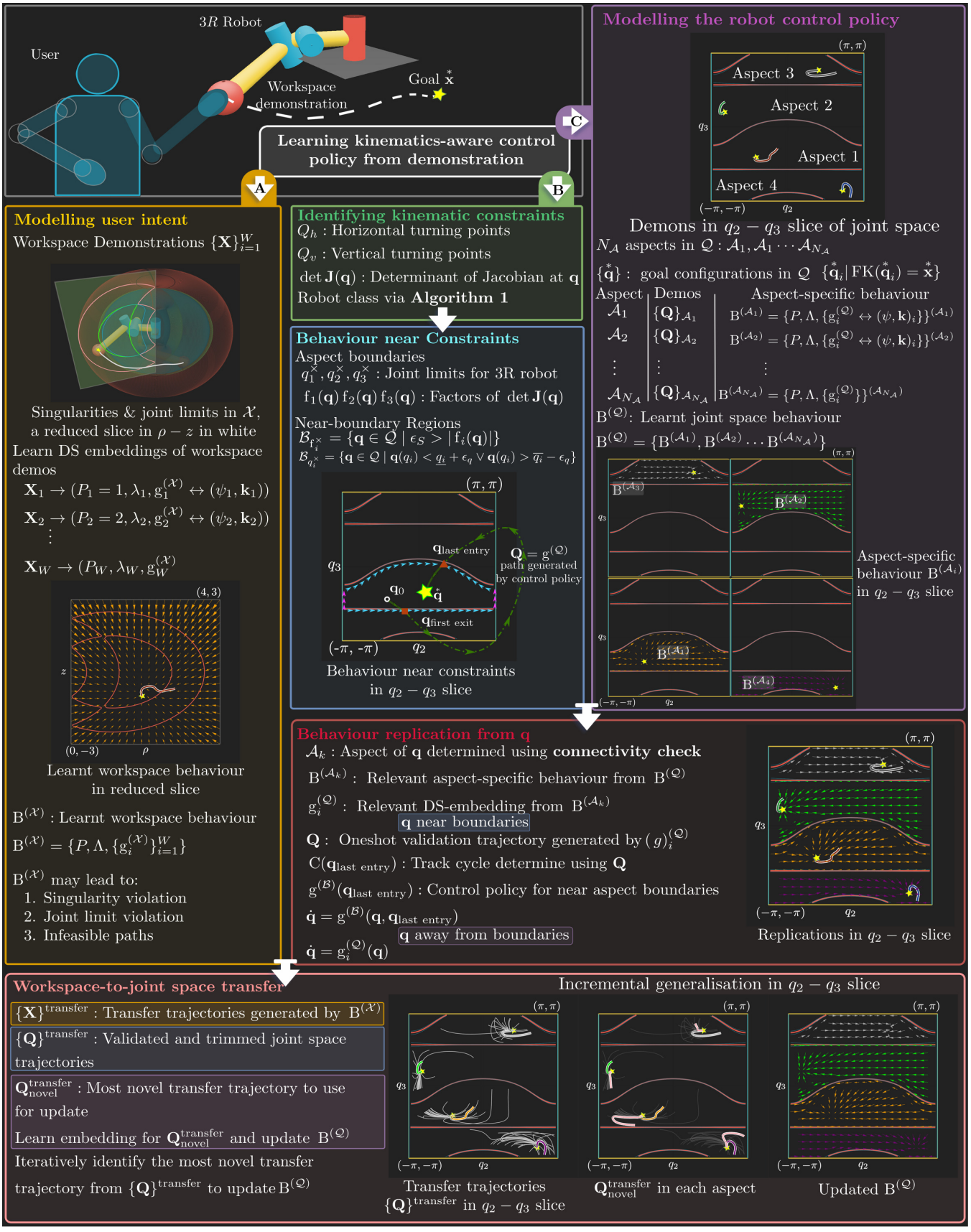


Figure S1: Overview of the proposed framework for learning kinematics-aware control policies from demonstration, and incremental generalization via workspace-to-joint space transfer. The figure illustrates both the operational flow, from demonstration to generalization, and highlights the sections corresponding to each component.

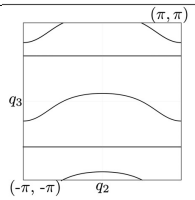
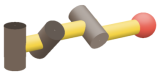
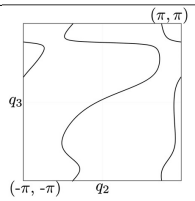
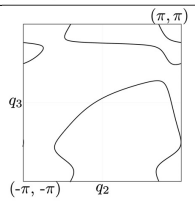
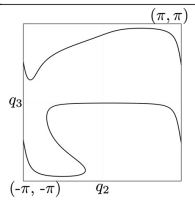
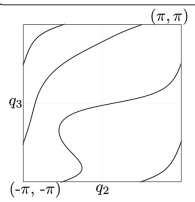
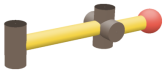
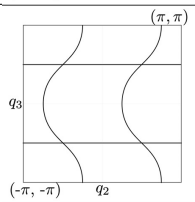
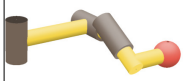
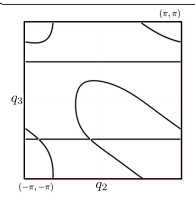
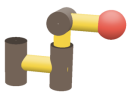
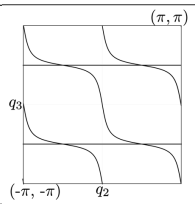
Robot parameters	Robot schematic	Joint space singularity in $q_2 - q_3$ slice	f_1		f_2		Singularity class	Robot category
			Q^h	Q^v	Q^h	Q^v		
(A) $\mathbf{d} = [0, 0.5, 0]$ $\mathbf{a} = [1, 0.5, 2]$ $\alpha = [\frac{\pi}{2}, \frac{\pi}{2}, 0]$			4	0	∞	0	$2(1,0)[4,0]$ + $2(1,0)[\infty,0]$	Category 1
(B) $\mathbf{d} = [0, 0.3, 0.3]$ $\mathbf{a} = [0.8, 1, 1.4]$ $\alpha = [\frac{\pi}{4}, \frac{7\pi}{18}, 0]$			0	6	–	–	$2(0,1)[0,6]$	Category 2
(C) $\mathbf{d} = [0, 0.5, 0.3]$ $\mathbf{a} = [1, 1, 1.4]$ $\alpha = [\frac{\pi}{4}, \frac{7\pi}{18}, 0]$			2	6	–	–	$(0,0)[2,6]$	Category 5
(D) $\mathbf{d} = [0, 0, 0.6]$ $\mathbf{a} = [1, 1, 1]$ $\alpha = [\frac{\pi}{2}, \frac{4\pi}{9}, 0]$			4	2	–	–	$2(1,0)[4,2]$	Category 4
(E) $\mathbf{d} = [0, 0, 1]$ $\mathbf{a} = [1, 1, 1]$ $\alpha = [\frac{7\pi}{18}, \frac{\pi}{2}, 0]$			0	2	–	–	$2(1,1)[0,2]$	Category 2
(F) $\mathbf{d} = [0, 0, 0.24]$ $\mathbf{a} = [1.2, 0, 0.24]$ $\alpha = [0, \frac{\pi}{2}, 0]$			0	4	∞	0	$2(0,1)[4,0]$ + $2(1,0)[\infty,0]$	Category 3
(G) $\mathbf{d} = [0, 1, 1]$ $\mathbf{a} = [1, 0, 3]$ $\alpha = [\frac{\pi}{2}, \frac{\pi}{4}, 0]$			2	2	∞	0	$(0,0)[2,2]$ + $2(1,0)[\infty,0]$	Category 6
(H) $\mathbf{d} = [0, 1, 0]$ $\mathbf{a} = [1, 0, 1]$ $\alpha = [0, \frac{4\pi}{9}, 0]$			0	0	∞	0	$2(1,1)[0,0]$ + $2(1,0)[\infty,0]$	Category 3

Figure S2: Eight $3R$ robots, along with their proposed categories. The figure presents the D-H parameters $(\mathbf{d}, \mathbf{a}, \alpha)$ of each robot in the first column; robot schematic in second column; singularity plot in the $q_2 - q_3$ slice of joint space in third column; the factors - f_1, f_2 - of the expression of $\det \mathbf{J}$ in conjunction with each factor's total horizontal and vertical turning points (Q^h, Q^v) in the fourth column; the proposed homotopy-based singularity classification in the fifth column; resultant robot category in the sixth column. All robots except in Row C are noncuspidal robots.

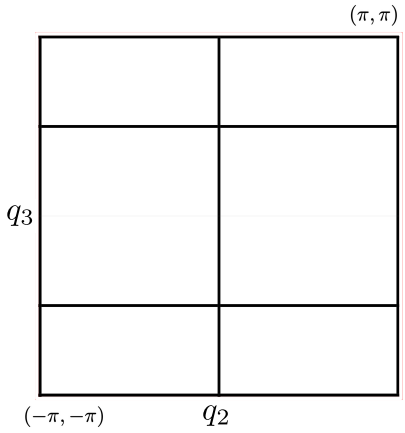


Figure S3: Robot class:
$4(1,0)[\infty, 0] + 2(0,1)[0, \infty]$,
$\mathbf{d} = [0, 0, 0]$ $\mathbf{a} = [1, 0.2, 0.2]$
$\alpha = [0, \frac{\pi}{2}, 0]$

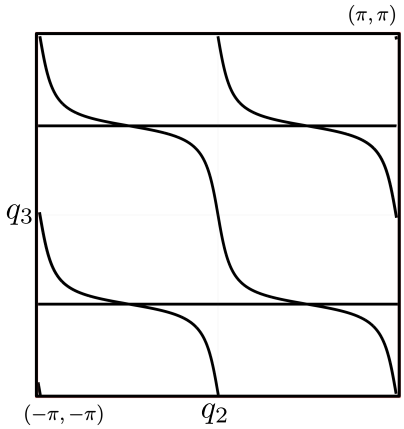


Figure S4: Robot class:
$2(1,0)[\infty, 0] + 2(1,1)[0, 0]$,
$\mathbf{d} = [0, 0.2, 0]$ $\mathbf{a} = [0, 1, 0]$
$\alpha = [0, \frac{4\pi}{9}, 0]$

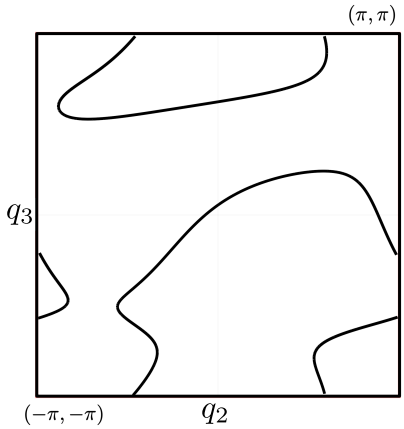


Figure S5: Example of a $(0,0)[2,6]$ branch, $\mathbf{d} = [1, 1, 1]$ $\mathbf{a} = [1.5, 1, 2]$
$\alpha = [\frac{\pi}{4}, \frac{4\pi}{9}, 0]$

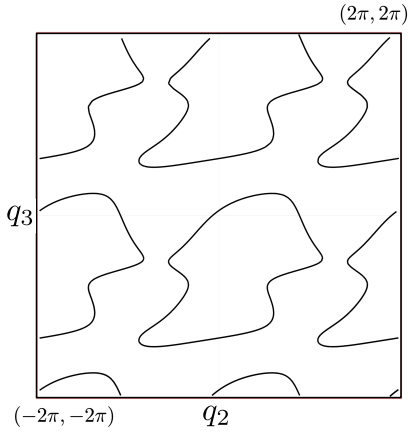


Figure S6: Scaled version of Figure S5.

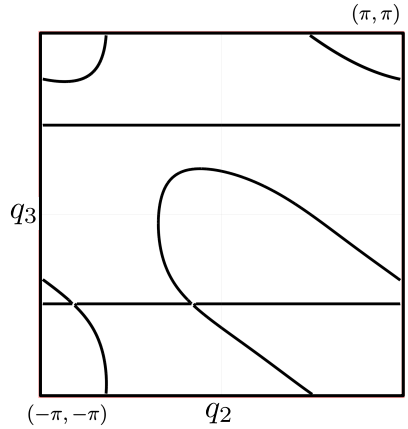


Figure S7: Robot class:
$2(1,0)[\infty, 0] + (0,0)[2,2]$,
$\mathbf{d} = [0, 1, 1]$ $\mathbf{a} = [1, 0, 3]$ $\alpha = [\frac{\pi}{2}, \frac{\pi}{4}, 0]$

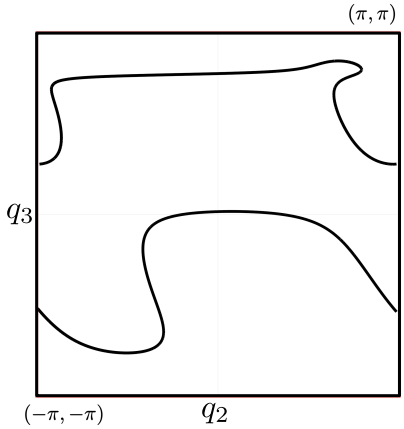


Figure S8: Robot class:
$(1,0)[2,4] + (1,0)[2,2]$, $\mathbf{d} = [0, 0.4, 1]$
$\mathbf{a} = [1, 1.2, 1.5]$ $\alpha = [\frac{\pi}{2}, \frac{\pi}{4}, 0]$

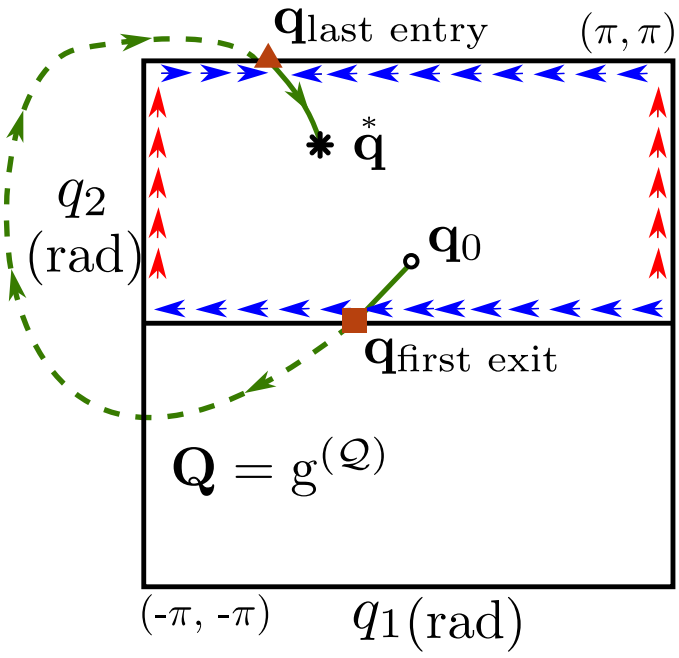


Figure S9: The track cycle in the joint space of a $2R$ robot, that determines the behaviour to reach $\mathbf{q}_{\text{last entry}}$ from any configuration in the boundary region of the relevant aspect.

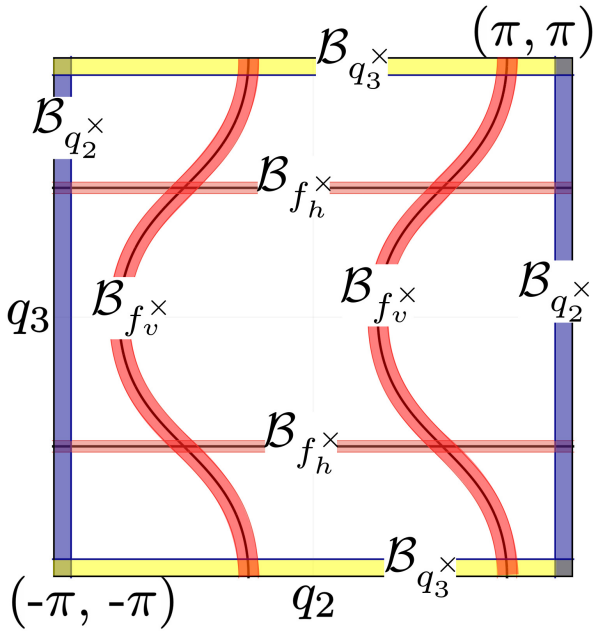


Figure S10: Singularity and joint limit boundaries of a $3R$ robot from Category III. Visualized in the q_2–q_3 slice of the joint space.

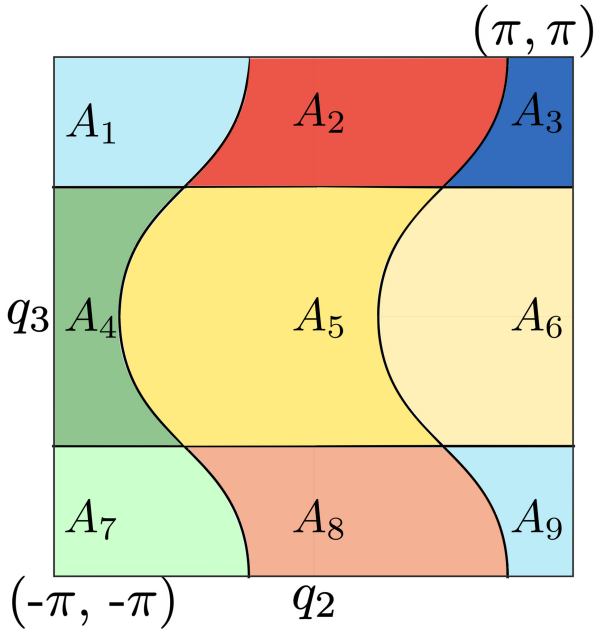


Figure S11: Colored aspect boundaries for the same $3R$ robot with class $2(1,0)[\infty, 0] + 2(0,1)[0,2]$ in the q_2–q_3 slice.

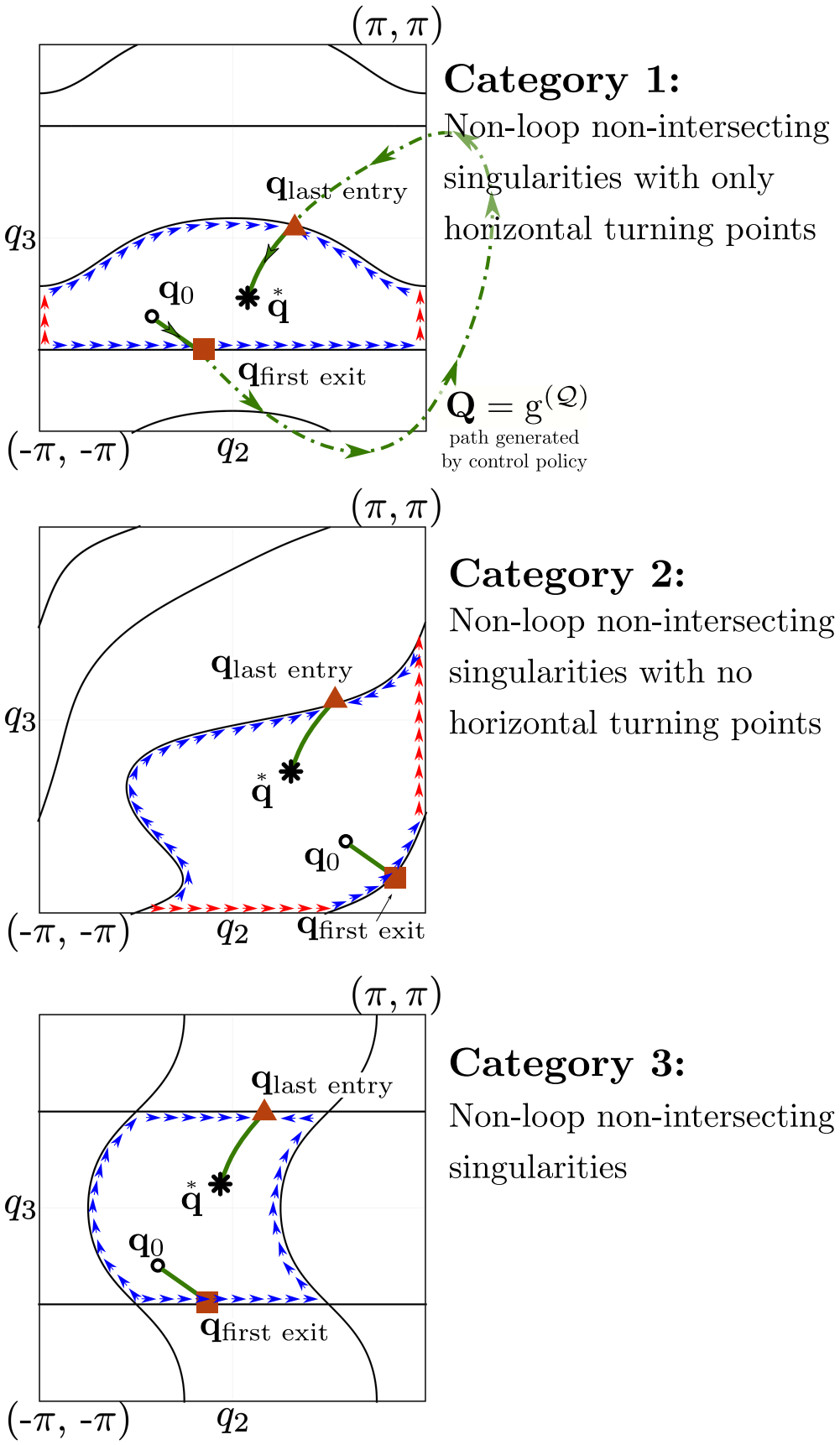


Figure S12: Track cycle for 3R robots from Category I through 3 (top to bottom) joint space, as discussed in Algorithm 8.

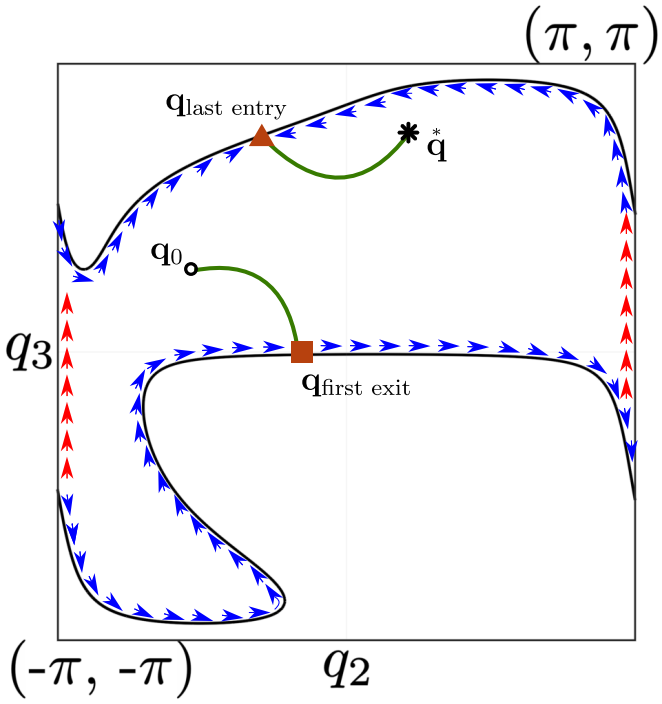


Figure S13: Track cycle for a $3R$ robot from Category VI, with singularity class $(1,0)[2,0] + (1,0)[2,2]$.

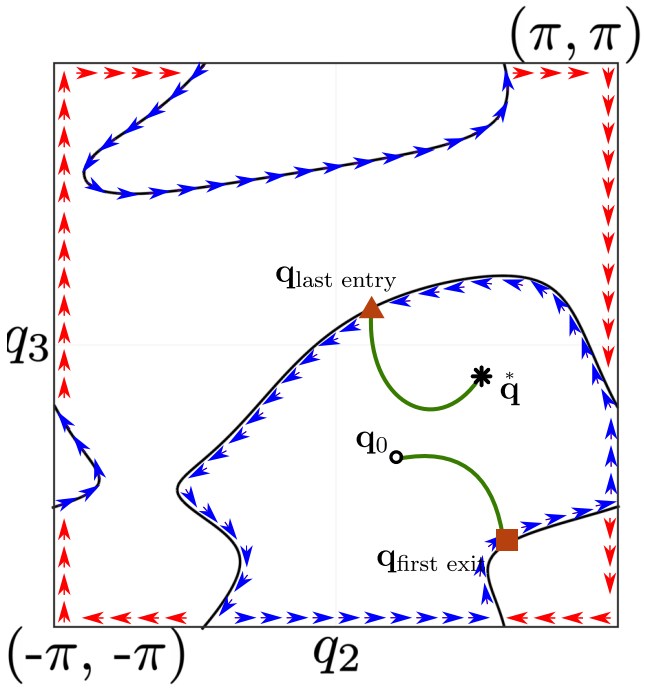


Figure S14: Track cycle in joint space for loop type singularities

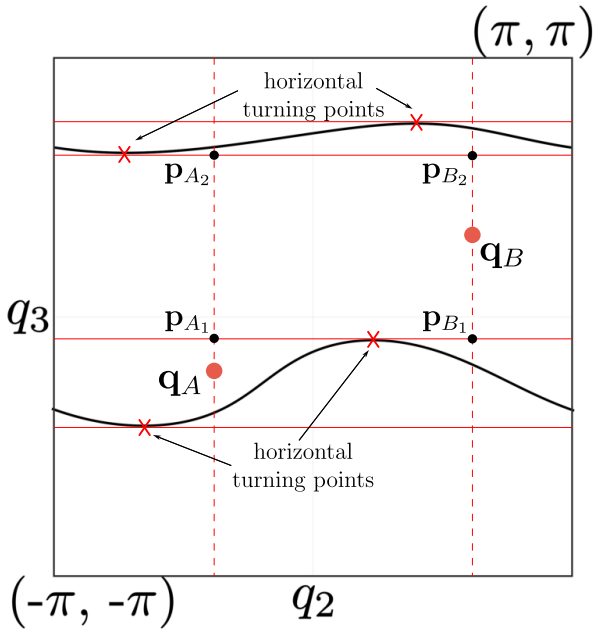


Figure S15: An illustration of the connectivity check for factor class $2(1,0)[2,0]$ in $q_2 - q_3$ slice for a robot from Category I

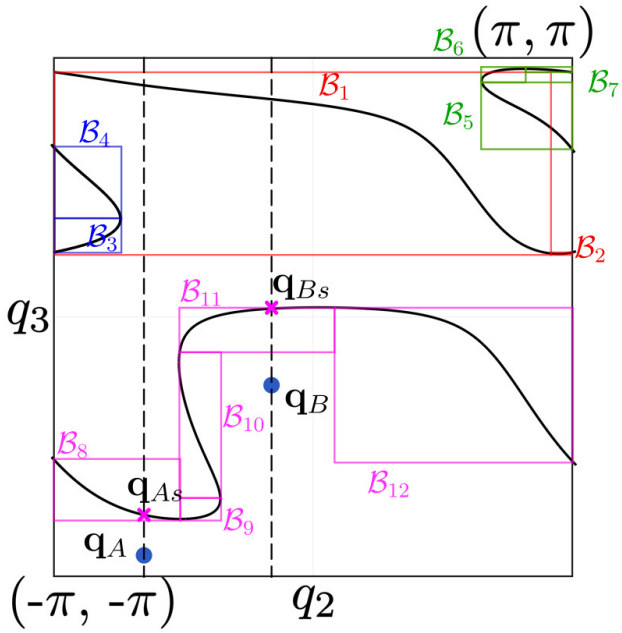


Figure S16: An illustration of the connectivity check for factor class $2(1,0)[2,2]$.

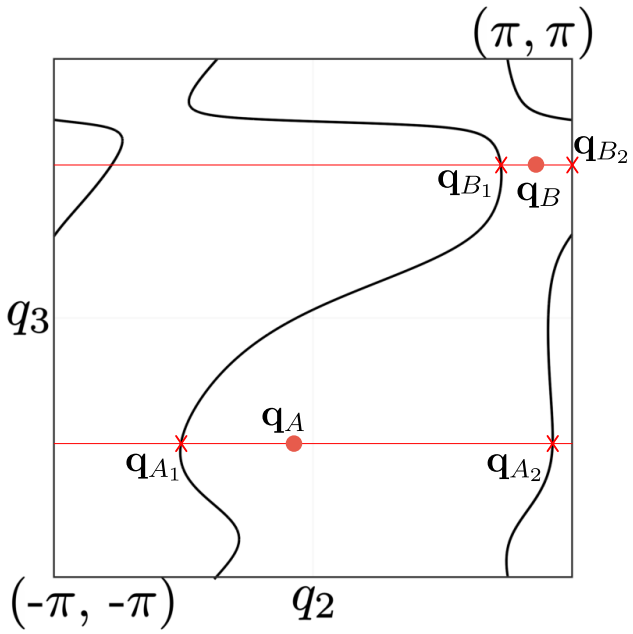


Figure S17: An illustration of the connectivity check for factor class $2(0,1)[0,*]$ in $q_2 - q_3$ slice for robot from Category II (B)

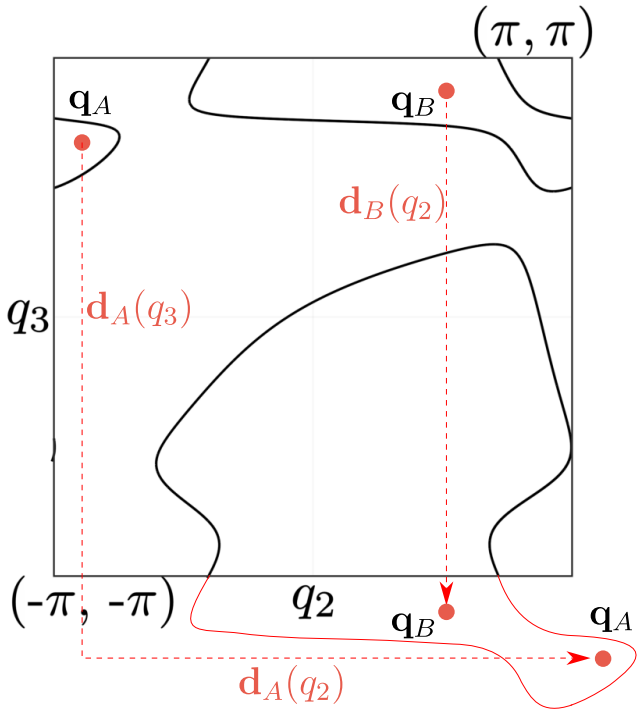


Figure S18: An illustration of the connectivity check for factor class $(0,0)[2,6]$ in $q_2 - q_3$ slice for robot from Category IV (C)

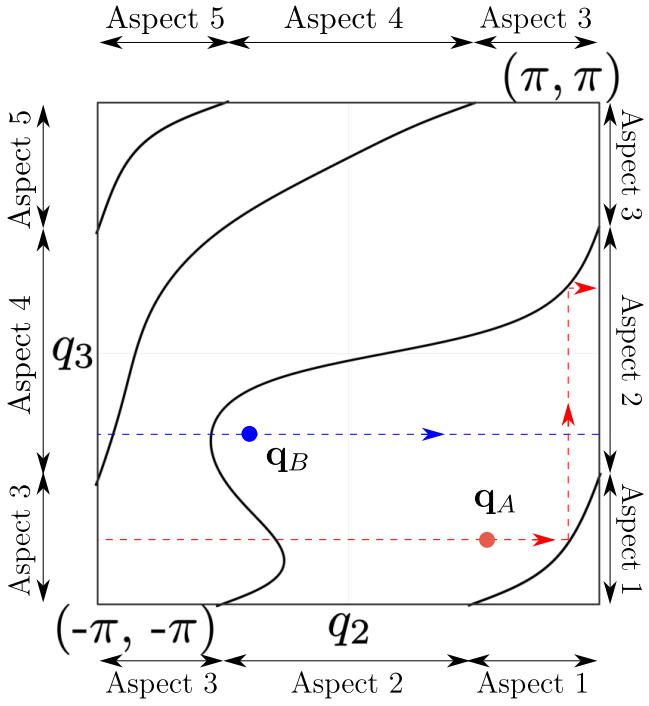
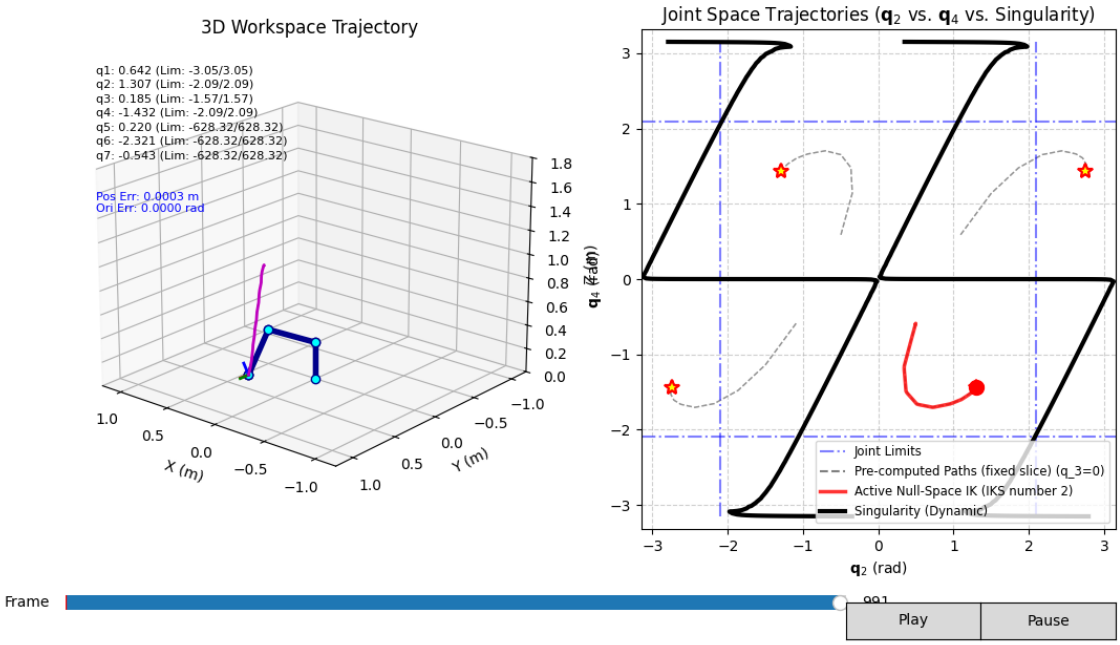
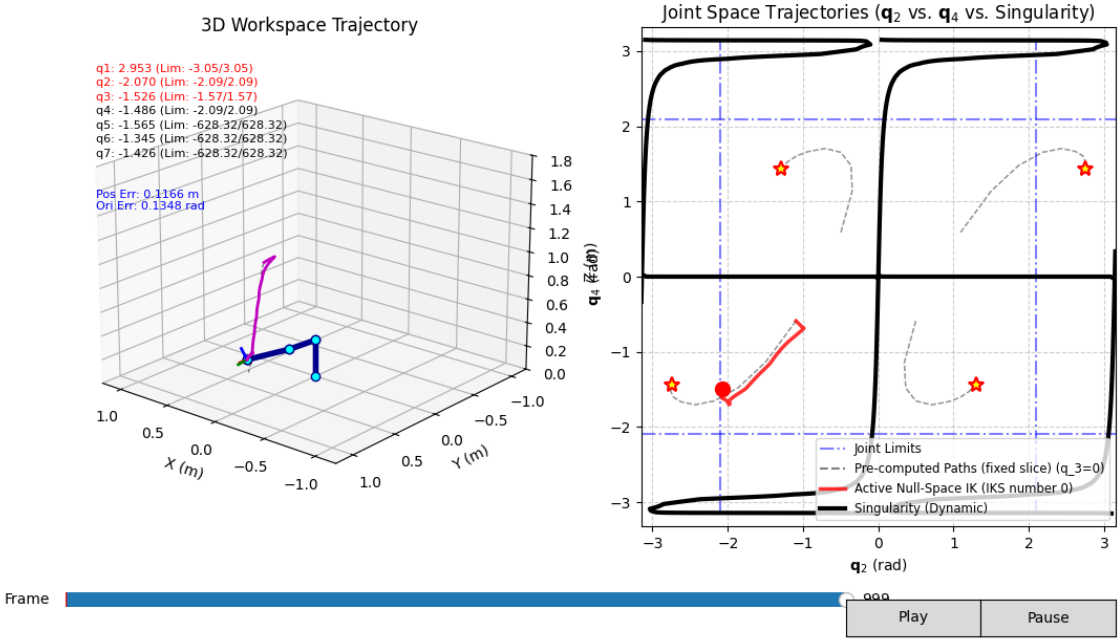


Figure S19: Connectivity check for factor class $(1,1)[0,*]$ for robot from Category II (E).

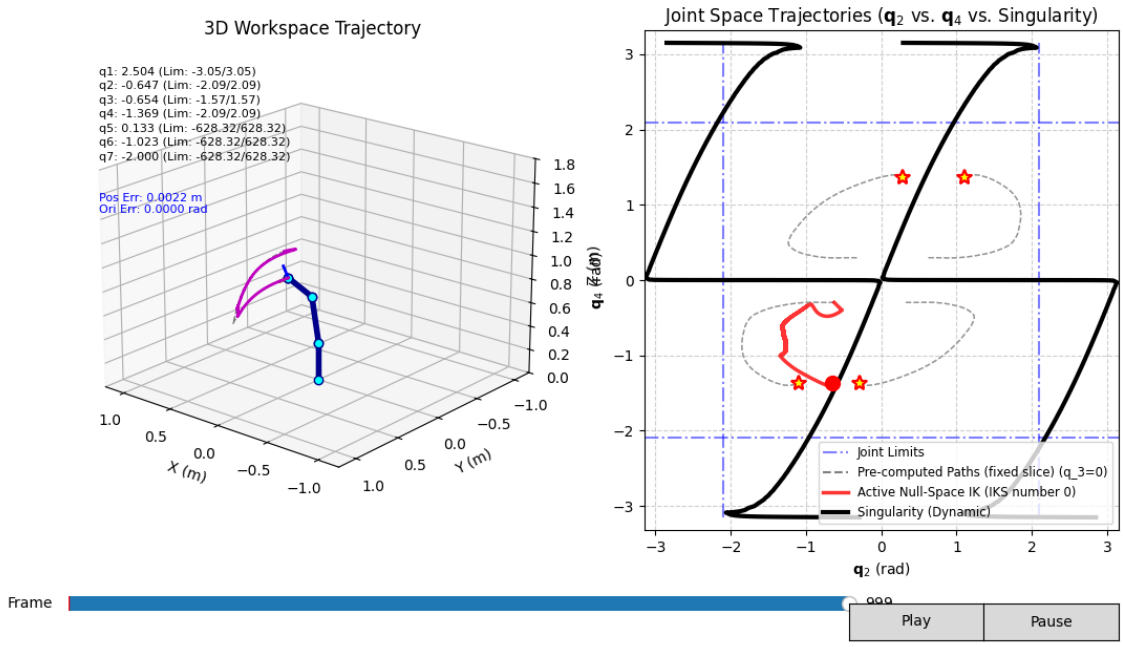


(a) Successful execution: the null-space strategy tracks the demonstrated path and the redundant angle q_R remains essentially constant and well inside joint limits.

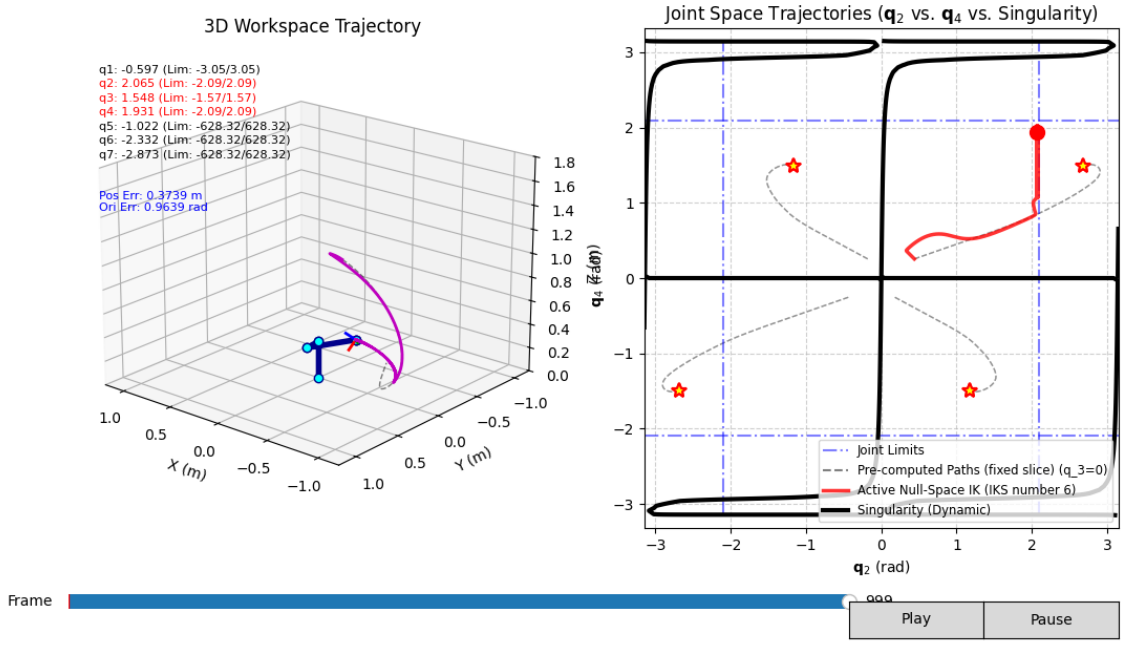


(b) Same demonstration, but the null-space evolution leads to end-of-path shaking due to posture oscillations in the arm angle q_3.

Figure S20: **Failure modes of numerical null-space strategies for redundant robots.** Small changes in initialization and secondary-task tuning can yield qualitatively different outcomes for the same demonstration: from clean execution (A), to undesirable oscillations (B), to trajectories that stay near singularities (C), or even clear task failure with multiple joints saturating (D).

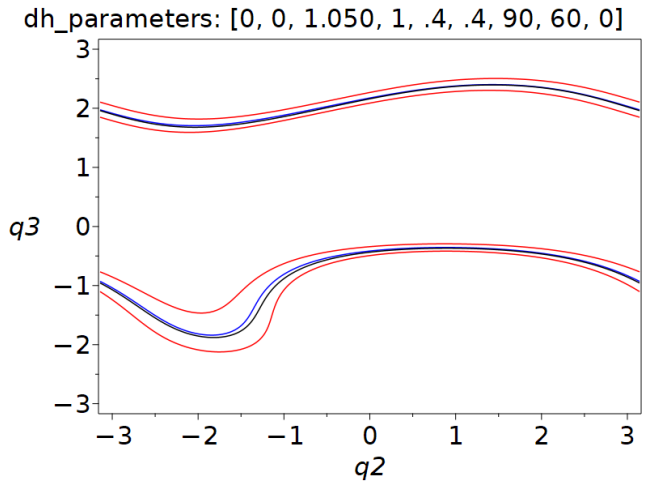


(c) Redundant posture selection yields a q_R that keeps the robot close to singularities along the entire path, although it completes the workspace task.

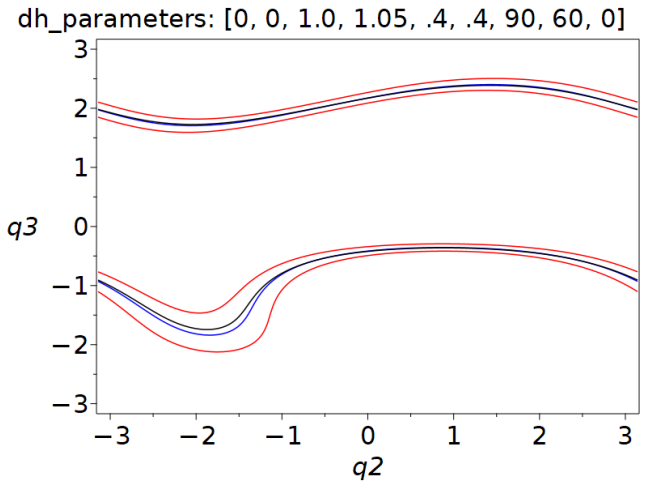


(d) Deviation from the intended behaviour: the null-space policy drives the arm into a configuration where two joint limits are reached, and the path departs from the demonstrated motion.

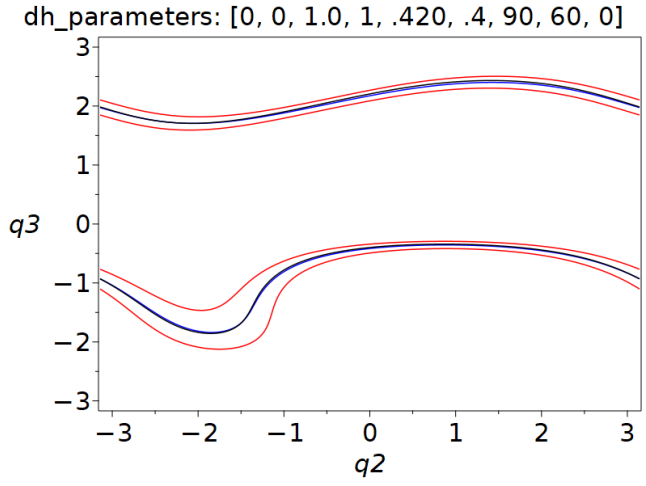
Figure S20: **Failure modes of numerical null-space strategies for redundant robots.** (continued)



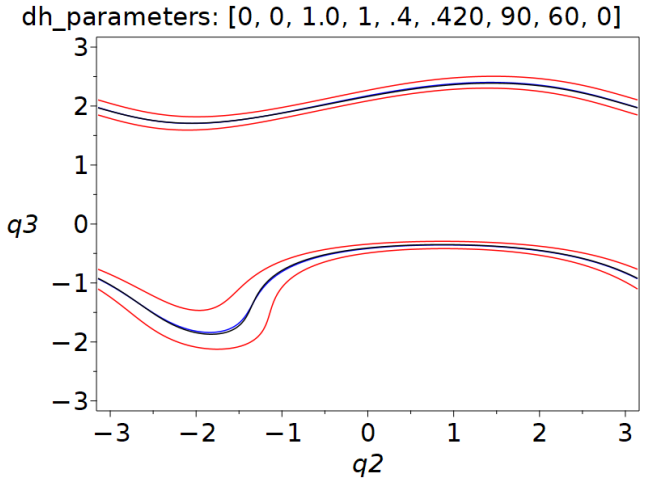
(a) Effect of 5% noise in d_3



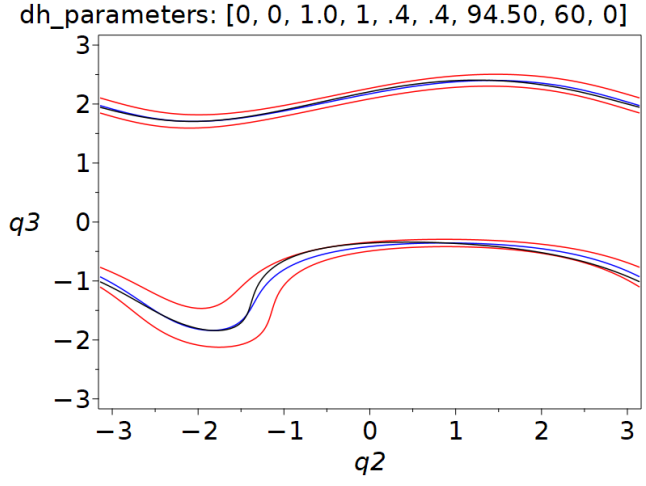
(b) Effect of 5% noise in a_1



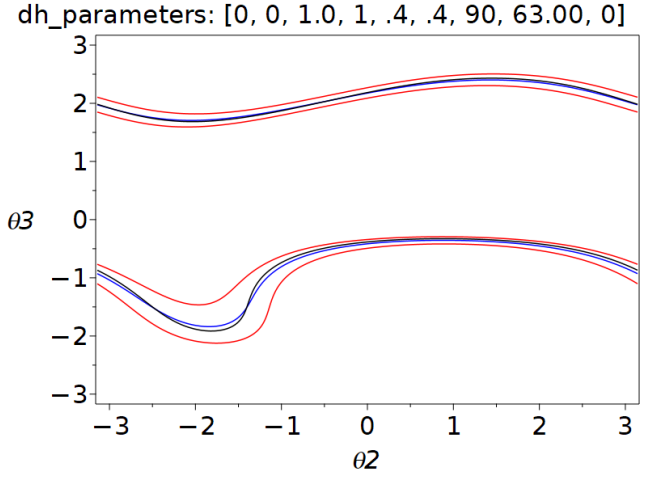
(c) Effect of 5% noise in a_2



(d) Effect of 5% noise in a_3

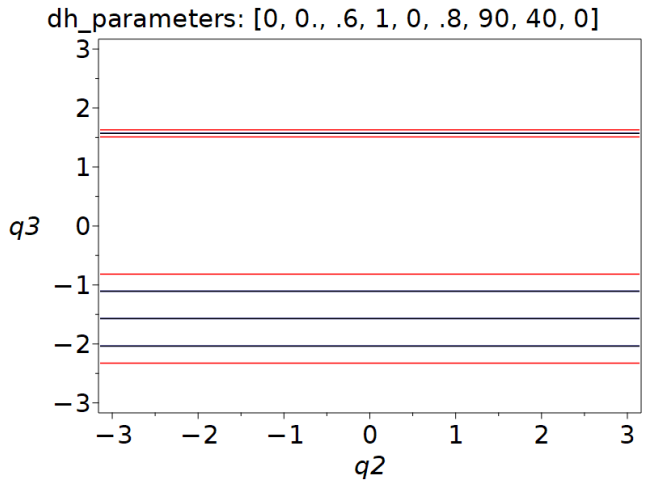


(e) Effect of 5% noise in α_1

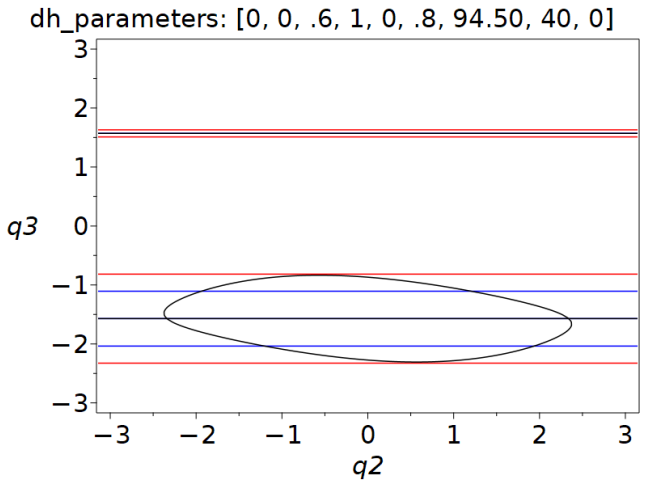


(f) Effect of 5% noise in α_2

Figure S21: The blue curves corresponds to original singularity; the red is a safety margin chosen at $\epsilon = 0.03$. The black curves are the singularity obtained by adding 5% noise in each D-H parameter separately. The first one is in d_3 and progressively in a_1, a_2, a_3, α_1, and α_2. The remaining D-H parameters do not affect singularity (and $d_2 = 0$ for this case). The D-H parameters of the base architecture is : $\mathbf{d} = [0, 0, 1]$, $\mathbf{a} = [1, 0.4, 0.4]$, $\alpha = [\pi/2, \pi/3, 0]$



(a) Effect of 5% noise in d_3



(b) Effect of 5% noise in $alpha_1$

Figure S22: Effect of kinematic parameter perturbations on the singularity layout. Left: a 5% perturbation in d_3 (black) produces no significant change with respect to the nominal singularity (blue), and remains well within the safety margin. Right: a 5% perturbation in α_1 alters the singularity curve more noticeably, but the perturbed singularity still lies entirely inside the padded band defined by $|\det \mathbf{J}| < 10^{-3}$ (red), so the controller continues to treat this region as forbidden.

$q_2 - q_3$ slice	Along q_1	$\dot{\mathbf{q}}(q_2, q_3)$	$\dot{\mathbf{q}}(q_1)$
No violation	No violation	$\mathrm{g}^{(\mathcal{Q})}(\mathbf{q})$	$\mathrm{g}^{(\mathcal{Q})}(\mathbf{q})$
No violation	Violation	$\mathrm{g}^{(\mathcal{B})}(\mathbf{q}_{\text{target}}, \mathbf{q})$ (Eq. S12)	0
Violation	No violation	$\mathrm{g}^{(B)}(\mathbf{q}, \mathbf{q}_{\text{last entry}})$ (Eq. S10)	$\mathrm{g}^{(B)}(\mathbf{q}, \mathbf{q}_{\text{last entry}})$ (Eq. S11)
Violation	Violation	$\mathrm{g}^{(\mathcal{B})}(\mathbf{q}, \mathbf{q}_{\text{last entry}})$ (Eq. S10)	$\mathrm{g}^{(\mathcal{B})}(\mathbf{q}, \mathbf{q}_{\text{last entry}})$ (Eq. S10)

Table S1: The resultant of the modelled behaviour policy $\mathrm{g}^{(\mathcal{Q})}$ and the near-boundary policy $\mathrm{g}^{(\mathcal{B})}$ in an aspect of the joint space of a $3R$ robot.

Case	**Homotopy class**	**Robot Examples** (Figure S2)	**Connectivity check**
(i)	$(1, 0), Q_v = 0$	(A) Category I; (F), (H) Category III; Category V	Algorithm 14
(ii)	$(0, 1)$	(B) Category II; (F) Category III	Algorithm 16
(iii)	$(0, 0)$	(C) Category IV; (G) Category V	Algorithm 17
(iv)	$(1, 0), Q_v > 0$	(D) Category VI	Algorithm 18
(v)	$(1, 1)$	(E) Category II; (H) Category III	Algorithm 19

Table S2: Cases based on the types of factors in expression of Jacobian determinant $\det \mathbf{J}(\mathbf{q})$, homotopy class of branches in the factor, number of branches in the factor, corresponding robot examples from Figure S2, and the connectivity check algorithm specific to the case. Q_v is the set of vertical turning points in the factor

Robot Category	Connectivity	Track cycle
Category I (A)	Algorithm 14	Algorithm 8
Category II (B)	Algorithm 16	Algorithm 8
Category V (C)	Algorithm 17	Algorithm 11
Category IV (D)	Algorithm 18	Algorithm 10
Category II (E)	Algorithm 19	Algorithm 8
Category III (F)	Algorithm 14 + 16	Algorithm 8
Category VI (G)	Algorithm 14 + 17	Algorithm 11 + 8
Category III (H)	Algorithm 14 + 19	Algorithm 8

Table S3: Different types of robots and their respective connectivity analysis algorithms and motion near boundaries.

Caption for Movie S1 Different types of singularities in noncuspidal $3R$ robots The joint space singularities for multiple cases of noncuspidal robots with their respective DH parameters are presented.

Caption for Movie S2 Transferred behaviour on KUKA IIWA The demonstrated characters from SCIENCE are used to learn control policies for the commercially available anthropomorphic robot.

Caption for Movie S3 Transferred behaviour on MAiRA robot from Neura Robotics The demonstrated characters from SCIENCE are used to learn control policies for the commercially available anthropomorphic robot.

Caption for Movie S4 Experimental setup and cross-platform skill transfer The video presents the physical experimental setup used with three distinct robotic manipulators and illustrates how a single learned DS-based control policy, obtained from a reference demonstration, is executed across them. It highlights how the same task is reproduced across platforms while respecting each robot's kinematic constraints and feasible workspaces.

Caption for Movie S5 Overview of behaviour transfer to anthropomorphic robots This video shows the challenges in transfer of demonstrated behaviour, briefly introduces the redundancy parametrisation in $7R$ robots. Furthermore, few kinematic properties for $3R$ robots are next illustrated. Lastly, the resulting behaviour on different commercial robots is presented.

Caption for Movie S6 Learning control policies from multiple demonstrations The video presents three cases for a $2R$ robot with varying number of user demonstrations to learn the behaviour from.